Федеральное государственное бюджетное учреждение науки «Институт проблем передачи информации им. А.А. Харкевича Российской академии наук»

На правах рукописи

Чеканов Михаил Олегович

Алгоритмы комплексирования разнородных данных при построении модели окружающей сцены беспилотного транспортного средства

Специальность: 2.3.8. Информатика и информационные процессы

ДИССЕРТАЦИЯ
на соискание ученой степени
кандидата технических наук

Научный руководитель:
Шипитько Олег Сергеевич — кандидат технических наук

Москва — 2026

Оглавление

Стр.

Введение 6

Глава 1. Модели окружающего пространства БТС и алгоритмы их построения на основе комплексирования данных разной природы 14

1.1 Современное состояние беспилотных транспортных средств 14

1.2 Используемые в колёсной робототехнике датчики и их ограничения 17

1.3 Модели окружающего пространства БТС 20

1.4 Алгоритмы комплексирования данных 23

1.5 Метрики для оценки качества алгоритмов построения моделей окружающего пространства 26

1.5.1 Метрики для оценки объектно-ориентированных представлений 26

1.5.2 Метрики для оценки сеточных представлений 28

1.5.3 Метрики для оценки непрерывных представлений 32

1.6 Выбор модели окружающего пространства и метрик её оценки . . . 32

1.6.1 Выбор метрик, используемых в работе 33

1.7 Постановка частных задач исследования 34

1.7.1 Алгоритмы проецирования данных камеры на плоскость движения БТС 34

1.7.2 Влияние числа источников данных на производительность алгоритмов построения карты проходимости пространства . 38

1.8 Требования к разрабатываемым модели окружающего пространства и алгоритмам её построения 43

1.9 Заключение к главе 1 44

Глава 2. Алгоритмы проецирования визуальных данных на плоскость движения БТС 45

2.1 Алгоритм проекции визуальных детекций на плоскость движения БТС на основе модели L-Shape 45

Стр.

2.1.1 Этап 1. Проекция двумерной детекции изображения на вид сверху . . . 46
2.1.2 Этап 2. Вписывание проекции объекта на виде сверху в ориентированную ограничивающую рамку . . . 48
2.2 Исследование точности алгоритма проекции визуальных детекций на основе модели L-Shape . . . 50
2.2.1 Сравниваемые алгоритмы . . . 50
2.2.2 Набор данных и методика эксперимента . . . 52
2.2.3 Результаты экспериментов . . . 53
2.3 Алгоритм проекции визуальных детекций на плоскость движения БТС на основе комплексирования визуальных и лидарных данных . 54
2.3.1 Разделение облака точек на точки земли и препятствий . . . 57
2.4 Исследование точности алгоритма проекции визуальных детекций на основе комплексирования визуальных и лидарных данных . . . 61
2.4.1 Описание эксперимента . . . 63
2.4.2 Результаты экспериментов . . . 64
2.4.3 Вычислительная эффективность . . . 65
2.5 Алгоритм проекции сегментации проезжей части на изображении на плоскость движения БТС . . . 66
2.5.1 Вычисление функции $z(x,y)$ . . . 70
2.5.2 Комплексирование полученной проекции с базовой картой проходимости пространства . . . 71
2.6 Исследование точности алгоритма проекции сегментации проезжей части на основе комплексирования визуальных и лидарных данных . . . 74
2.6.1 Тестовый набор данных . . . 74
2.6.2 Сравниваемые алгоритмы и метрики . . . 75
2.6.3 Результаты экспериментов . . . 77
2.6.4 Вычислительная эффективность . . . 79
2.7 Заключение к главе 2 . . . 80

Стр.

Глава 3. Программный комплекс построения карты проходимости пространства на основе комплексирования разнородных данных 82
3.1 Общие сведения 82
3.2 Назначение и функциональные возможности 83
3.3 Структура программного комплекса 84
3.3.1 Библиотека «occupancy_map_fusion» 88
3.3.2 Программа «evo_occupancy_map_fusion» 98
3.4 Заключение к главе 3 99

Глава 4. Параллельный алгоритм построения карты проходимости пространства на основе комплексирования разнородных данных 101
4.1 Производительность алгоритма построения карты проходимости пространства 101
4.2 Постановка задачи построения карты проходимости пространства . 103
4.3 Модификации базовой реализации 108
4.3.1 Базовая реализация 108
4.3.2 Расширенная карта проходимости пространства 109
4.3.3 Карта с разделяемыми блокировками 110
4.3.4 Карта проходимости пространства с полным обновлением без блокировок 113
4.4 Сравнение задержки обновления локальной карты проходимости пространства 116
4.5 Реализация финальной модификации для двумерной карты проходимости пространства 122
4.6 Заключение к главе 4 123

Заключение 125

Список сокращений и условных обозначений 128

Список рисунков 143

Список таблиц 146

Стр.

Приложение А. **Вывод решения задачи вписывания точек в ориентированную ограничивающую рамку с использованием L-Shape модели** 147

Приложение Б. **Свидетельства о государственной регистрации программ для ЭВМ** 149

Приложение В. **Патенты** 152

Приложение Г. **Акты о внедрении результатов диссертационного исследования** 153

Введение

Последние десятилетия характеризуются активным внедрением и распространением беспилотных транспортных средств (БТС) в различных сферах человеческой деятельности. Примеры таких сфер – логистика, промышленность, сельское хозяйство, горное дело, городская инфраструктура. Эксплуатация БТС четвёртого уровня автоматизации по классификации SAE J3016 вышла за пределы испытательных полигонов, и с расширением области эксплуатации ужесточаются требования к безопасности их движения. Развитие беспилотного транспорта играет важную роль и в достижении национальных целей и задач развития Российской Федерации. Так, «Транспортные технологии для различных сфер применения (море, земля, воздух), в том числе беспилотные и автономные системы» включены в раздел «Критические технологии» перечня важнейших наукоёмких технологий, утверждённого Указом Президента Российской Федерации от 18 июня 2024 г. № 529. Транспортная стратегия Российской Федерации до 2030 года с прогнозом на период до 2035 года предусматривает организацию движения БТС на автомобильных дорогах общего пользования. Перечнем инициатив социально-экономического развития Российской Федерации до 2030 года утверждена инициатива «Беспилотные логистические коридоры», предусматривающая проведение мероприятий по внедрению на автомобильных дорогах Государственной компании интеллектуальных транспортных систем, ориентированных в том числе на обеспечение движения высокоавтоматизированных транспортных средств.

Основой безопасного движения БТС служит подсистема восприятия, строящая по данным бортовых датчиков модель окружающей сцены. Под окружающей сценой здесь и далее понимается пространство, окружающее БТС, вместе с находящимися в нём объектами, а строящееся её описание для краткости именуется моделью окружающего пространства. К такой модели предъявляются два встречных требования. С одной стороны, она должна быть достаточно точной, чтобы обеспечить решение задач использующими её алгоритмами. С другой – лишённой избыточной детализации: чем сложнее модель, тем больше времени требует работа с ней, а минимизация задержек в системах автопилотирования критически важна, поскольку любая задержка означает, что решение принимается по устаревшему описанию окружения. Поэтому состав сведений, хранимых в модели,

определяется тем, какие задачи по ней решаются. В частности, алгоритмам планирования пути требуется модель, описывающая расположение зон, движение в которых возможно, невозможно либо о проходимости которых сведения отсутствуют.

Точность описания каждого из этих классов зон одинаково существенна, а ошибки модели опасны в обе стороны: занижение занимаемой препятствием области ведёт к столкновению, завышение – к ложным остановкам и снижению транспортной работы, а области, ошибочно отнесённые к свободным при фактическом отсутствии наблюдений, лишают алгоритмы планирования возможности действовать консервативно в условиях окклюзии. При этом ни один тип бортовых датчиков не позволяет построить такую модель в одиночку: каждый ограничен собственным принципом работы – угловым разрешением, дальностью, чувствительностью к погодным условиям либо отсутствием прямого измерения расстояния. Поэтому современные БТС оснащаются наборами разнородных датчиков, а качество модели определяется тем, каким образом их показания комплексируются в единое непротиворечивое описание окружения.

Задача построения модели окружающего пространства, в том числе и модели проходимости окружающего пространства, в робототехнике широко исследована, однако её решение в условиях ограниченных вычислительных ресурсов остаётся предметом исследований. Исследование этой задачи нашло отражение в работах М. Павоне, А. Элфеса, Х. Моравека, С. Труна, Б.М. Миллера, Д.А. Юдина и Л.М.Г. Гонсалвеса. Среди исследований задачи комплексирования данных стоит выделить работы Б. Халеги, Д.П. Николаева, Ю.В. Визильтера.

Наибольший объём сведений об окружающей БТС сцене даёт камера: по угловому разрешению она превосходит остальные бортовые датчики и позволяет получить семантическое описание наблюдаемого – границы проезжей части, классы объектов, дорожную разметку. Однако изображение задаёт лишь направление на объект и не содержит расстояния до него. Поэтому учёт визуальных данных в модели окружающего пространства требует отдельного этапа – проецирования результатов обработки изображения, детекций объектов и семантической сегментации, из системы координат изображения на плоскость движения БТС. От точности этого проецирования напрямую зависит корректность положения препятствий в модели.

Существующие способы такого проецирования тяготеют к двум крайностям: высокой точности либо низкой вычислительной стоимости. На одном краю

– методы, опирающиеся на известные калибровки камеры и геометрическое предположение о форме поверхности движения. Распространённое допущение о совпадении этой поверхности с плоскостью нарушается как рельефом местности, так и требованиями к конструкции дорожного покрытия. К этому добавляется отклонение фактической внешней калибровки от измеренной, вызванное загрузкой БТС, работой подвески и смещением датчиков при длительной эксплуатации. Вносимая обоими факторами погрешность растёт с удалением от камеры и на тех дальностях, на которых принимается решение об объезде или остановке, составляет единицы метров. На другом краю – обучаемые преобразования признаков изображения в вид сверху, не использующие явной модели поверхности. Методы этой группы демонстрируют наивысшее качество проецирования, однако требуют производительного графического ускорителя для запуска нейросетевой модели, принимающей данные от каждого источника данных; размеченных мультимодальных наборов данных для обучения; а также переобучения при любом изменении состава и расположения датчиков, что для мелкосерийных платформ с уникальной сенсорной конфигурацией практически неприменимо. Предлагаемые в настоящей работе алгоритмы проецирования визуальных данных занимают промежуточное положение между этими крайностями: предположение о совпадении поверхности движения с плоскостью заменяется оценкой её формы в существенно более широком классе поверхностей, но без привлечения обучаемых преобразований в вид сверху.

Второе ограничение – вычислительная стоимость добавления дополнительных источников данных при учёте данных в общей модели. Увеличение числа источников данных, которое наблюдается на существующих БТС – естественный способ уменьшить размер слепых зон БТС и повысить достоверность модели, однако показания каждого из датчиков должны быть сведены в общую модель, а эта операция выполняется над разделяемым ресурсом и потому в существующих реализациях сериализуется. В результате задержка между регистрацией данных датчиком и их отражением в модели растёт с числом источников. Положение усугубляется тем, что для широкораспространённых моделей, описывающих пространство в виде двумерного массива – сеточных представлений, требуемый размер массива определяется тормозным путём и растёт как четвёртая степень рабочей скорости БТС, вследствие чего предельное число одновременно учитываемых источников с ростом скорости сокращается столь же быстро. Известные способы распараллеливания построения сеточных моделей применяются на дру-

гих уровнях – внутри одного обновления карты на графическом ускорителе либо между независимыми слоями, сведение которых в общую модель всё равно выполняется последовательно, – а неблокирующие структуры данных к разделяемой сетке локальной карты, следующей за БТС и потому требующей операций сдвига и приведения всех элементов к общему моменту времени, не применялись.

Таким образом, качество и актуальность модели, описывающей проходимость окружающего БТС пространства, ограничены двумя факторами: точностью проецирования визуальных данных на плоскость движения без опоры на предположение о плоской поверхности и в пределах вычислительных возможностей бортового вычислителя, а также ростом задержки учёта данных при увеличении числа источников. Оба ограничения имеют общий корень: вычислительная стоимость известных решений растёт с числом учитываемых источников данных, тогда как ресурсы бортового вычислителя разделяются между всеми подсистемами БТС. Разработка вычислительно эффективных алгоритмов комплексирования разнородных данных, а также структуры данных и алгоритма её обновления, допускающих одновременный учёт показаний нескольких источников, представляет собой актуальную научно-техническую задачу.

Целью работы является снижение задержки построения и повышение точности модели проходимости окружающего пространства БТС при одновременном учёте данных нескольких разнородных источников в условиях ограниченных вычислительных ресурсов бортового вычислителя.

Для достижения поставленной цели необходимо было решить следующие **задачи**:

1. На основе анализа моделей окружающего пространства БТС, алгоритмов их построения и способов комплексирования разнородных данных обосновать выбор модели, описывающей проходимость пространства, алгоритма её построения и метрик оценки её качества, а также сформулировать требования к разрабатываемым модели и алгоритмам её построения.
2. Разработать алгоритмы оценки положения объектов в окружающем пространстве БТС, а также областей проезжей части, доступных для движения БТС, на основе визуальных и лидарных данных, предназначенные для работы на бортовом вычислителе БТС в режиме реального времени.

3. Разработать программный комплекс, обеспечивающий построение модели окружающего пространства БТС на основе комплексирования разнородных данных с бортовых датчиков БТС.
4. Разработать структуру данных для представления модели окружающего пространства и параллельный алгоритм её обновления несколькими источниками данных.

Научная новизна:

1. Предложен алгоритм оценки положения окружающих ТС на плоскости движения БТС по данным монокулярной камеры, отличающийся от известных алгоритмов вписывания в ориентированную ограничивающую рамку тем, что в его основу положена модель L-образной формы видимой границы ТС, ранее применявшаяся лишь к кластерам точек дальномерных датчиков.
2. Предложен алгоритм оценки положения окружающих ТС на плоскости движения БТС по комплексированию визуальных и лидарных данных, отличающийся от известных тем, что сегментация лидарного облака на точки земли и точки препятствий выполняется без предположения о плоской либо кусочно-плоской поверхности движения – на основе оценки высоты поверхности регрессией на гауссовских процессах, – что повышает точность оценки положения во всём диапазоне рассмотренных дальностей.
3. Предложен алгоритм проекции семантической сегментации проезжей части с изображения на плоскость движения БТС, отличающийся от известных тем, что проецирование выполняется не на фиксированную плоскость, а на оценённую по лидарным данным поверхность движения в проецируемой области, что снижает ошибку описания свободного пространства на строящейся карте проходимости.
4. Впервые количественно показано, что для класса алгоритмов построения карты проходимости пространства, не использующих обучаемых преобразований в вид сверху и применимых на бортовом вычислителе БТС, учёт визуальных данных с камер повышает качество карты по сравнению с картой, строящейся исключительно по лидарным данным, причём улучшение проявляется в том числе по метрике, учитывающей задачу планирования пути (PFC-MSE).

5. Предложены структура данных для представления карты проходимости пространства и алгоритм её обновления, отличающиеся от известных реализаций тем, что операции над картой выполняются без использования примитивов взаимного исключения, что позволяет обновлять карту нескольким источникам данных одновременно и снижает время задержки между моментом регистрации данных датчиком и моментом обновления карты.

Практическая значимость. Разработанные алгоритмы и программно-аппаратный комплекс, обеспечивающие построение модели окружающего пространства БТС и реализованные в данной работе, внедрены в беспилотные транспортные средства «ВАТС ЭВО 1.1» и «ВАТС ЭВО-1.2», разработанные компанией ООО «ЭвоКарго». Внедрение разработанных в данной диссертации методов подтверждено соответствующими актами. На результаты, полученные в работе, получены свидетельства о государственной регистрации программ для ЭВМ:

1. № 2025617212 Программное обеспечение «Система восприятия N1 V3».
2. № 2025669744 Программное обеспечение «Способ построения карты проходимости в окрестности высокоавтоматизированного беспилотного грузового транспортного средства».

На основные результаты работы получен патент № 2853062 «Способ построения карты проходимости в окрестности высокоавтоматизированного беспилотного грузового транспортного средства».

Методология и методы исследования. В диссертации используются методы вероятностной робототехники, методы байесовской оценки, вычислительной оптимизации, компьютерного зрения, анализ вычислительной сложности, а также численный эксперимент.

Основные положения, выносимые на защиту:

1. Алгоритм оценки положения окружающих ТС на основе комплексирования визуальных и лидарных данных, выделяющий точки поверхности движения оценкой её формы регрессией на гауссовских процессах вместо предположения о плоскости, даёт наибольший средний коэффициент Жаккара среди алгоритмов, не использующих обучаемых преобразований в вид сверху, – $0{,}439$ против $0{,}399$ у лучшего лидарного и $0{,}337$ у визуального – и сохраняет превосходство во всём диапазоне рассмот-

ренных дальностей. Среднее время обработки кадра $66{,}7$ мс, что меньше периода регистрации данных 100 мс.

2. Алгоритм проецирования семантической сегментации проезжей части с изображения на оценённую по лидарным данным поверхность движения в сочетании с базовым лидарным алгоритмом повышает качество карты проходимости пространства по сравнению с картой, строящейся исключительно по лидарным данным: среднеквадратичная ошибка стоимости поиска пути (PathFinding Cost Mean Squared Error, PFC-MSE) снижается с $1019{,}3$ до $924{,}4$ в пяти рассмотренных сценариях, а среднеквадратичная ошибка относительно эталонной карты – с $0{,}231$ до $0{,}218$. Проецирование той же сегментации на фиксированную плоскость увеличивает PFC-MSE до $1059{,}4$.
3. Структура данных, представляющая локальную карту проходимости пространства набором массивов атомарных пар «логарифм шансов – время», делает операции добавления фрагмента, сдвига и получения карты взаимно неблокирующими и уменьшает коэффициент линейной аппроксимации времени итерации цикла публикации карты восемью источниками в $13{,}5$ раза по сравнению с исходной реализацией.
4. Архитектура программного комплекса построения карты проходимости пространства, в которой источник данных представлен поставщиком фрагмента карты, оставляет последовательным только сведение фрагментов в общую карту и сводит подключение датчика нового типа к реализации единственного преобразования его данных во фрагмент. При восьми источниках обновление карты занимает до $22{,}8$ мс, что меньше периода регистрации данных 100 мс.

Достоверность полученных результатов обеспечивается использованием строгого математического аппарата и результатами вычислительных и натурных экспериментов, а также внедрением разработанных методов в БТС, находящиеся в эксплуатации.

Апробация работы. Основные результаты работы докладывались на международных конференциях:

– 16th International Conference on Machine Vision (ICMV 2023, Ереван, Армения);
– 40th ECMS International Conference on Modelling and Simulation (ECMS 2026, Гримстад, Норвегия).

Личный вклад. Все результаты диссертации, вынесенные на защиту, получены автором самостоятельно. Постановка задач и обсуждение результатов проводились совместно с научным руководителем.

Публикации. Основные результаты по теме диссертации изложены в 5 печатных изданиях, 3 из которых изданы в журналах, рекомендованных ВАК, 2 — в изданиях, индексируемых Web of Science и Scopus. Зарегистрированы 1 патент и 2 программы для ЭВМ.

Объём и структура работы. Диссертация состоит из введения, 4 глав, заключения и 4 приложений. Полный объём диссертации составляет 153 страницы, включая 36 рисунков и 10 таблиц. Список литературы содержит 130 наименований.

Глава 1. Модели окружающего пространства БТС и алгоритмы их построения на основе комплексирования данных разной природы

В данной главе проводится анализ моделей окружающего пространства БТС; датчиков, по измерениям которых выполняется их построение; алгоритмов построения и обновления моделей; способов комплексирования разнородных данных, а также метрик оценки качества моделей. На основании проведённого анализа обосновывается выбор модели, описывающей проходимость окружающего БТС пространства, алгоритма её построения и метрик оценки её качества, которые будут использованы в следующих главах работы. Также формулируются требования к разрабатываемым модели и алгоритмам её построения.

1.1 Современное состояние беспилотных транспортных средств

Развитие беспилотных транспортных средств является одним из наиболее активных направлений интеллектуальных роботизированных систем. Разрабатываемые решения в этой области значительно отличаются по функционалу, а также условиям, в которых они применяются. Для классификации автономного транспорта Обществом автомобильных инженеров (англ. Society of Automotive Engineers, SAE) в 2014 году были предложены шесть уровней автоматизации (SAE J3016) [1].

1. **Уровень 0. Отсутствие автоматизации (No Automation)**
   На данном уровне водитель полностью контролирует движение транспортного средства. Рулевое управление, ускорение/торможение и выполнение манёвров осуществляются исключительно водителем. Автоматические системы, доступные на транспортных средствах данного уровня, осуществляют только информационные функции либо краткосрочно берут на себя вспомогательные функции управления транспортным средством. Примерами систем такого уровня являются системы предупреждения о сходе с полосы, СПСП (Lane Departure Warning, LDW) [2], системы обнаружения объектов в слепых зонах транспортного средства (Blind Spot Monitoring, BSM) [3], системы ав-

томатического экстренного торможения, САЭТ (Autonomous Emergency Braking, AEB) [4] и др.

2. **Уровень 1. Вспомогательные системы (Driver Assistance)**
   Автоматические системы автомобиля на данном уровне могут выполнять одну из основных функций управления транспортным средством в определённых условиях, например ускорение или торможение или рулевое управление. При этом ответственность за безопасность остаётся на водителе, который обязан контролировать окружающую ситуацию и быть готовым взять управление транспортным средством на себя. Типичные примеры систем этого уровня – адаптивный круиз-контроль (Adaptive Cruise Control, ACC) [5] – система, позволяющая поддерживать фиксированные скорость и дистанцию позади идущего впереди транспортного средства, а также система удержания полосы движения, СУПД (Lane Keeping System, LKS) [6].
3. **Уровень 2. Частичная автоматизация (Partial Automation)**
   На данном уровне системы транспортного средства способны одновременно выполнять и рулевое управление, и торможение/ускорение. Однако, как и на предшествующих уровнях, водитель несёт полную ответственность за безопасность движения транспортного средства. К системам данного уровня относят системы удержания полосы, которые, в отличие от аналогичных систем первого уровня, контролируют не только рулевое управление, но и способны корректировать скорость транспортного средства. Также к этому уровню принадлежат системы автоматической парковки (Automatic Parking) [7].
4. **Уровень 3. Условная автоматизация (Conditional Automation)**
   Автоматические системы автомобиля на этом уровне способны полностью осуществлять управление транспортным средством без участия водителя в ограниченных условиях: как правило, это определённый тип дороги (автомагистрали с отчётливо распознаваемой разметкой) и хорошие погодные условия. В случае возникновения нестандартной ситуации на дороге система должна своевременно передать управление транспортным средством водителю. Соответственно, в отличие от предшествующих уровней, от водителя не требуется постоянное слежение за окружающей обстановкой, но он должен быть готов к управлению транс-

портным средством в нештатных ситуациях. Пример такой системы – ассистент движения в пробках (Traffic Jam Assist) [8].

5. **Уровень 4. Высокая автоматизация (High Automation)**
   Аналогично предыдущему уровню, системы автомобиля уровня 4 способны осуществлять полное управление транспортным средством в чётко определённых сценариях. От систем 3-го уровня их отличает способность самостоятельного завершения поездки или возвращения в безопасное состояние в случае возникновения внештатной ситуации и отсутствия реакции от водителя. Как следствие, на автомобили данного уровня не накладывается требование к обязательному наличию водителя. К системам этого уровня относят беспилотное такси и автоматизированные грузовые транспортные средства.
6. **Уровень 5. Полная автоматизация (Full Automation)**
   На данном уровне автономности системы автомобиля способны осуществлять полное управление транспортным средством без ограничения на условия эксплуатации. Автомобили данного уровня не требуют наличия механизмов ручного управления, таких как педали торможения/ускорения, рулевое колесо и пр. На сегодняшний день не существует транспортных средств, обладающих 5-м уровнем автоматизации, несмотря на большое количество работ, направленных на их создание.

В настоящее время идёт бурное развитие технологий, направленных на усовершенствование беспилотных транспортных средств, обладающих 4-м уровнем автоматизации, а также создание транспортных средств с 5-м уровнем автоматизации. Среди проектов по разработке таких транспортных средств можно выделить Waymo [9], Tesla [10], Яндекс [11], Baidu Apollo [12] и др. Несмотря на то, что данные проекты показывают высокую степень автономности [13], перед отраслью стоит множество задач, требующих решения для повышения уровня безопасности движения беспилотных транспортных средств, что будет способствовать внедрению данных технологий в повседневное использование [14]. Одна из таких задач – разработка надёжных систем восприятия транспортного средства, отвечающих за построение точной и полной модели окружающего пространства. В рамках данной работы исследуются и предлагаются алгоритмы построения таких моделей.

1.2 Используемые в колёсной робототехнике датчики и их ограничения

Обновление любой модели окружающей сцены производится на основании показаний датчиков, установленных на БТС. Основные датчики, применяемые в колёсной робототехнике для построения модели, – лидары, радары, сонары и камеры [15].

1. **Радары (Radio Detection and Ranging, RADAR)**. Технология, основанная на создании радиоволн и регистрации их отражений от объектов. По времени регистрации и степени затухания отражённой волны датчик определяет расстояние до объекта и его скорость. Радары разделяют на радары дальнего диапазона (РДД) и короткого диапазона (РКД) [16]. Первые обладают наибольшей дальностью видимости (около 250 м) среди упомянутых датчиков и используются для обнаружения препятствий на пути движения машины. Вторые обладают меньшей дальностью, но лучшим разрешением и могут быть использованы для реализации систем парковки и перестроения между полосами движения. В то же время радары обладают низким угловым разрешением и могут давать ложные срабатывания из-за металлических конструкций, например, решёток ливневых канализаций в поле видимости, и переотражений сигнала излучателя.
2. **Лидары (Light Detection and Ranging, LiDAR)**. Технология, начавшая развитие во второй половине XX века, в 1960-х годах, в космической и авиастроительной сферах [17]. Принцип работы схож с радаром, но вместо радиоволн использует пучки инфракрасного спектра. Существует два типа лидаров: механические и твердотельные. В механическом лидаре источники и приёмники сигнала вращаются на высокой ($300 - 1200$ оборотов в минуту) скорости. В твердотельном лидаре источник и приёмник остаются неподвижны, а изменение направления сканирующего луча производится путём вращения призмы или зеркала, через которые он проходит. Помимо измерения расстояния до объекта лидары способны регистрировать его светоотражательную способность, что может быть применено, например, для детекции разметки. Лидары обладают высокой точностью и лучшим разрешением, чем радары, и хорошей дальностью обзора. Однако, как и радары, лидары требова-

тельны к погодным условиям: качество измерений снижается в условиях тумана и/или осадков. Также ограничивающим фактором для использования лидаров является их дороговизна, из-за этого многие компании, занимающиеся разработкой беспилотных автомобилей, вынуждены отказываться от лидаров в пользу альтернативных способов детекции местонахождения машины и построения маршрута движения. Лидары могут использоваться для распознавания и классификации объектов, создания трёхмерных карт.

3. **Камеры**. Наиболее распространённые датчики [18], используемые в колёсной робототехнике. В отличие от остальных описанных датчиков, камеры – пассивные датчики: они не излучают никаких сигналов. Для активных же датчиков отдельной проблемой является взаимная интерференция. Это позволяет без опасений использовать большое количество камер без необходимости синхронизировать их работу друг с другом. В основном в БТС используют камеры видимого (400–780 нм) спектра, но также встречаются модели, использующие камеры инфракрасного спектра. Камеры обладают наилучшим разрешением среди описанных датчиков, но не способны измерять расстояние до объектов. Использование нескольких камер в связке позволяет использовать алгоритмы стереозрения для измерения расстояния, однако точность и дальность этих измерений ниже, чем у лидаров и радаров. Камеры используются для захвата изображений, которые затем обрабатываются с помощью алгоритмов компьютерного зрения. Камеры в компьютерном зрении широко используются в различных областях, таких как робототехника, автоматическое управление производственными процессами, медицинская диагностика, безопасность, распознавание лиц, автоматическое управление транспортом, анализ изображений и многих других.
4. **Сонары**. Как лидары и радары, принцип работы сонаров основан на измерении времени регистрации отражения волн от объектов, однако в сонарах используются ультразвуковые (40–70 кГц) волны [19]. Данные волны не слышны и не могут нанести вред слуху человека, что немаловажно, поскольку громкость создаваемого импульса достигает 100 дБ. Сонары обладают наименьшей областью обзора (менее десятка метров) и потому, как правило, используются для систем парковки. Сонары используются для измерения расстояния до объектов и созда-

ния трёхмерной модели окружающей среды. Они работают на основе эхолокации, отправляя звуковые импульсы и затем регистрируя отражённые от объектов сигналы. Полученные данные могут быть обработаны компьютером для определения расстояния и формы объектов. Одним из наиболее распространённых применений сонаров является их использование при парковке БТС. Также сонары могут использоваться в системах обнаружения движущихся объектов, таких как пешеходов и других транспортных средств.

Из описания датчиков следует, что каждый тип датчика обладает рядом преимуществ и недостатков. Использование нескольких датчиков разных типов на одном транспортном средстве позволяет компенсировать недостатки отдельных датчиков и добиться повышения качества восприятия, тем самым повышая безопасность движения и открывая новые сценарии применения беспилотных транспортных средств. Сводное сравнение характеристик рассмотренных датчиков приведено в таблице 1.

Характеристика	**Лидар**	**Радар**	**Камера**	**Сонар**
Разрешение	Среднее	Низкое	Высокое	Низкое
Дальность области обзора[1]	150 м	250 м	200 м	5 м
Устойчивость к погодным условиям	Средняя	Высокая	Низкая	Высокая
Чувствительность к условиям освещения	Нет	Нет	Высокая	Нет
Информация о дистанции	Да	Да	Нет	Да
Вычислительная нагрузка обработки	Средняя	Низкая	Высокая	Низкая
Стоимость	Высокая	Средняя	Низкая	Низкая

Таблица 1 — Сравнение характеристик датчиков, используемых в колёсной робототехнике. Приведены типичные для колёсной робототехники значения, обобщающие описание датчиков в разделах 1.2 и источниках [15; 18; 19]

[1] Для лидара, радара и сонара указана дальность, на которой датчик измеряет расстояние до объекта. Для камеры измерение расстояния невозможно, поэтому указана дальность уверенного обнаружения объекта характерных для дорожной сцены размеров при типичном для БТС фокусном расстоянии объектива.

1.3 Модели окружающего пространства БТС

Данные, регистрируемые перечисленными датчиками, служат основой для построения модели окружающего пространства. Рассмотрим, какие модели применяются для этого в колёсной робототехнике.

В робототехнике под моделью окружающей сцены, или, что то же самое, моделью окружающего пространства, понимается приближённое описание окружающих БТС объектов: их наличие и положение в некоторой окрестности транспортного средства. Помимо информации о положении объектов, такая модель может описывать их скорость и предполагаемую траекторию движения. Также модель может описывать форму и расположение проезжей части, слепых зон транспортного средства (как вызванных окклюзией, так и обусловленных отсутствием показаний датчиков для соответствующей области пространства), а также объектов транспортной инфраструктуры: полос разметки, дорожных знаков, бордюров, светофоров и т.д. Отметим, что описание окружения используется не только при планировании маршрута: в работе Миллера [20] по последовательности наблюдений окружающего ландшафта оцениваются линейная и угловая скорости аппарата, что позволяет решать задачу навигации при отсутствии сигналов спутниковых навигационных систем.

В рамках данной работы исследуются двумерные модели – модели, проецирующие объекты, окружающие БТС, на плоскость его движения. Такие модели не хранят информацию о высоте объекта и его положении над землёй, т.к., как правило, эти данные избыточны для решения задачи планирования движения колёсного транспорта. В то же время следует отметить, что среди предложенных на сегодняшний день моделей существуют и трёхмерные, обзор которых приведён в работе Юдина [21]. Так, в работе [22] окружающая сцена представлена совокупностью полигонов, в работе [23] пространство разбивается трёхмерной сеткой, каждому элементу которой сопоставляется вероятность нахождения в нём препятствия, а в работе [24] в отдельный класс выделены модели, описывающие окружающее пространство как облако точек.

Среди существующих двумерных моделей можно выделить три категории:

1. **Объектно-ориентированные представления**. В них окружающая сцена описывается как множество объектов. Как правило, объект представлен в виде ограничивающего параллелепипеда: положением его центра,

ориентацией и габаритами [25]. Дополнительно описание объекта может содержать тип (транспортное средство, пешеход, животное и т.д.), а также линейные и угловые скорость и ускорение [26—28], предсказываемую траекторию перемещения [29]. В работе Павоне [30] такое предсказание выполняется сразу для произвольного числа объектов разного типа с учётом их динамики и семантической карты сцены. Такие модели имеют достаточно краткую форму записи: множество векторов, описывающих отдельный объект, что делает их вычислительно эффективными. В то же время они ограничены в возможности описания объектов сложной формы: тягачи с полуприцепами, перевёрнутые машины, объекты нетипичной формы.

2. **Сеточные представления**. Модели этой категории призваны обойти ограничение предыдущей категории и описывать объекты с произвольной геометрией. В них пространство вокруг транспортного средства разбивается на сетку фиксированного разрешения, каждая клетка которой хранит информацию о наличии (или вероятности наличия) препятствия в соответствующей области; построению и уточнению таких карт по показаниям датчиков посвящены, в частности, работы Трун [31]. Такое представление получило название карты проходимости пространства (Occupancy Grid Map, OGM). К этой же категории относятся динамические карты проходимости пространства (Dynamic Occupancy Grid Map, DOGMa) [32], в них для клеток, в которых обнаружено препятствие, добавляется информация о его скорости и направлении движения. Другим расширением служит сетка занятости с высотой (occupancy-elevation grid), предложенная в работе Гонсалвеса [33]: клетка хранит, помимо вероятности занятости, высоту рельефа и её дисперсию, что позволяет судить о проходимости соответствующей области, а не только о наличии в ней препятствия. Модели этой категории также чаще, чем объектно-ориентированные модели, учитывают неклассифицируемые объекты: мусор, открытые люки, бордюры и т.п., что делает их более надёжным представлением окружающей сцены. Однако более детальное описание сцены приводит к большему размеру структуры данных в памяти компьютера, что негативно влияет на вычислительную сложность алгоритмов, работающих с моделями в этой категории. Ограниченная

разрешающая способность данных моделей также может рассматриваться как недостаток для некоторых задач автономной навигации.

3. **Непрерывные представления**. Как и модели прошлой категории, модели непрерывного представления стремятся описывать объекты со сложной геометрией. Однако в отличие от сеточных представлений модели этой категории представляют пространство как непрерывное отображение $\mathbb{R}^2 \rightarrow O$, где каждой точке плоскости движения транспортного средства сопоставляется некоторая характеристика: бинарная классификация принадлежности точки проезжей части дороги ($O = \{0,1\}$), класс объекта ($O = \{0, \dots N\}$), плотность вероятности нахождения препятствия в данной точке ($O = \mathbb{R}^+$) и т.п. Таким образом, модели данной категории имеют бесконечную степень детализации, что позволяет обойти главное ограничение сеточных представлений. Недостаток же моделей данной категории заключается в том, что алгоритмы для их построения и обновления сложнее в реализации, а вычислительная сложность может быть хуже, чем у предыдущих двух категорий (для сеточных представлений последнее зависит от требований к размеру и разрешению сетки). Примерами таких представлений служат полигональные случайные поля (Polygonal Random Field, PRF) [34], мультимодальное нормальное распределение [35].

Наибольшее распространение среди перечисленных получила карта проходимости пространства, поэтому опишем эту модель и методы её построения подробнее.

Представление окружающего пространства в виде карты проходимости пространства было впервые предложено в работе Х. Моравека и А. Элфеса [36]. В рамках данного представления окружающее пространство разбивается на двумерную сетку M, каждой клетке которой m_i сопоставляется вероятность нахождения препятствия в пространстве $p(m_i)$, ограниченном этой клеткой. В работах [37; 38] модель карты проходимости пространства была расширена: в неё включена обратная модель наблюдения – вероятностная модель, моделирующая процесс измерений датчика и учитывающая принцип работы, а также особенности среды, в которой производятся наблюдения. Расширение модели позволило сформулировать алгоритм уточнения карты проходимости пространства на основании показаний датчика. В рамках данной модели дополнительно делается предположение, что каждая вероятность занятости клетки в карте проходимости не зависит

от вероятностей занятости других клеток карты. Для каждой клетки вычисляется величина так называемого логарифма шансов. Приводимые далее выражения представляют собой каноническую форму бинарного байесовского фильтра, изложенную в монографии Труна, Бургарда и Фокса [39]:

$$l_{t,i} = \ln \frac{p(m_i|z_{1:t}, x_{1:t})}{1 - p(m_i|z_{1:t}, x_{1:t})}, \tag{1.1}$$

где $p(m_i|z_{1:t}, x_{1:t})$ – вероятность занятости i-й клетки карты, при условии, что к моменту времени t БТС прошло траекторию $x_{1:t}$, и его датчики давали показания $z_{1:t}$. Заметим, что здесь используется обратная причинно-следственная зависимость: измеряется вероятность причины (занятость пространства), при условии наступления её следствия (показания датчика), чем и обусловлено название модели. С получением новых данных значения в тех клетках, о которых появились новые данные (иными словами, которые попали в область видимости датчика), обновляются по следующей формуле:

$$l_{t,i} = l_{t-1,i} + \ln \frac{p(m_i|z_t, x_t)}{1 - p(m_i|z_t, x_t)} - l_{0,i}, \tag{1.2}$$

где $l_{0,i} = \ln \dfrac{p(m_i)}{1 - p(m_i)}$ – логарифм шансов, посчитанный для априорной вероятности занятости клетки. Зачастую о карте проходимости не делается априорных предположений и $p(m_i)$ устанавливают равным $0{,}5$, что приводит к $l_{0,i} = 0$.

Такой формат обновления удобен, так как позволяет независимо друг от друга посчитать значение $\ln \dfrac{p(m_i|z_t, x_t)}{1 - p(m_i|z_t, x_t)} - l_{0,i}$, на основании полученных данных от каждого датчика, а затем просто добавить полученное значение к уже имеющейся карте проходимости (при условии атомарности данной операции).

1.4 Алгоритмы комплексирования данных

На сегодняшний день предложено множество алгоритмов построения карты проходимости пространства. Их реализация сравнительно проста в случае, если информация об окружающем пространстве поступает от единственного датчика. Примеры таких систем представлены в статьях [40] и [41]. Однако для большинства практических задач единственного датчика недостаточно из-за

ограниченного поля зрения, невозможности полностью компенсировать зашумлённость входных данных и общих ограничений, обусловленных принципом его работы.

Чтобы получить более точную модель окружения и повысить безопасность движения БТС, на них устанавливают несколько датчиков. Примеры систем с несколькими датчиками описаны в работах [42; 43]. Встречаются как системы, использующие несколько датчиков одного типа (например, камеры, как в работах [44; 45]), так и комбинирующие различные датчики, например:

– лидары и радары. Примеры таких БТС представлены в [46—48];
– лидары и камеры [49—51];
– и другие комбинации [52—54].

Алгоритмы построения модели окружающего БТС пространства для таких систем в общем случае генерируют модель достаточно полную и точную, но вынуждены решать задачу комплексирования данных для разрешения противоречий в показаниях разных датчиков. Существуют разные алгоритмы, осуществляющие комплексирование данных, которые можно разделить на три группы [55]:

1. Раннее комплексирование – необработанные данные с датчиков объединяются в один пакет данных и передаются системе для последующего анализа. Примером такой агрегации является построение карты глубины в стереокамере, как например в работе [56].
2. Промежуточное комплексирование – по сырым данным с датчиков вычисляются отдельные признаки (в общем случае отличные для каждого из датчиков), которые затем объединяются для дальнейшего решения задачи. Так в работах [57; 58] данные с разных датчиков обрабатываются разными нейросетевыми моделями, после чего выходные тензоры конкатенируются.
3. Позднее комплексирование – для каждого датчика целевая задача (в настоящей работе – построение модели окружающего БТС пространства) решается отдельно, что даёт независимую гипотезу. На основе этих гипотез более высокоуровневый алгоритм вычисляет итоговое решение.

Обзор состояния исследований в области комплексирования разнородных данных, включая используемые математические аппараты и характерные для задачи трудности, приведён в работе Халеги [59]. В работе Сасядека [60] предложена общая классификация алгоритмов комплексирования данных по исполь-

зуемому математическому аппарату. Применительно к задаче построения карты проходимости пространства она выделяет три группы:

1. Алгоритмы, использующие вероятностные модели. К ним относятся байесовский вывод, теория Демпстера—Шафера, робастные методы и т.д.
2. Алгоритмы, использующие метод наименьших квадратов (МНК): фильтр Калмана, методы оптимизации, эллипсоиды неопределённости (англ. uncertainty ellipsoids) и пр.
3. Алгоритмы интеллектуального комплексирования: методы нечёткой логики, нейросетевые и генетические алгоритмы.

В настоящее время продолжается активная разработка алгоритмов каждой из трёх групп. Примером актуальных исследований в первой группе является работа [61], в которой используется метод релевантных векторов (метод машинного обучения, использующий байесовский вывод) для построения непрерывной карты проходимости пространства. Во второй группе можно выделить работу [62], в которой положение динамических препятствий на карте обновляется с помощью фильтра Калмана, а также работу Николаева [63], где карта проходимости пространства восстанавливается по показаниям нескольких сонаров методом градиентного спуска – то есть на основе прямой, а не обратной модели наблюдения. Стоит отметить, что в первой из этих двух работ [62] обновление информации о статических препятствиях происходит с помощью байесовского вывода, поэтому предлагаемый в ней алгоритм является объединением методов из двух представленных групп. Наконец, не менее активно развиваются алгоритмы третьей группы, одним из которых является модель глубокого обучения BEVFusion, представленная в работе [64].

Помимо уже приведённых классификаций, которые характеризуют то, как устроена обработка данных, также существует классификация по характеру взаимодействия комплексируемых данных. Такая классификация предложена в работе Дюррант-Уайта [65] и выделяет три типа комплексирования:

1. **Конкурирующее** – входные данные содержат оценки одного и того же свойства наблюдаемого объекта, которое, соответственно, может быть уточнено при их объединении. К этому же типу относится и выявление противоречий между независимо зарегистрированными измерениями.
2. **Дополняющее** – входные данные содержат различные свойства наблюдаемого объекта, которые при объединении дополняют друг друга, но не уточняются.

3. **Кооперативное** – входные данные объединяются для оценки свойств, недоступных ни одному из источников по отдельности.

Заметим, что некоторые алгоритмы комплексирования данных могут включать несколько этапов, для каждого из которых могут применяться отдельные типы комплексирования из представленных классификаций. Так, в работе Визильтера [66] изображение бортовой камеры совмещается с синтезированным изображением внешнего вида, формируемым бортовым вычислителем, причём совмещение и последующее объединение выполняются в режиме реального времени. Пример такого алгоритма приведён также в разделе 2.5.

1.5 Метрики для оценки качества алгоритмов построения моделей окружающего пространства

Одной из ключевых задач при построении моделей окружающей сцены БТС является оценка качества таких моделей. Учитывая сложность сенсорного восприятия в условиях реального мира, построенные модели нередко подвержены искажению информации об окружающей сцене из-за неоднородности, шумов, неполноты и несогласованности данных, в частности при использовании разных типов датчиков, регистрирующих данные разных модальностей. Поэтому разработка и выбор адекватных метрик, способных количественно охарактеризовать точность, полноту, согласованность и устойчивость моделей окружающей среды, становится неотъемлемым этапом при анализе и сравнении алгоритмов картографирования.

В данном разделе рассматриваются существующие подходы к оценке качества моделей окружающего пространства. Обсуждаются метрики, разработанные для моделей из классификации, представленной в разделе 1.3.

1.5.1 Метрики для оценки объектно-ориентированных представлений

Среди метрик для объектно-ориентированных представлений широкое применение получил коэффициент Жаккара, введённый в работе Жаккара [67] и

закрепившийся в задачах детекции объектов как мера перекрытия ограничивающих рамок после его использования в наборе данных PASCAL VOC [68]. Пусть p_a – полигон, полученный в результате работы алгоритма построения модели по полученным с датчиков данным, описывающий форму и положение одного из окружающих БТС объектов. Пусть также p_r – полигон, описывающий эталонные форму и положение того же объекта. Тогда коэффициент Жаккара равен:

$$J = \frac{S(p_a \cap p_r)}{S(p_a \cup p_r)}, \tag{1.3}$$

где $S(p_a \cap p_r)$ – площадь полигона, полученного при пересечении p_a и p_r, $S(p_a \cup p_r)$ – площадь полигона, полученного при объединении p_a и p_r. Таким образом, значение J лежит в диапазоне $[0; 1]$. Чем больше значение J, тем точнее p_a совпадает с p_r и тем качественнее модель. Отметим также, что зачастую в качестве сравниваемых полигонов выступают произвольно ориентированные ограничивающие прямоугольники. Коэффициент Жаккара вычисляется для каждой пары сопоставленных вычисленного и эталонного полигонов, поэтому качество модели на наборе данных характеризуется выборочным распределением величины J по всем таким парам, а для краткого представления результатов приводятся выборочные статистики этого распределения: среднее значение, среднеквадратичное отклонение, минимум, квартили и максимум.

Помимо коэффициента Жаккара при оценке качества моделей можно встретить метрики, используемые при оценке решения задачи классификации. Например, F-score. Для этого вводится некоторый порог $j_{thr} \in (0,1)$. Корректно обнаруженными объектами (верными срабатываниями алгоритма – TP) считаются только те p_a, для которых $J > j_{thr}$; остальные p_a считаются ложными срабатываниями алгоритма – FP. Наконец, эталонные p_r, для которых не нашлось p_a с достаточным коэффициентом Жаккара, считаются пропусками алгоритма – FN. Итоговый F-score равен:

$$\text{F-score} = \frac{2}{Recall^{-1} + Precision^{-1}}, \tag{1.4}$$

где $Recall = \frac{|TP|}{|TP|+|FN|}$ – полнота, $Precision = \frac{|TP|}{|TP|+|FP|}$ – точность.

Существует также метрика локализационной точности (localization accuracy), которая, как и коэффициент Жаккара, оценивает качество вычисленных p_a, но только для верных срабатываний алгоритма:

$$\text{LocA} = \frac{1}{|TP|} \sum_{c \in TP} J_c. \tag{1.5}$$

Отдельную группу образуют метрики, оценивающие не только положение объектов в сцене (задача детекции объектов), но и постоянство их идентификаторов между кадрами (задача отслеживания объектов). Исторически первой из них является MOTA (Multi Object Tracking Accuracy) [69], объединяющая в одном показателе доли пропусков, ложных срабатываний и переключений идентификатора; её недостатком является преимущественная чувствительность к качеству детекции, а не отслеживания. Метрика IDF1 [70] оценивает качество отслеживания непосредственно: она вычисляется по наилучшему сопоставлению вычисленных и эталонных идентификаторов на всей последовательности кадров. Метрика HOTA (Higher Order Tracking Accuracy) [71] представляет собой среднее геометрическое двух вспомогательных величин – точности детекции DetA и точности ассоциации AssA, проинтегрированное по порогу коэффициента Жаккара, что даёт сбалансированную оценку качества решения обеих задач. Поскольку в настоящей работе задача отслеживания объектов не решается, подробное описание перечисленных метрик здесь не приводится.

1.5.2 Метрики для оценки сеточных представлений

Для сеточных представлений было также предложено множество метрик для оценки их качества, как, например, метрики, представленные в работе Ши [72], обозревающей существующие сеточные представления, алгоритмы их построения и методы оценки их качества. Многие метрики, применимые для объектно-ориентированных представлений, применимы и для моделей этой категории. Так, бинарные карты проходимости пространства, т.е. карты, клетки которых могут принимать только два значения, оцениваются коэффициентом Жаккара J: площадью пересечения при этом считается число клеток, отнесённых к занятому пространству и на оцениваемой, и на эталонной картах, а площадью объединения – число клеток, отнесённых к занятому пространству хотя бы на одной из них. Если клетки классифицируются более чем на два класса, коэффициент Жаккара вычисляется для каждого класса отдельно и усредняется по классам [73]:

$$mJ = \frac{1}{|K|} \sum_{k \in K} J_k, \tag{1.6}$$

где K – множество классов клеток, J_k – коэффициент Жаккара, вычисленный для класса k. В отличие от доли верно классифицированных клеток, такое усреднение не позволяет высокому качеству описания преобладающего по площади класса (как правило, свободного пространства) маскировать ошибки в описании остальных классов. Также встречается оценка с использованием F-score [74].

Если клетки карты хранят не класс, а непрерывную величину – вероятность занятости либо соответствующий ей логарифм шансов, – качество модели оценивают среднеквадратичной ошибкой относительно эталонной карты:

$$\text{MSE}(M, M_r) = \frac{1}{|M|} \sum_{i} \left(p(m_i) - p(m_i^r)\right)^2, \tag{1.7}$$

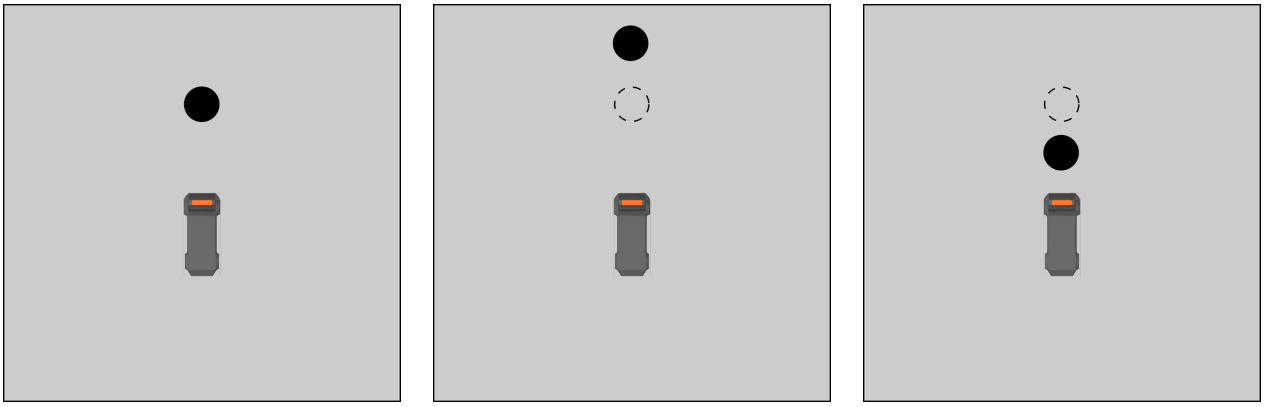
где M – оцениваемая карта проходимости пространства, M_r – эталонная карта того же размера и разрешения, $p(m_i)$ и $p(m_i^r)$ – вероятности занятости i-й клетки оцениваемой и эталонной карт соответственно, $|M|$ – общее число клеток карты. В отличие от коэффициента Жаккара, эта метрика не требует бинаризации значений клеток и потому учитывает степень уверенности модели в занятости пространства. Однако, как и коэффициент Жаккара, она штрафует ошибку одинаково независимо от того, в какой части карты эта ошибка допущена.

Для некоторых задач важно оценить не только полноту и корректность отображения текущей сцены на карте проходимости пространства, но и качество предсказания её значений в будущем. Такая задача исследуется, например, в работе Xy [75]: получая на вход набор изображений, зарегистрированных на камерах БТС, предлагаемая в работе модель строит экземплярную сегментацию для текущего момента времени t и предсказывает её для набора будущих моментов времени вплоть до некоторого горизонта H. Для оценки качества создаваемого решения этой задачи в работе использовалось паноптическое качество видео (англ. Video Panoptic Quality) [76]. Данная метрика задаётся следующим выражением:

$$VPQ = \sum_{t=0}^{H} \frac{\sum_{(x_t, y_t) \in TP_t} J(x_t, y_t)}{|TP_t| + \frac{1}{2}|FP_t| + \frac{1}{2}|FN_t|}, \tag{1.8}$$

где TP_t – совпавшие с эталонными значениями экземпляры, FP_t – экземпляры предсказания, отсутствующие в эталонных данных, FN_t – эталонные экземпляры, отсутствующие среди найденных.

В контексте задачи планирования маршрута перечисленные метрики могут быть не показательны. Действительно, на рисунке 1.1 приведён пример эталонной карты проходимости и двух вариантов карт, которые с точки зрения приведённых метрик имеют одинаковое качество. В то же время легко видеть, что карта 1.1б потенциально опаснее, т.к. согласно ей БТС может продолжать движение вперёд, что в реальности приведёт к столкновению с препятствием.



а) Эталонная карта проходимости пространства.

б) Регистрация препятствия дальше эталонного положения.

в) Регистрация препятствия ближе эталонного положения.

Рисунок 1.1 — Пример эталонной карты и предсказанных карт с одинаковым качеством согласно метрикам, не учитывающим контекст решаемой задачи

Чтобы учесть это, в работе [77] была предложена метрика PathFinding Cost Mean Squared Error (PFC-MSE). Для её подсчёта производятся следующие шаги:

1. Для каждого типа клетки выбирается стоимость её «проезда» $c > 0$. Для клеток, соответствующих занятым областям, эта стоимость должна быть значительно выше, чем для свободных клеток (в оригинальной работе стоимость занятых клеток составила $\text{cost}_{\text{occ}} = 100$, свободных: $\text{cost}_{\text{free}} = 1$).
2. Для эталонной и оцениваемой карт проходимости строятся ориентированные графы. Вершинами графа являются клетки карты, рёбрами соединены соседние клетки карты, вес ребра – стоимость проезда клетки, являющейся конечной вершиной ребра, умноженная на расстояние между центрами клеток $w(c_1 \rightarrow c_2) = cost(c_2) \cdot ||c_2 - c_1||_2$ (в оригинальной работе клетки, расположенные по диагонали, также считаются соседними).

3. С помощью алгоритма Дейкстры [78] определяется стоимость кратчайшего пути $path_cost(c)$ от центральной клетки карты, соответствующей положению БТС на карте.
4. Для каждой клетки стоимость пути затем нормируется:

$$path_cost_{\text{final}}(c) = \frac{path_cost(c) - L(c) \cdot \text{cost}_{\text{free}}}{\text{cost}_{\text{occ}} - \text{cost}_{\text{free}}}, \tag{1.9}$$

   где $L(c)$ – длина пути до вершины c.
5. Значения $path_cost_{\text{final}}$ по всем клеткам образуют карту стоимостей путей. Для двух таких карт, построенных по эталонной и по оцениваемой картам проходимости, находится среднеквадратичная ошибка, взвешенная вероятностью того, что соответствующая клетка является свободным пространством:

$$\text{PFC-MSE}(GT, I) = \frac{\sum_c w_c \left(GT_{cost}(c) - I_{cost}(c)\right)^2}{\sum_c w_c}, \tag{1.10}$$

   где GT_{cost} и I_{cost} – карты стоимостей путей, построенные по эталонной и по оцениваемой картам проходимости, $w_c = p(c = \text{free})$.

Отметим особенности этой метрики. Во-первых, её значения зависят от выбора стоимостей cost_{occ} и $\text{cost}_{\text{free}}$: отношение этих величин задаёт, во сколько раз обход препятствия «дороже» движения по свободному пространству, и потому определяет, насколько сильно метрика штрафует ошибку в описании занятого пространства по сравнению с ошибкой в описании свободного. Сравнивать между собой можно лишь значения, полученные при одинаковых стоимостях. Во-вторых, метрика зависит от выбора стартовой клетки: пути строятся от положения БТС, поэтому ошибки вблизи него сказываются на стоимостях путей ко всем последующим клеткам, а ошибки на периферии карты – лишь на стоимостях путей к самим этим клеткам. В-третьих, области, недостижимые из стартовой клетки, не влияют на значение метрики вовсе: ошибка в описании пространства, отгороженного от БТС препятствием, метрикой не фиксируется. Последнее свойство является не только недостатком: оно отражает то обстоятельство, что для планирования пути такие области действительно несущественны.

1.5.3 Метрики для оценки непрерывных представлений

Работы, представляющие модели из категории непрерывных представлений, часто сравниваются с моделями сеточных представлений [79—81]. Поэтому типичный подход к их оценке следующий: разбить пространство на сетку (двумерную или трёхмерную в зависимости от размерности оцениваемой модели), вычислить преобладающее значение внутри ограниченного клеткой пространства и тем самым преобразовать оцениваемую модель в модель сеточного представления, которую можно сравнивать с аналогами из этой категории.

Для моделей, которые сопоставляют каждой точке пространства плотность вероятности нахождения в ней препятствия, стандартный метод к оценке качества – построение precision-recall (PR) кривой. Каждой клетке после дискретизации пространства соответствует вероятность занятости соответствующего пространства, для вычисления PR-кривой перебираются все возможные значения порога p, по которому бинаризуются значения клеток. Для полученной бинаризации вычисляется точность и полнота:

$$Precision_p = \frac{|TP_p|}{|TP_p| + |FP_p|}, \tag{1.11}$$

$$Recall_p = \frac{|TP_p|}{|TP_p| + |FN_p|}. \tag{1.12}$$

Полученное множество точек $(Recall_p, Precision_p) \in [0; 1]^2$ образует PR-кривую, площадь под которой (англ. Area Under Curve, AUC) описывает качество построенной модели.

1.6 Выбор модели окружающего пространства и метрик её оценки

Проведённый обзор позволяет остановить выбор на модели окружающего пространства, которая будет использована в работе, и на метриках оценки её качества.

В качестве базовой модели в работе принята карта проходимости пространства из категории сеточных представлений (раздел 1.3). Выбор обоснован

тремя обстоятельствами. Во-первых, модель описывает объекты произвольной геометрии и не требует их отнесения к заранее определённым классам, что существенно для БТС, движущегося в неструктурированной среде, где встречаются объекты, не представленные в обучающих выборках детекторов. Во-вторых, она допускает явное представление областей, о проходимости которых информация отсутствует, что необходимо для безопасного планирования пути в условиях окклюзии. В-третьих, принятый метод её обновления – байесовский вывод с использованием обратной модели наблюдения (формула (1.2)) – сводит учёт данных отдельного источника к сложению слагаемых, вычисляемых по данным этого источника независимо от остальных. Тем самым выбранная модель реализует позднее комплексирование в классификации по стадии обработки данных раздела 1.4. Существенно, что учёт данных источника сводится при этом к сложению вкладов в клетки, считаемые независимыми, а сложение коммутативно: результат не зависит от порядка, в котором источники внесли свои вклады, и потому не требует глобальной ассоциации данных между источниками. Для объектно-ориентированных представлений такая ассоциация необходима: прежде чем объединить сведения двух источников, нужно установить, описывают ли они один и тот же объект.

1.6.1 Выбор метрик, используемых в работе

Из рассмотренных метрик в настоящей работе используются следующие. Положение и форма объектов в окружающем пространстве БТС оцениваются коэффициентом Жаккара J, вычисляемым для каждой пары сопоставленных вычисленного и эталонного положений объекта. Обнаружение объектов на изображении выполняется существующей нейросетевой моделью и в число решаемых в работе задач не входит, поэтому предметом оценки является геометрическая точность описания объекта, а не полнота его обнаружения. По этой причине метрики F-score и LocA, характеризующие в первую очередь полноту обнаружения и требующие бинаризации значений J по порогу j_{thr}, не используются: непрерывная величина J позволяет сравнивать качество во всём диапазоне значений, а не только выше выбранного порога. Качество карты проходимости пространства, построенной с учётом визуальных данных, оценивается среднеквадратичной

ошибкой относительно эталонной карты (1.7) и метрикой PFC-MSE: первая характеризует поклеточное расхождение моделей, вторая – влияние этого расхождения на решение задачи планирования пути, ради которой модель и строится. Метрики отслеживания объектов и метрики непрерывных представлений в работе не используются, поскольку соответствующие задачи в ней не решаются.

1.7 Постановка частных задач исследования

В рамках данной диссертационной работы при разработке алгоритмов построения карты проходимости пространства помимо прочих необходимо решить две задачи. Первая связана с тем, что камера – наиболее распространённый датчик БТС – регистрирует данные в системе координат изображения, тогда как модель описывает плоскость движения: результаты обработки изображения требуется спроецировать в пространство модели. Вторая состоит в том, что данные всех источников должны быть учтены в единой карте, и вычислительная стоимость этого учёта определяет, сколько источников может быть задействовано без потери актуальности модели. Для каждой из задач рассмотрим существующие подходы, оценим их применимость в условиях, характерных для бортового программного обеспечения БТС, и выделим ограничения, которые будут положены в основу требований к разрабатываемым алгоритмам (раздел 1.8).

1.7.1 Алгоритмы проецирования данных камеры на плоскость движения БТС

Как отмечено в разделе 1.2, камера является наиболее распространённым датчиком в колёсной робототехнике, однако она не позволяет непосредственно измерить расстояние до наблюдаемых объектов. Результаты обработки изображения, например, визуальные детекции объектов и семантическая сегментация, получаются в однородной системе координат изображения, тогда как двумерные модели окружающего пространства, рассмотренные в разделе 1.3, описывают плоскость движения в трёхмерной системе координат БТС. Поэтому учёт визу-

альных данных в таких моделях требует отдельного алгоритма проецирования, от точности которого напрямую зависит корректность положения препятствий в итоговой модели.

Изображение задаёт лишь направление на наблюдаемый объект, поэтому для восстановления расстояния до него требуется либо дополнительное предположение о геометрии сцены, либо дополнительный источник данных. Определяющим для точности проецирования является то, какая модель поверхности движения БТС при этом используется, и по этому признаку существующие подходы разделим на три группы:

1. **Проецирование с использованием геометрических предположений о поверхности движения.** Если внутренняя и внешняя калибровки камеры известны, а поверхность движения считается плоскостью, то соответствие между плоскостью изображения и плоскостью движения задаётся проективным преобразованием. Уравнение данной плоскости может быть фиксированным, либо пересчитываться на данных, регистрируемых датчиками, например, широкое применение получил способ поиска плоскости в облаке точек с лидаров с помощью метода RANSAC [82]. Проективное преобразование применяется как для проецирования всего изображения на вид сверху, так и для проецирования отдельных признаков: нижних границ детекций, разметки, границ проезжей части. Пример системы, использующей данный тип проекции, – система GOLD [83], применявшаяся в проекте автономного автомобиля ARGO [44]. Достоинствами методов этой группы являются вычислительная простота и отсутствие необходимости в обучении; кроме того, при использовании единственной камеры иного источника информации о геометрии сцены попросту нет. Основной их недостаток – само предположение о плоскости поверхности движения. Оно нарушается как из-за рельефа местности, так и в силу требований к конструкции дорожного покрытия: согласно СП 34.13330.2021 [84] покрытие имеет двускатный поперечный профиль на прямых участках и односкатный на виражах. Дополнительным источником погрешности является отклонение фактической внешней калибровки от измеренной, вызванное загрузкой БТС, работой подвески и смещением датчиков при длительной эксплуатации БТС. Ошибка проецирования при этом растёт с удалением от камеры.

2. **Проецирование с восстановлением формы поверхности движения.** Если БТС оснащено дальномерным датчиком, предположение о плоскости можно заменить оценкой формы поверхности $z = z(x, y)$, полученной по измерениям, и выполнять проецирование уже на эту поверхность. В качестве примеров такого проецирования выступают методы, основанные на разбиении облака точек сеткой в полярных координатах с последующей аппроксимацией высоты земли в каждом секторе, предложенные в работах Химмельсбаха [85] и Зермаса [86]; их развитием является алгоритм Patchwork [87], использующий кусочную аппроксимацию поверхности с оценкой достоверности каждого участка. Наиболее общим является представление поверхности земли как реализации гауссовского процесса [88]: данный метод не требует ни плоскостной, ни кусочно-плоскостной модели и позволяет оценить дисперсию предсказания, но обладает большой вычислительной сложностью по отношению к числу точек в обучающей выборке: $O(n^3)$, где n – число точек. Это затрудняет его применение в режиме реального времени. Применение регрессии на основе гауссовских процессов к выделению точек земли рассмотрено в работе Чена [89].
3. **Обучаемые преобразования в вид сверху.** Третью группу образуют нейросетевые методы, выполняющие преобразование признаков изображения в вид сверху без явно заданной модели поверхности: геометрия сцены восстанавливается неявно, в процессе обучения. В работе Филиона [90] для каждого пикселя изображения предсказывается распределение по глубине, после чего признаки проецируются в трёхмерное пространство на сетку вида сверху с учётом веса предсказанного распределения. В работе Ли [91] преобразование реализовано с применением механизма внимания над обучаемыми запросами, соответствующими клеткам сетки. Модель BEVFusion [64; 92] объединяет в едином представлении вида сверху признаки камер и лидаров. Развитием этого направления стал сложившийся в последние годы класс методов предсказания трёхмерной занятости (3D occupancy prediction), в которых по изображениям камер строится не двумерная сетка вида сверху, а трёхмерная воксельная сетка с семантической разметкой. Так, в работе Хуана [93] сцена описывается тремя взаимно ортогональными проекциями, что снижает вычислительную стоимость по сравнению с полным

воксельным представлением, а в работе Тонга [94] предсказанная занятость используется как общее промежуточное представление для задач детекции, сегментации и планирования. Методы этой группы демонстрируют высокое качество, однако требуют графического ускорителя для инференса, размеченных мультимодальных наборов данных для обучения и переобучения при изменении состава и расположения датчиков БТС. Согласно требованию, сформулированному далее в разделе 1.8, разрабатываемые алгоритмы должны выполняться на бортовом вычислителе БТС при ограниченных вычислительных ресурсах, разделяемых между всеми подсистемами, и допускать изменение состава датчиков без переобучения и переразметки данных. По этим двум признакам вся группа обучаемых преобразований в вид сверху, включая методы предсказания трёхмерной занятости, в настоящей работе не рассматривается.

Результатом проецирования любым из перечисленных способов является множество точек либо полигон на плоскости движения БТС. Потребители модели окружающего пространства, как правило, не работают с таким представлением непосредственно, и тому есть две причины. Во-первых, в измерениях датчиков присутствует только обращённая к ним часть объекта, поэтому проекция систематически занижает занимаемую объектом область: недоступная для наблюдения часть объекта не будет учтена ни в модели, ни, следовательно, при планировании пути. Во-вторых, сложившимся интерфейсом между подсистемой восприятия и последующими подсистемами БТС служит описание объекта ориентированным ограничивающим параллелепипедом либо прямоугольником: в этой форме размечены общепринятые наборы данных [26], в ней формулируется задача в обзорной литературе по трёхмерной детекции [25], её же принимают на вход алгоритмы предсказания траекторий окружающих объектов [29] и алгоритмы планирования пути, работающие с представлением препятствий в виде фигур ненулевой площади [95]. Поэтому проецирование, как правило, дополняется этапом вписывания полученной проекции в ориентированную ограничивающую рамку.

Простейшим решением этой задачи является построение рамки минимальной площади, для которого существует алгоритм с линейной по числу вершин выпуклой оболочки вычислительной сложностью [96]. В работе Ванга [97] для кластеров лидарных точек предложено семейство методов, использующих главные компоненты множества точек, наибольшую диагональ и вписанный треугольник. В работе Чжана [98] для той же задачи предложено вписывание в

L-образную модель: точки кластера считаются принадлежащими двум смежным ортогональным сторонам объекта, а ориентация рамки определяется перебором угла с фиксированным шагом. Общим ограничением перечисленных методов является то, что они разработаны для кластеров лидарных точек, природа которых отличается от таковых для проекции визуальной детекции; кроме того, определение ориентации рамки перебором угла требует вычислительных затрат, пропорциональных числу проверяемых ориентаций.

Таким образом, существующие подходы к проецированию данных камеры на плоскость движения БТС либо опираются на предположение о плоской поверхности движения и потому вносят растущую с расстоянием погрешность, либо требуют обучения и вычислительных ресурсов, недоступных на бортовом вычислителе при одновременной обработке данных нескольких датчиков. Методы вписывания проекции в ориентированную рамку, в свою очередь, разработаны для лидарных кластеров и не учитывают специфику проекции визуальной детекции. Разработка вычислительно эффективных алгоритмов проецирования визуальных детекций и семантической сегментации проезжей части, не использующих предположение о плоской поверхности движения, а также алгоритма вписывания полученной проекции в ориентированную ограничивающую рамку составляет содержание главы 2.

1.7.2 Влияние числа источников данных на производительность алгоритмов построения карты проходимости пространства

В разделах 1.2 и 1.4 показано, что увеличение числа и разнообразия датчиков повышает полноту и достоверность модели окружающего пространства. Однако каждый дополнительный источник данных увеличивает вычислительную нагрузку на бортовой вычислитель БТС, а следовательно, и задержку между моментом регистрации данных и моментом, когда эти данные учтены в модели. Рассмотрим, как эта задержка формируется и какие ограничения она накладывает.

Обозначим через Δt время между регистрацией данных датчиком и завершением обновления модели. Процесс обновления можно условно разделить на четыре составляющие: передачу данных от датчика к вычислителю; извлечение из них признаков (визуальных детекций, семантической сегментации изображения,

семантической сегментации облака точек и так далее); преобразование признаков в общий для всех источников формат; добавление результатов измерения в итоговую модель. Первые три составляющие выполняются для каждого источника независимо и потому допускают параллельное вычисление. Последняя составляющая является операцией над разделяемым ресурсом – общей картой проходимости пространства – и потому, в общем случае, требует синхронизации потоков выполнения либо реализации всего алгоритма с помощью последовательных вычислений. Так, в библиотеке инструментов для навигации робототехнических систем navigation2 [99] построение карты проходимости пространства реализовано в пакете costmap_2d. В данном пакете карта представлена набором слоёв, каждый из которых соответствует отдельному источнику данных. С заданной периодичностью доступ к слоям на запись блокируется для агрегации их в единую модель и передачи результата получателям. Другие библиотеки grid_map [100] и OctoMap [101], реализующие схожий функционал, не содержат встроенных средств блокировок и не являются потокобезопасными.

Параллельные вычисления при построении сеточных моделей окружающего пространства применяются, однако на других уровнях. Первый из них – вычисление одного обновления карты: в работе Хомма [102] на графическом ускорителе распараллелено построение карты занятости по данным лидара и радара, а в работе Нусса [32] тем же способом реализовано построение динамической карты проходимости пространства в реальном времени; вопрос одновременного доступа к карте нескольких независимых источников данных в этих работах не рассматривается. Второй уровень – независимые слои карты, как в описанном выше пакете costmap_2d: слои, соответствующие разным источникам данных, обновляются независимо друг от друга, однако сведение их в общую сетку выполняется последовательно. Наконец, самостоятельное направление образуют неблокирующие структуры данных, допускающие одновременное изменение общего объекта несколькими потоками выполнения без использования примитивов взаимного исключения [103; 104].

В рассмотренных библиотеках одновременное обновление общей сетки несколькими источниками данных не поддерживается: доступ к ней либо сериализуется, либо потокобезопасность не обеспечивается вовсе и возлагается на пользователей. Неблокирующие структуры данных к разделяемой сетке карты проходимости пространства при этом не применялись. Причина этого состоит не в сложности их реализации: локальная карта, следующая за БТС, требует операций,

затрагивающих её целиком, – сдвига вслед за перемещением БТС и приведения всех клеток к общему моменту времени. Такие операции по своей природе не сводятся к независимым изменениям отдельных клеток, поэтому применение неблокирующих структур данных требует изменения не механизма синхронизации, а самого представления модели.

Оценим количественно, к каким ограничениям приводит последовательный учёт данных в карте. Задержка Δt между регистрацией данных датчиком и их отражением в карте подчиняется двум независимым условиям, которые далее в работе используются раздельно.

Условие 1 (непотеря данных). Поступившие измерения должны быть учтены до поступления следующих, иначе очередь на обновление карты растёт неограниченно:

$$\Delta t \leqslant \frac{1}{f}, \tag{1.13}$$

где f – частота работы источников данных. При типичной для бортовых датчиков БТС частоте $f = 10$ Гц условие 1 даёт 100 мс.

Условие 2 (содержательность разрешения). Собственное перемещение БТС за время Δt на актуальность карты не влияет: каждый фрагмент хранится вместе с положением датчика и временем регистрации и может быть приведён к текущему моменту при добавлении на карту. Некомпенсированным остаётся перемещение подвижных объектов сцены: за время Δt объект, движущийся со скоростью v_{obj}, смещается на $v_{obj}\Delta t$, тогда как на карте он остаётся в прежнем положении. Существенно, что в это смещение входит лишь собственная скорость объекта: перемещение самого БТС учтено при добавлении фрагмента на карту, и потому скорость сближения БТС с объектом на величину ошибки не влияет. Естественно потребовать, чтобы смещение объекта не превышало размера элемента карты: в противном случае разрешение модели оказывается фиктивным, поскольку описание строится с точностью, которой заведомо не соответствует момент его актуальности. Отсюда допустимая задержка составляет

$$\Delta t \leqslant \frac{1}{v_{obj}\, res}. \tag{1.14}$$

При разрешении $res = 10$ элементов на метр, то есть при размере элемента $0{,}1$ м, и скорости объекта 40 км/ч условие 2 даёт около 9 мс. Значение

$v_{obj} = 40$ км/ч принято расчётным исходя из области эксплуатации рассматриваемых БТС – закрытых территорий и складов открытого типа. Наиболее быстрые подвижные объекты в такой обстановке – это ТС того же класса, и их скорость ограничена тем же режимом движения, что и рабочая скорость самого БТС.

Условия различаются более чем на порядок, и определяющим является более строгое из них – условие 2. Соблюдение условия 1 тем самым не означает, что задержка приемлема: обоим условиям должен удовлетворять алгоритм построения карты, и проверка этого выполнена в главе 4. Отметим также, что при сериализации задержка конкретного источника зависит от его места в очереди и потому не постоянна: для подсистемы, работающей в режиме реального времени, недетерминированность задержки нежелательна сама по себе, независимо от её абсолютной величины.

Оценим теперь, как ограничение на задержку сказывается на предельном числе источников данных. Размер карты выбирается из требования безопасной остановки: если препятствие обнаружено на границе карты, БТС должно успеть плавно затормозить до его достижения. Пусть R – расстояние от БТС до границы карты, a – ускорение при плавном торможении БТС. Тогда, чтобы БТС успело остановиться перед обнаруженным на краю карты препятствием, должно выполняться неравенство:

$$v_{max} < \sqrt{2aR}. \tag{1.15}$$

Полученная оценка является оценкой сверху, так как не учитывает время реакции системы автопилотирования на обновление карты проходимости пространства. Дальнейший вывод опирается на следующие допущения: карта квадратная, со стороной $2R$; расстояние до её границы определяется исключительно тормозным путём; время учёта данных одного источника линейно по числу клеток и одинаково для всех источников; все источники работают с одной и той же частотой. Пусть res – разрешение карты, выраженное числом её элементов, приходящихся на метр, тогда при фиксированном расстоянии до границы карты R число клеток в ней составляет $N = (2Rres)^2$. Подставив v_{max} из формулы (1.15), получаем $N \propto v_{max}^4$. В главе 4 будет показано, что в базовой реализации учёт данных одного источника в карте проходимости пространства требует $O(N)$ операций. Условие 1 в применении к S источникам состоит в том, что все S измерений должны быть учтены до поступления следующих. Для последовательного обновления всеми имеющимися источниками данных оно принимает вид

$ST(N) \leqslant 1/f$, где $T(N)$ – время учёта в карте с N клетками данных одного источника; f – частота работы источников данных. Совмещая эти зависимости, получаем, что предельное число одновременно учитываемых источников данных обратно пропорционально четвёртой степени рабочей скорости БТС:

$$\frac{S_{max}(v_2)}{S_{max}(v_1)} = \left(\frac{v_1}{v_2}\right)^4. \tag{1.16}$$

Результаты расчёта по (1.16) для скоростей 20, 40, 60 и 80 км/ч приведены в таблице 2. 40 км/ч используются в качестве референса. Можно видеть, что увеличение скорости с 40 до 80 км/ч сокращает число источников данных, которые могут быть учтены в модели, в 16 раз.

Таблица 2 — Изменение предельного числа одновременно учитываемых источников данных с ростом рабочей скорости БТС

Рабочая скорость БТС v_{max}	**20 км/ч**	**40 км/ч**	**60 км/ч**	**80 км/ч**
Минимальное расстояние до границы карты R, м	7,7	30,9	69,4	123,5
Размер карты N (относительно 40 км/ч)	$\times 0{,}06$	$\times 1$	$\times 5{,}06$	$\times 16$
Число источников S_{max} (относительно 40 км/ч)	$\times 16$	$\times 1$	$\times 0{,}20$	$\times 0{,}06$

Чтобы представить порядок абсолютных значений, отметим следующее. Используемая в настоящей работе карта проходимости пространства имеет размер 600×600 клеток при разрешении $res = 10$ элементов на метр, то есть при размере элемента $0{,}1$ м, что согласно (1.15) при $a = 2$ м/с^2 соответствует рабочей скорости около 40 км/ч. Приняв для карты такого размера $T(N) \approx 3$ мс – порядок величины, полученный при профилировании в разделе 4.1, – и частоту работы источников $f = 10$ Гц, получаем, что по условию 1 одновременно может быть учтено порядка трёх десятков источников данных. Эта оценка получена по менее строгому из двух условий: тот же расчёт по условию 2 с ограничением в 9 мс оставляет лишь порядка трёх источников, то есть на порядок меньше. Кроме того, запас расходуется быстро: он убывает как четвёртая степень рабочей скорости и как квадрат разрешения карты, так что при попытке двигаться со скоростью

80 км/ч то же соотношение оставляет лишь два источника данных, чего недостаточно даже для описания окружающего пространства по данным одного датчика с несколькими обработчиками.

Таким образом, последовательный учёт данных ограничивает построение модели по двум направлениям сразу. Задержка обновления карты растёт с числом источников и при этом не постоянна, а требование содержательности разрешения (условие 2) оказывается ощутимо строже, чем требование непотери данных. Запас же по числу источников, имеющийся на текущей рабочей скорости, сокращается тем быстрее, чем выше скорость и разрешение карты. Оба ограничения происходят из того, что учёт данных выполняется над разделяемым ресурсом, а не из вычислительной стоимости самой операции.

Один из способов увеличить число источников данных с сохранением работы в режиме реального времени – обеспечить возможность одновременного обновления карты несколькими источниками данных. Это требует, во-первых, программной архитектуры, позволяющей подключать произвольное число источников данных без изменения кода построения карты (глава 3), и, во-вторых, соответствующей модификации самого алгоритма построения карты (глава 4).

1.8 Требования к разрабатываемым модели окружающего пространства и алгоритмам её построения

Выбор модели окружающего пространства и обзор методов решения возникающих при её построении задач, проведённые в разделах 1.2–1.7.2, позволяют сформулировать требования к разрабатываемым модели окружающего пространства и алгоритмам её построения.

1. алгоритмы построения модели должны обеспечивать комплексирование данных камер и лидаров, поскольку ни один из этих типов датчиков в отдельности не позволяет одновременно получить высокое разрешение и непосредственное измерение расстояния (раздел 1.2);
2. алгоритмы построения модели не должны опираться на предположение о совпадении поверхности движения с плоскостью;
3. разрабатываемые алгоритмы должны выполняться в режиме реального времени на бортовом вычислителе БТС при ограниченных вычислитель-

ных ресурсах, что исключает применение методов, требующих инференса крупных нейросетевых моделей для каждого источника данных (разделы 1.7.1, 1.7.2);

4. модель должна быть масштабируемой по количеству используемых датчиков БТС и расширяемой под новые типы датчиков;
5. модель окружающего пространства и алгоритм её построения должны предоставлять возможность одновременного обновления модели несколькими источниками данных.

Разработке моделей и алгоритмов, удовлетворяющих данным требованиям, посвящены последующие главы данной работы.

1.9 Заключение к главе 1

В данной главе выполнено следующее:

1. определены типичные виды датчиков, используемых на БТС для построения и уточнения моделей окружения, выделены преимущества и ограничения каждого из них. Сделан вывод о необходимости комплексирования данных с датчиков для построения более полной и точной модели окружения;
2. проведён анализ существующих моделей представления окружающего пространства вокруг БТС, алгоритмов их построения и существующих способов комплексирования разнородных данных. Обоснован выбор карты проходимости пространства в качестве модели, описывающей проходимость окружающего БТС пространства, и байесовского вывода с использованием обратной модели наблюдения в качестве алгоритма её построения;
3. определены основные метрики, используемые для оценки качества различных моделей представления окружающего пространства. Обоснован выбор метрик, используемых в дальнейшей работе: коэффициента Жаккара для объектных представлений окружающего пространства и метрики PFC-MSE для карты проходимости пространства;
4. сформулированы требования к разрабатываемым в последующих главах работы модели и алгоритмам её построения.

Глава 2. Алгоритмы проецирования визуальных данных на плоскость движения БТС

В главе 1 была показана необходимость комплексирования данных для построения более точной модели окружающего пространства. В данной главе предлагаются алгоритмы, реализующие комплексирование данных с RGB-камер и лидаров, установленных на БТС. Вначале предложен алгоритм оценки положения объектов, обнаруженных на изображении с камеры, в трёхмерном пространстве с использованием только данных с камеры и показана ограниченность качества его работы. Затем предложен алгоритм для решения той же задачи, но с использованием комплексирования данных с камеры и лидаров. Наконец, предложен алгоритм проецирования информации о свободной проезжей части с изображения на плоскость движения БТС и учёта полученной проекции для улучшения базового алгоритма построения карты проходимости пространства.

2.1 Алгоритм проекции визуальных детекций на плоскость движения БТС на основе модели L-Shape

В данном разделе описан метод, определяющий положение транспортных средств – наиболее распространённого типа объектов окружения в городских условиях – по изображению с одной монокулярной камеры и представляющий результат произвольно ориентированными ограничивающими рамками на виде сверху (bird’s-eye view). Метод выполняется в два этапа. Первый этап вычисляет проекцию видимой границы ТС на вид сверху, опираясь на двумерные детекции (2D-детекции) препятствий и семантическую сегментацию проезжей части на изображении; полученная проекция трактуется как зашумлённые измерения двух ортогональных сторон ТС. Второй этап строит вокруг этой проекции ориентированную ограничивающую рамку; для него предложен новый алгоритм, опирающийся на предположение об L-образности проекции, – L-Shape алгоритм.

2.1.1 Этап 1. Проекция двумерной детекции изображения на вид сверху

Пусть имеется изображение, зарегистрированное камерой БТС, с известной внутренней и внешней калибровкой. На этом изображении производится двумерное детектирование ТС и семантическая сегментация проезжей части. По этим данным требуется определить положение, ориентацию и габариты обнаруженных ТС на виде сверху. Далее описаны два подхода к определению границ окружающих ТС на плоскости движения БТС.

Наивный подход

Пусть дан набор ограничивающих рамок на зарегистрированном изображении, полученный с помощью нейросетевой модели YOLOP [105], осуществляющей детекцию ТС на изображении. Для дальнейшего определения положения обнаруженных ТС вводятся два предположения:

1. Все объекты находятся на земле (проезжей части дороги).
2. Поверхность дороги плоская. Далее в работе используется система координат, в которой уравнение плоскости, в которой лежит данная поверхность, имеет вид $z = 0$.

Для оценки трёхмерных координат ТС два нижних угла ограничивающей рамки проецируются на плоскость дороги (рисунок 2.1). При известных внутренней и внешней калибровках операция выполняется в два шага: сначала находятся однородные координаты p_u интересующих точек в системе координат камеры, затем по внешней калибровке они проецируются на плоскость движения БТС $z_l = 0$ в системе O_l, что даёт окончательный результат p_l. Поскольку алгоритмы планирования пути работают с представлением объектов в виде геометрических фигур ненулевой площади или объёма [95], полученный отрезок достраивается до прямоугольника ортогональными отрезками фиксированной длины.

Алгоритм удовлетворительно проецирует небольшие объекты, например людей, а также объекты в центре кадра, ориентированные вдоль оптической оси камеры. Однако для крупного объекта, границы которого не параллельны осям изображения, проекция оказывается неверной: занимаемая объектом область завышается, и алгоритм планирования движения может принять решение об остановке там, где объезд в действительности возможен (рисунок 2.2).

Уточнение положения объекта с помощью семантической сегментации проезжей части

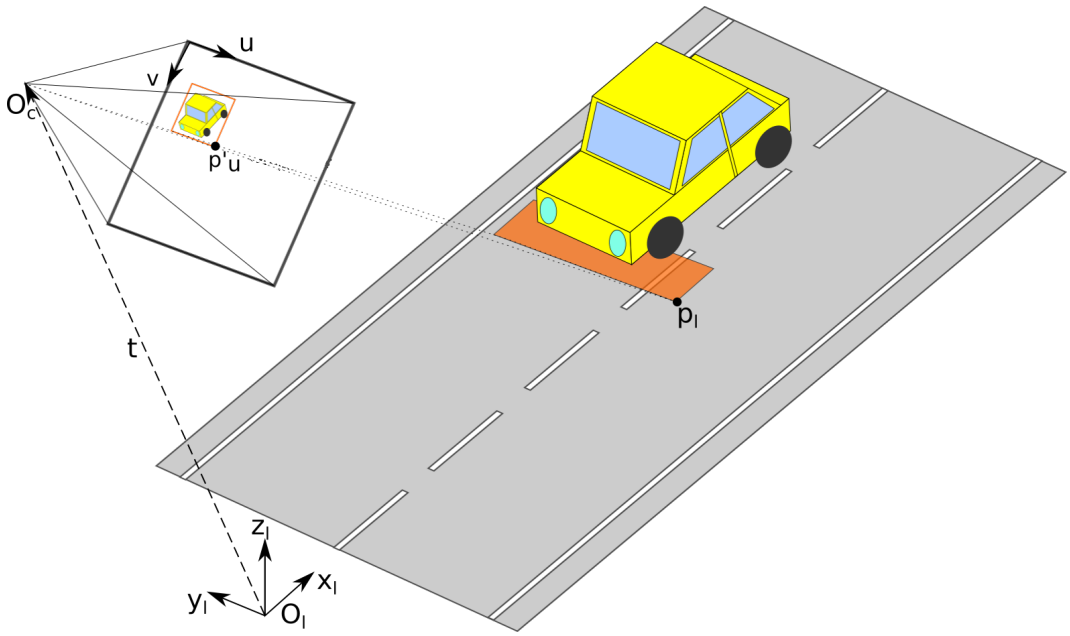


Рисунок 2.1 — Наивный подход к проецированию ограничивающей рамки в 2D на вид сверху

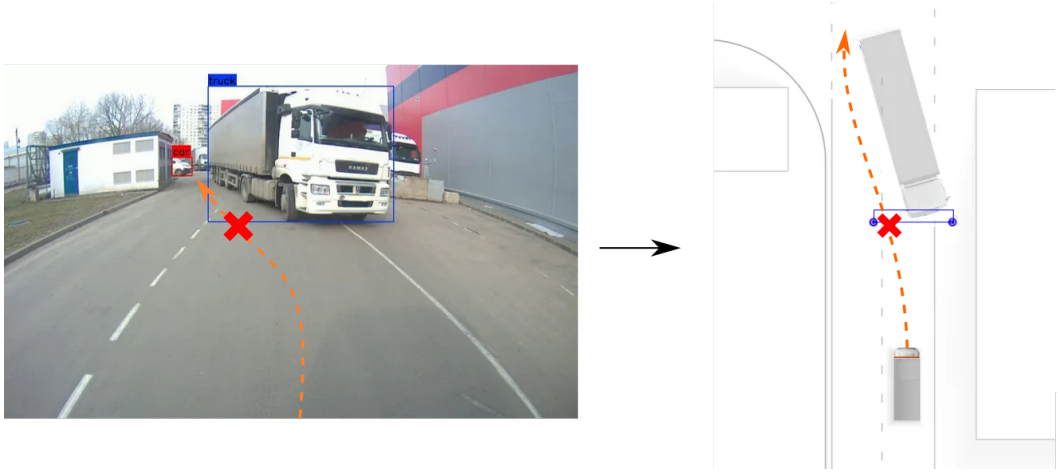


Рисунок 2.2 — Неверная проекция грузовика. Зона слева от грузовика воспринимается, как часть объекта, и мешает объехать его

Продемонстрированная проблема решается уточнением положения детектируемых объектов по семантической сегментации проезжей части, также получаемой моделью YOLOP. Уточнённый алгоритм состоит из трёх шагов:

1. Берём N равномерно распределённых точек на нижней границе ограничивающей рамки.
2. Каждая полученная точка сдвигается к верху изображения пиксель за пикселем, пока она не выйдет за пределы проезжей части либо не достигнет верхнего края ограничивающей рамки.
3. Выпуклая оболочка полученных точек проецируется на дорогу аналогично наивному подходу.

На рисунке 2.3 приведена демонстрация принципа работы алгоритма.

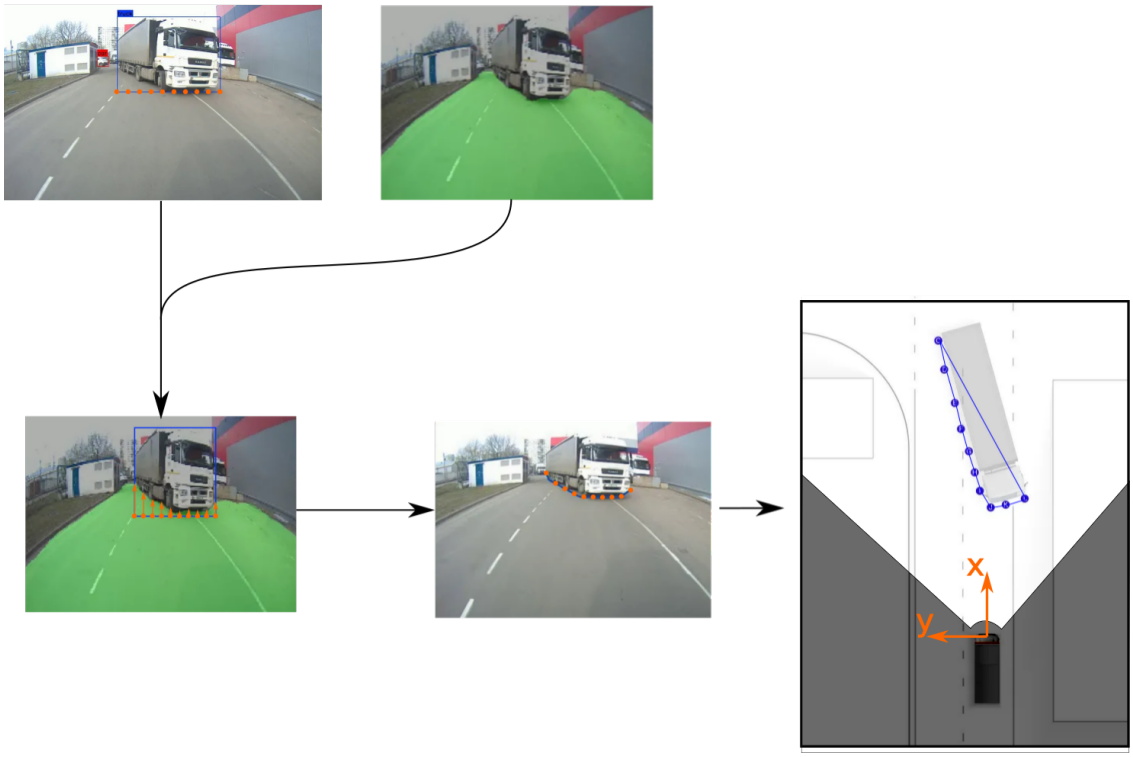


Рисунок 2.3 — Проекция двумерной ограничивающей рамки на вид сверху с учётом сегментации проезжей части

Несмотря на недостатки этого метода, включая ошибки, вызванные изменяющимся зазором между ТС и дорогой, он может быть применим как способ построения одного из кандидатов для более сложных детекторов, учитывающих результаты работы нескольких алгоритмов, как это сделано в работе [97].

2.1.2 Этап 2. Вписывание проекции объекта на виде сверху в ориентированную ограничивающую рамку

Полученная на первом этапе проекция вписывается далее в ориентированную рамку на плоскости дороги. Такое представление удобно потребителям трёхмерных детекций – ряду алгоритмов планирования пути и большинству алгоритмов отслеживания объектов. Кроме того, оно компенсирует систематическое занижение занимаемой объектом области: из-за окклюзии часть детектируемого ТС не наблюдается, и проекция покрывает лишь долю фактически занятого им пространства.

Для построения ограничивающей рамки вводятся дополнительные предположения об L-образной форме полученной проекции:

1. Вписываемые в рамку точки – зашумлённые точки, принадлежащие двум смежным и ортогональным сторонам ТС.
2. Точки упорядочены таким образом, что сначала идут точки, принадлежащие одной стороне ТС, затем точки второй стороны.

Далее в работе данная модель именуется L-Shape моделью. Согласно ей существует неизвестный заранее номер $m \in \{1, \dots, N_p - 1\}$, где N_p – число вписываемых точек, разделяющий эти точки между двумя перпендикулярными прямыми $l : y = kx + d_1$ и $l' : x = -ky + d_2$: точки $p_1, \dots, p_m$ отнесены к прямой l, точки $p_{m+1}, \dots, p_{N_p}$ – к прямой l'. Нахождение ограничивающей рамки сводится к поиску методом наименьших квадратов (МНК) таких прямых, которые минимизируют сумму квадратов отклонения (Residual Sum of Squares, RSS) вписываемых точек от соответствующей прямой. Тем самым ориентация рамки определяется не геометрическим построением на выпуклой оболочке вписываемых точек, как в рассматриваемых далее алгоритмах вписывания, а оценкой параметров модели по всем точкам. При фиксированном m задача минимизации приводится к стандартному виду $\operatorname{argmin}_a (Xa - y)^2$, где $a = (k, d_1, d_2)^T$ – вектор искомых параметров, а строки матрицы X и столбца y составлены из координат вписываемых точек в зависимости от того, к какой из двух прямых точка отнесена (полный вид X и y приведён в приложении А). Решением данной задачи минимизации является:

$$a^* = \left(X^T X\right)^{-1} X^T y \tag{2.1}$$

$$\text{RSS} = a^{*T} X^T X a^* - 2a^{*T} X^T y + y^T y \tag{2.2}$$

Наивная реализация алгоритма вписывания перебирает все значения m, для каждого из них заново составляет матрицы X и y и вычисляет a^* и RSS по формулам (2.1), (2.2). Решением для l, l' служит a^* с наименьшей RSS, а искомой рамкой – рамка минимального размера, включающая все вписываемые точки, стороны которой параллельны l и l'. Составление матриц и вычисление по этим формулам занимают $\Theta(N_p)$ операций и повторяются $N_p - 1$ раз, что даёт итоговую сложность $\Theta(N_p^2)$.

Алгоритм можно оптимизировать, снизив сложность до линейной $\Theta(N_p)$. Как показано в приложении А, все элементы матриц $X^T X$ и $X^T y$ выражаются через семь сумм координат вписываемых точек, а при переходе от m к $m + 1$

каждая из этих сумм пересчитывается за $\Theta(1)$ операций: очередная точка «перемещается» с одной прямой на другую, и её вклад вычитается из одной суммы и добавляется в другую. Зная эти суммы, вычисление (2.1), (2.2) также занимает не зависящее от N_p число операций, а итоговая временная сложность составляет $\Theta(N_p)+(N_p-1)\Theta(1) = \Theta(N_p)$. Алгоритм, вычисляющий ориентированную ограничивающую рамку описанным способом, далее именуется L-Shape алгоритмом.

2.2 Исследование точности алгоритма проекции визуальных детекций на основе модели L-Shape

Для оценки качества второго этапа алгоритма он был сравнён с подходом, минимизирующим площадь вычисляемой рамки, а также с алгоритмами построения рамок для кластеров точек из статьи [97], адаптированными под проекции на вид сверху.

2.2.1 Сравниваемые алгоритмы

Для сравнения работы L-Shape алгоритма были взяты пять альтернативных подходов к решению задачи вписывания полигона в ориентированную ограничивающую рамку, названных в разделе 1.7.1: построение рамки минимальной площади [96] и четыре подхода из работы Ванга [97] – Rotating PCA, Diagonal PCA, Longest Diagonal и Rotated Triangle. Последние были разработаны для трёхмерной детекции в виде ориентированных ограничивающих параллелепипедов вокруг кластеров облаков точек: точки кластера проецировались на вид сверху, для полученной проекции строилась выпуклая оболочка, и далее выполнялось её вписывание в ориентированную рамку на виде сверху. Так как в качестве входа алгоритма использовались не кластеры точек, а проекции объектов на вид сверху, полученные на первом этапе, эти подходы были адаптированы под такие входные данные. Отметим также, что в работе [97] получаемые перечисленными подходами детекции затем передавались в нейросетевую модель для создания итоговой трёхмерной детекции; в настоящей работе эта модель не рассматривается, по-

скольку согласно требованиям, сформулированным в разделе 1.8, исследуются вычислительно эффективные алгоритмы, не требующие инференса крупных нейросетевых моделей для каждого источника данных.

Построение рамки минимальной площади. Тривиальным решением задачи является построение рамки наименьшей площади, включающей все вписываемые точки; как отмечено в разделе 1.7.1, для двумерного случая существует алгоритм с линейной по числу вершин выпуклой оболочки временной сложностью [96]. Алгоритм опирается на то, что хотя бы одна сторона выпуклой оболочки лежит на одной из сторон рамки, и потому правильно определяет положение ТС только тогда, когда среди точек оболочки есть хотя бы по одной точке каждой стороны ТС. На практике редко удаётся получить достаточное количество точек даже двух сторон, не говоря уже о трёх. В таком случае алгоритм выдаёт неверное положение объекта (рисунок 2.4), что может привести к нежелательным остановкам БТС или, что хуже, к неправильно спланированной траектории объезда препятствия. Несмотря на этот недостаток, метод применим для порождения одного из кандидатов в составе более сложных детекторов, объединяющих результаты нескольких алгоритмов [97].

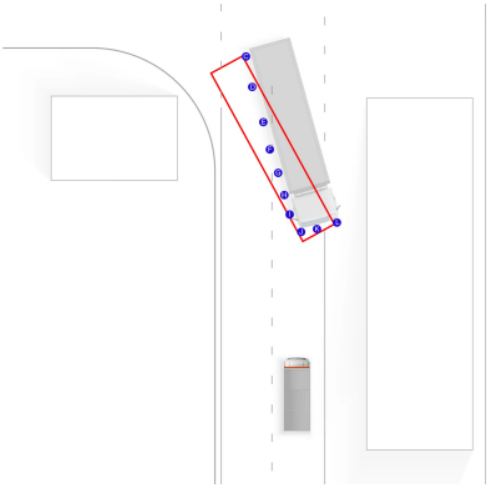


Рисунок 2.4 — Пример неправильно построенной ограничивающей рамки

Алгоритмы построения рамок для кластеров точек, адаптированные под проекции на вид сверху. Из статьи Ванга [97] были взяты и адаптированы четыре алгоритма вписывания полигона в ориентированную ограничивающую рамку, различающиеся способом определения ориентации рамки (таблица 3). Во всех четырёх случаях после определения ориентации рамка расширяется так, чтобы заключить в себе все точки выпуклой оболочки.

Алгоритм	**Способ определения ориентации рамки**
Rotating PCA	Стороны рамки параллельны собственным векторам, полученным анализом главных компонент множества точек оболочки.
Diagonal PCA	Собственный вектор, соответствующий наибольшему собственному значению, задаёт направление диагонали: выбирается пара точек оболочки, образующая наиболее близкую к нему прямую, а наиболее удалённая от этой прямой точка принимается за угол рамки.
Longest Diagonal	Схож с Diagonal PCA, но диагональ строится по паре наиболее удалённых друг от друга точек оболочки, а собственный вектор не используется.
Rotated Triangle	Перебираются пары соседних вершин оболочки. Для каждой линии, образованной парой, находится наиболее удалённая от неё точка. Пара, образующая с найденной точкой треугольник наибольшей площади, принимается за принадлежащую одной из сторон рамки.

Таблица 3 — Алгоритмы вписывания полигона в ориентированную ограничивающую рамку из работы Ванга [97], адаптированные под проекции на вид сверху

2.2.2 Набор данных и методика эксперимента

Сравнение алгоритмов выполнялось на наборе данных, полученном с БТС. Данный набор включает в себя реальные сцены из четырёх различных локаций (складов открытого типа и закрытых территорий) и содержит 12 последовательностей кадров, снятых в разные сезоны и время суток. Суммарный размер набора составил 93 кадра. Примеры сцен из набора приведены на рисунке 2.5. Каждая сцена, помимо RGB-изображений с монокулярной камеры содержит облака точек, полученные с двух 32-лучевых лидаров, в которых были вручную аннотированы (обрамлены ограничивающими параллелепипедами) все автомобили и пешеходы. Проекции полученных таким образом ограничивающих параллелепипедов автомобилей на вид сверху были взяты в качестве эталонных положений объектов.

На изображениях набора запускался первый этап алгоритма, вычисляющий проекцию препятствия на вид сверху. Для сопоставления полученных проекций с эталонными данными вычислялся коэффициент Жаккара [106] для каждой пары проекция–эталон, после чего применялся Венгерский алгоритм [107]. Для каждой



Рисунок 2.5 — Примеры сцен из набора данных, использованного для сравнения алгоритмов

полученной проекции затем запускались все сравниваемые алгоритмы второго этапа, и для результирующего положения препятствия измерялся коэффициент Жаккара. Примеры полученных проекций, построенных ограничивающих рамок и эталонных положений представлены на рисунке 2.6.

2.2.3 Результаты экспериментов

В таблице 4 представлена статистика измеренных коэффициентов Жаккара для каждого алгоритма. Алгоритм L-Shape превосходит по качеству остальные исследуемые алгоритмы; худшие результаты показало построение рамки минимальной площади.

Само значение среднего коэффициента Жаккара, однако, сравнительно невелико и для L-Shape алгоритма. Основная причина – ограниченная применимость предположения о плоскости поверхности дороги вместе с отклонением фактической внешней калибровки камеры от измеренной (раздел 1.7.1): предполагаемая плоскость дороги отходит от её действительной поверхности, и для удаляющихся препятствий ошибка накапливается. Дополнительный вклад вносит первый этап алгоритма: погрешность вычисления проекции объекта на вид

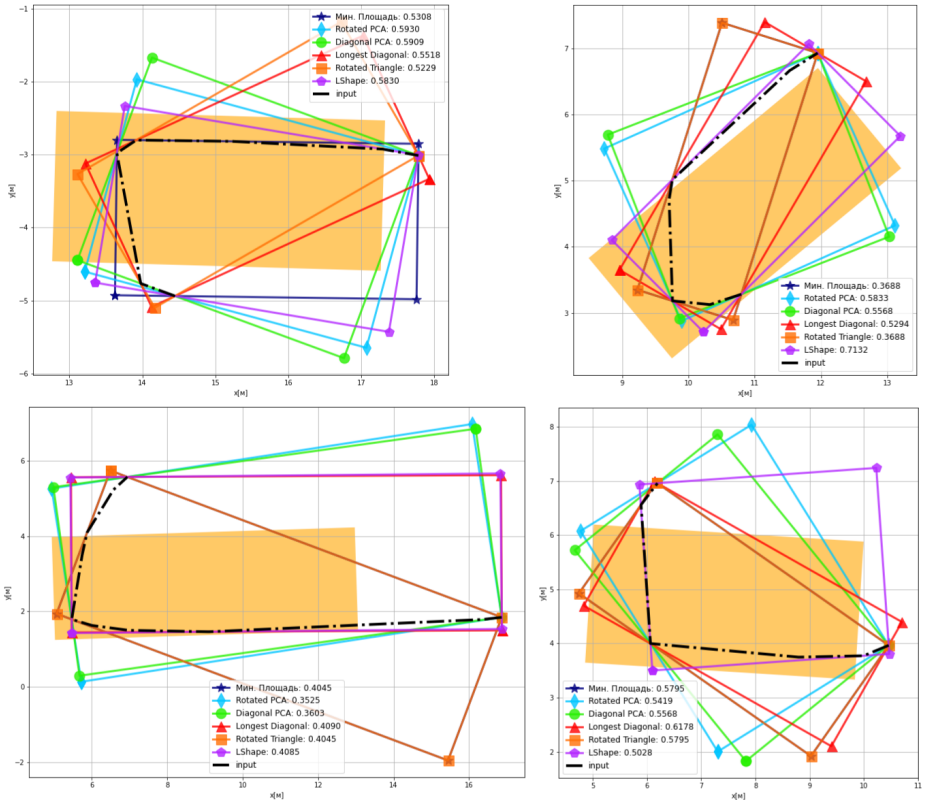


Рисунок 2.6 — Примеры работы сравниваемых алгоритмов построения ограничивающих рамок

сверху растёт с расстоянием из-за линейной перспективы изображения. Чтобы учесть это, рабочее расстояние алгоритмов было ограничено: препятствия дальше фиксированного порога не учитывались. Зависимость среднего коэффициента Жаккара от этого порога представлена на рисунке 2.7. Качество предсказываемого положения препятствия начинает падать после 10 метров для всех рассматриваемых алгоритмов, при этом L-Shape алгоритм сохраняет первенство по измеряемой метрике при всех значениях порога.

2.3 Алгоритм проекции визуальных детекций на плоскость движения БТС на основе комплексирования визуальных и лидарных данных

Одним из фундаментальных ограничений описанного в прошлом разделе L-Shape алгоритма является предположение о том, что поверхность проезжей части лежит в плоскости. Как отмечено в разделе 1.7.1, в реальном мире это предположение не выполняется ни в силу рельефа местности, ни согласно требованиям

Характеристика	**Мин. площадь**	**Rotating PCA**	**Diagonal PCA**	**Longest Diagonal**	**Rotated Triangle**	**L-Shape**
Среднее	0,294	0,328	0,321	0,323	0,301	**0,337**
Среднеквадратичное отклонение	0,146	0,144	0,149	0,156	0,141	0,141
Минимум	0,059	0,075	**0,078**	0,074	0,059	**0,078**
25 %	0,198	0,219	0,202	0,199	0,208	**0,230**
50 %	0,257	0,316	0,316	0,287	0,264	**0,328**
75 %	0,383	0,448	0,445	0,442	0,425	**0,451**
Максимум	0,649	0,607	0,622	0,651	0,579	**0,713**

Таблица 4 — Статистики полученных коэффициента Жаккара для сравниваемых алгоритмов. Наилучшие значения выделены полужирным

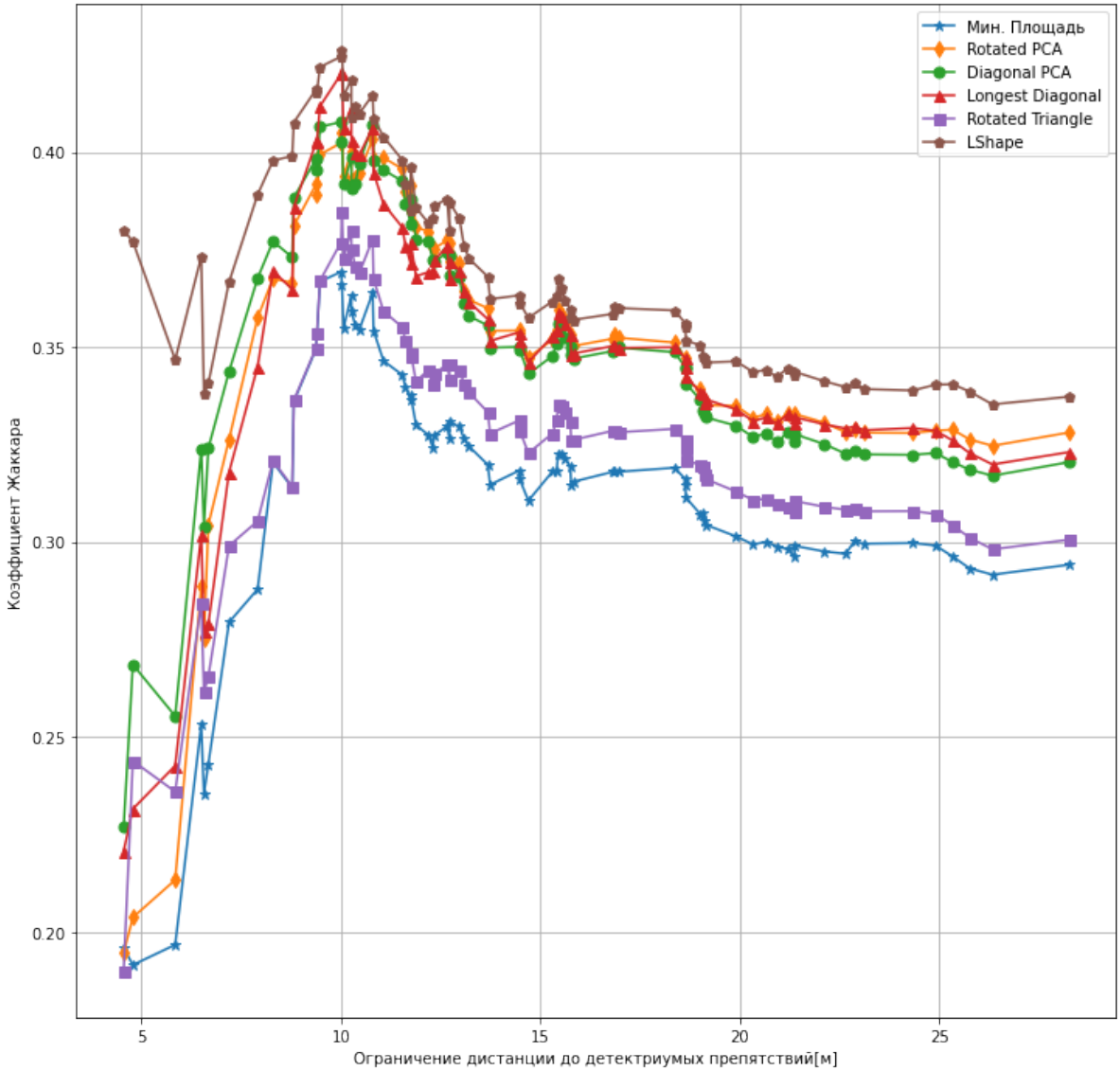


Рисунок 2.7 — График зависимости среднего коэффициента Жаккара от ограничения дистанции до детектируемых препятствий

к конструкции дорожного покрытия [84], а вносимая им погрешность растёт с удалением от камеры. Примеры оформления поперечного профиля дороги приведены на рисунке 2.8.

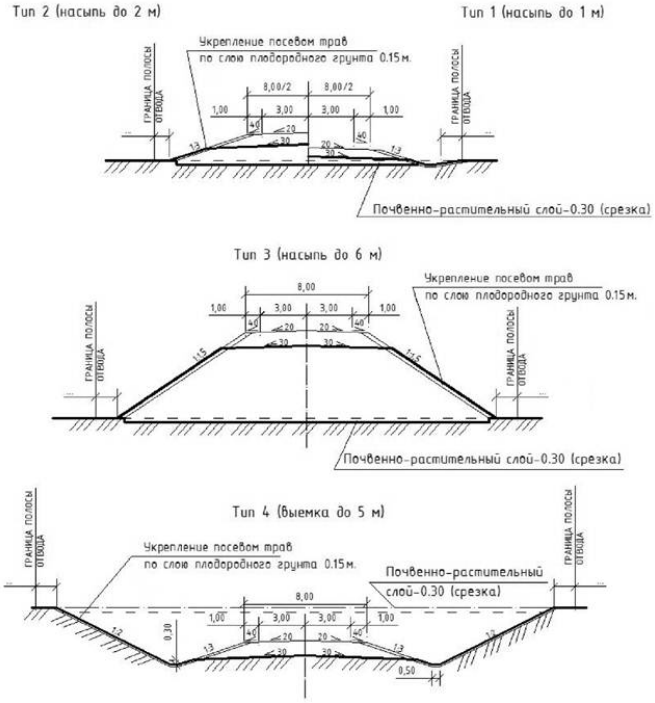


Рисунок 2.8 — Примеры оформления поперечного профиля из ГОСТ 21.701-2013 [108]

Для преодоления данного ограничения необходимо добавить источник информации, позволяющий оценить расстояние до наблюдаемых объектов без использования упомянутого предположения. В качестве такого источника информации были использованы механические лидары (лазерные дальномеры), установленные на БТС. Был разработан алгоритм комплексирования данных, полученных с камеры и лидаров, для оценки положения окружающих БТС объектов. Облако точек лидара представляет собой набор точек в трёхмерной системе координат лидара: $P_s = \{(x_i, y_i, z_i), i \in \{1, \ldots, N_s\}\}$, где s – идентификатор лидара, N_s – количество зарегистрированных им точек. Все облака точек, а также изображения, полученные камерой, имеют временную метку момента регистрации датчиком. Сенсоры работают с фиксированной частотой и синхронизированы друг с другом. Это позволяет однозначно сопоставить данные из последовательности измерений с датчиков в единые «кадры», описывающие окружающую БТС сцену в конкретный момент времени. Далее будем считать, что все облака точек с каждого лидара:

1. Приведены в локальную систему координат БТС O_l.

2. Точки, попадающие на БТС, удалены, что возможно при известной геометрии БТС.
3. Положение точек скорректировано и приведено к одному моменту времени, чтобы компенсировать собственное движение БТС во время регистрации облака точек.
4. После предыдущих манипуляций облака объединены в одно общее облако точек P.

После этих преобразований, один «кадр» представляет собой пару изображение–облако точек. В Листинге 1 представлен алгоритм, осуществляющий комплексирование данных одного кадра для оценки положения препятствий в окружающей БТС сцене: из облака выделяется подмножество точек, не принадлежащих земле, производится объединение близко расположенных точек в кластеры, которые затем сопоставляются с двумерными детекциями, найденными на изображении. Наконец, кластеры, которым нашлось сопоставление среди детекций, дополнительно фильтруются от точек, проецируемых за пределы соответствующей детекции, и находится ограничивающая рамка, описанная вокруг полученного набора точек. На рисунке 2.9 показан результат работы представленного алгоритма на данных, полученных с датчиков БТС. Отметим, что шаг 3 алгоритма 1 был реализован в виде алгоритма с использованием параллельных вычислений на платформе CUDA (англ. Compute Unified Device Architecture) [109]. Данная реализация была предложена в работе Нгуена [110].

2.3.1 Разделение облака точек на точки земли и препятствий

Отметим нетривиальность шага 2 алгоритма 1. Зачастую такое разделение производится поиском с помощью RANSAC плоскости, хорошо описывающей часть точек в облаке. Однако такой подход возвращает к предположению о совпадении поверхности дороги с некоторой плоскостью, от которого предлагаемый алгоритм и призван уйти. Поэтому в рамках данного алгоритма используется подход, не опирающийся на поиск плоскостей. Сначала выполняется отбор точек, заведомо принадлежащих земле. Затем решается задача регрессии: требуется найти зависимость высоты земли z_l от координаты в плоскости движения БТС (x_l, y_l). В качестве обучающей выборки используются отобранные на первом шаге точ-

Input: I – изображение с RGB-камеры, P – облако точек.

Output: b_i – ориентированные ограничивающие рамки на виде сверху окружающих БТС объектов.

1 Произвести поиск двумерных детекций объектов на I: $\{d_1, \dots d_n\}$;

2 Выделить в облаке точек P точки, не принадлежащие поверхности земли: P_o;

3 Кластеризовать полученное облако точек P_o с расстоянием Евклида: $P_o = \bigcup_{i=1}^{l} C_i; C_i \cap C_j = \varnothing, i \neq j$;

4 Спроецировать все кластеры на изображение I: $C_i \rightarrow \widehat{C}_i$. Все точки, проецируемые за границы I, а также находящиеся позади оптического центра камеры отбрасываются. Пустые после проекции кластеры – удаляются;

5 Для каждого спроецированного кластера найти ограничивающую рамку минимального размера: d_{C_i};

6 Используя Венгерский алгоритм найти наилучшее сопоставление найденных детекций d_i с построенными ограничивающими рамками d_{C_i}, максимизирующее суммарный коэффициент Жаккара. Не сопоставленные кластеры – удаляются.;

7 В оставшихся кластерах удалить точки, не попадающие при проекции в сопоставленную детекцию d_i: $C_i \rightarrow C_i^-$.;

8 **foreach** C_i^- **do**

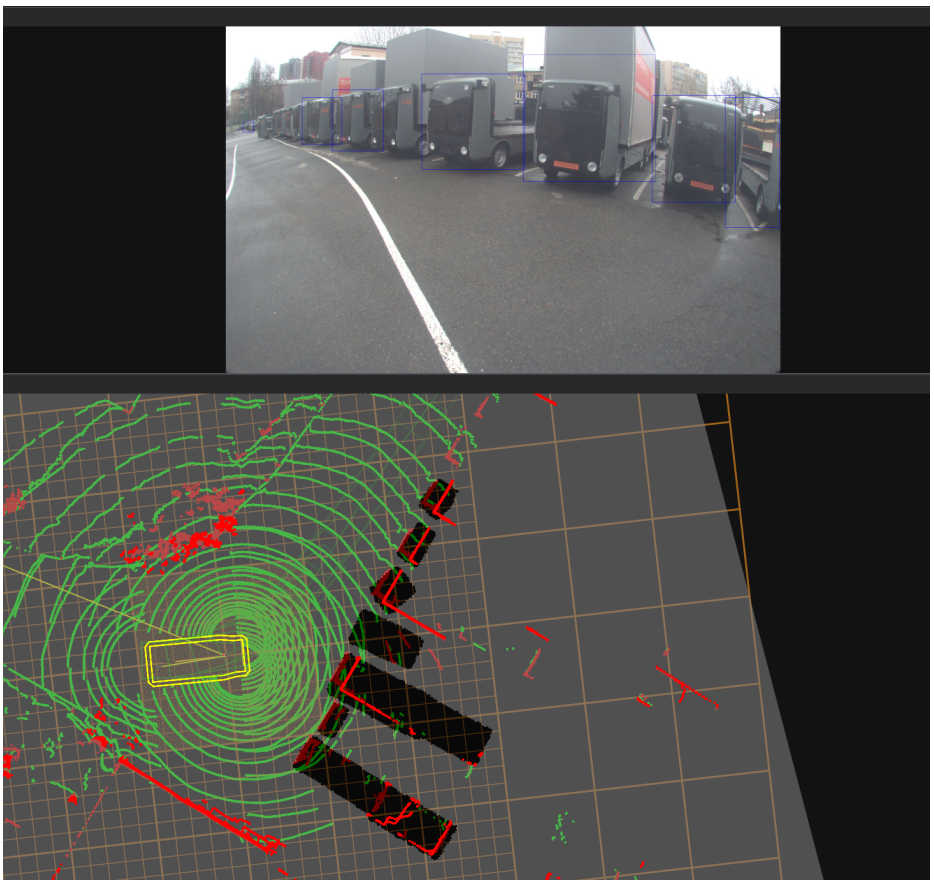
9 Используя RANSAC(RANdom SAmple Consensus) [82] найти прямую l_i, хорошо описывающую большую часть точек «очищенного» кластера C_i^-.;

10 Аналогично L-Shape алгоритму построить ограничивающую рамку b_i, со сторонами параллельными l_i и ортогональной ей l_i'.;

11 **end**

12 **return** $\{b_i\}$

Алгоритм 1: Алгоритм определения положения объектов на основе комплексирования данных с камеры и лидаров



На верхнем изображении с камеры БТС прямоугольниками выделены двумерные детекции, полученные моделью YOLOP. На нижнем красными точками отмечены точки, зарегистрированные лидаром и классифицированные как точки препятствия. Полученные алгоритмом детекции показаны в виде чёрных прямоугольников

Рисунок 2.9 — Демонстрация работы алгоритма 1 на данных, полученных с БТС

ки. В качестве тестовых точек берутся координаты (x_l, y_l) входного облака точек. Задача регрессии решается на основе гауссовских процессов (Gaussian Process Regression, GPR) [88].

Гауссовский процесс – байесовский непараметрический подход к аппроксимации функции, в данном случае функции $f : (x_l, y_l) \to z_l$, не требующий ни плоскостной, ни кусочно-плоскостной модели поверхности. Процесс полностью определяется средним значением $m(\mathbf{x})$ и ковариационной функцией (ядром) $k(\mathbf{x}, \mathbf{x}')$, где $\mathbf{x} = (x_l, y_l)$. В качестве ядра выбрано ядро Матерна $(\nu = 3/2)$ [111], которое лучше справляется с аппроксимацией в точках разрыва функции (рисунок 2.10); такие разрывы часто присутствуют в реальных сценах вокруг БТС, например в местах соединения проезжей части и тротуаров.

$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \left(1 + \frac{\sqrt{3}d}{\ell_k}\right) \exp\left(-\frac{\sqrt{3}d}{\ell_k}\right), \tag{2.3}$$

где $d = \|\mathbf{x} - \mathbf{x}'\|$ – евклидово расстояние между точками, σ_f^2 – дисперсия функции, а ℓ_k – гиперпараметр масштаба длины.

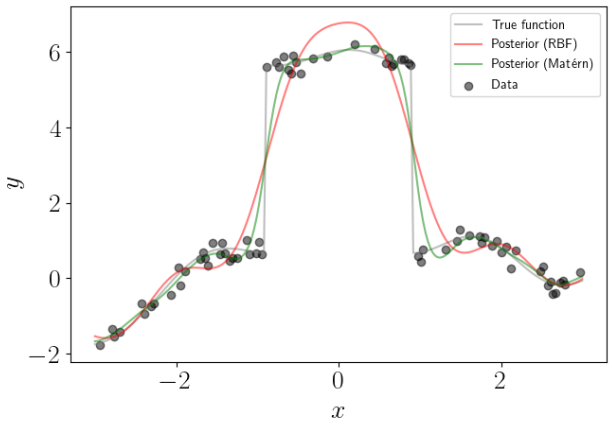


Рисунок 2.10 — Сравнение регрессии на основе гауссовских процессов с использованием в качестве ядра радиальной базисной функции и ядра Матерна [112]

Для обучающей выборки размером N_t с входами $\mathbf{X} = \{\mathbf{x}_i\}_{i=1}^{N_t}$ и наблюдениями высоты $\mathbf{z} = \{z_i\}_{i=1}^{N_t}$ задача сводится к вычислению апостериорного распределения значений z_* в N_* тестовых точках $\mathbf{X}_* = \{\mathbf{x}_{*_i}\}_{i=1}^{N_*}$. Обозначив ковариационные матрицы

$$K = [k(\mathbf{x}_i, \mathbf{x}_j)]_{i,j=1}^{N_t}, \quad K_* = [k(\mathbf{x}_*, \mathbf{x}_i)]_{i=1}^{N_t}, \quad K_{**} = [k(\mathbf{x}_{*_i}, \mathbf{x}_{*_j})]_{i,j=1}^{N_*}, \tag{2.4}$$

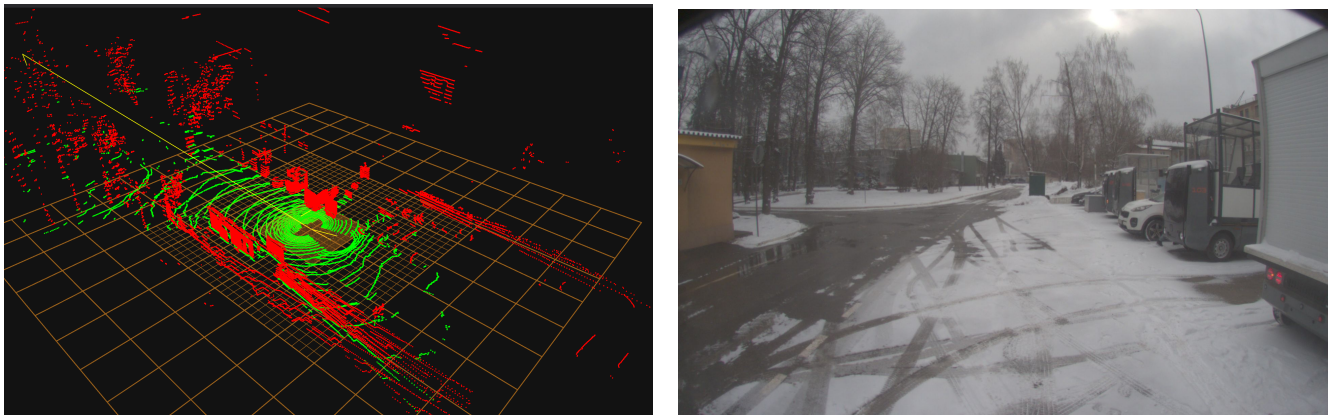
получаем, что условное распределение предсказаний $p(z_* \mid \mathbf{X}, \mathbf{z}, \mathbf{x}_*)$ является гауссовским с параметрами

$$\widehat{z} = \mathbb{E}[z_*] = K_* K^{-1} \mathbf{z}, \qquad \mathrm{Var}[z_*] = K_{**} - K_* K^{-1} K_*^\top. \tag{2.5}$$

Тем самым предсказанная высота земли в любой точке (x_l, y_l) есть взвешенная сумма наблюдений обучающей выборки с учётом ковариации между точками.

Регрессия на основе гауссовских процессов вычислительно дорога: стандартные реализации требуют $O(N_t^3)$ операций, поскольку включают разложение Холецкого. В реальных данных точек земли – десятки тысяч, поэтому для работы в режиме реального времени обучающая выборка формируется иначе: вычисляются усреднённые по областям координаты точек, отнесённых к земле, и из них берётся подвыборка в несколько сотен точек.

Наконец, для каждой точки входного облака сравниваются её измеренная координата z_l и предсказанная $\widehat{z_l}$: если z_l больше $\widehat{z_l}$ на некоторый порог z_{thr}, точка считается не принадлежащей земле. Отметим, что ввиду принципа формирования лидарных данных плотность точек уменьшается при отдалении от машины, ввиду чего растёт и дисперсия предсказываемого значения $\widehat{z_l}$. Чтобы учесть этот факт, величина z_{thr} делается зависимой от расстояния до БТС: $z_{thr} = f(||(x_l, y_l)||_2)$. В Листинге 2 представлена реализация описанного алгоритма. На рисунке 2.11 представлен результат работы алгоритма.



а) Результат работы алгоритма. б) Изображение сцены с камеры БТС.

Рисунок 2.11 — Пример работы алгоритма разделения облака точек земли и препятствий

Отметим, что данный алгоритм может быть реализован с помощью параллельных вычислений, в частности используя программно-аппаратную архитектуру CUDA, что позволяет снизить время выполнения данного алгоритма.

2.4 Исследование точности алгоритма проекции визуальных детекций на основе комплексирования визуальных и лидарных данных

Для оценки предложенного алгоритма 1 было проведено его сравнение с подходами, не использующими комплексирование: с L-Shape алгоритмом, опирающимся только на визуальные данные и описанным в разделе 2.1, и с четырьмя алгоритмами вписывания в ориентированную рамку, разработанными для лидарных облаков точек.

Input: P – облако точек.

Output: P_g – облако точек, принадлежащих земле, P_o – облако точек, не принадлежащих земле

Копируем облако P в P_c;

P_c разделяется на воксели V_i – области пространства, получаемые при накладывании трёхмерной сетки фиксированного разрешения;

P_c разделяется иным образом сеткой в полярных координатах на сектора S_i;

foreach V_i **do**

    Для точек в вокселе вычисляется среднее m и среднеквадратичное отклонение σ координаты z_l в локальной системе координат БТС O_l;

    Из P_c удаляются точки с $|z_l - m| > \alpha\sigma$, где α – фиксированный параметр алгоритма;

end

foreach S_i **do**

    Находится минимальное значение z_{min} z_l точек в S_i;

    Из P_c удаляются точки, у которых $z_l > z_{min} + T$, где T – фиксированный параметр алгоритма;

end

Из вокселей V_i случайным образом выбираются N_{train} вокселей, где N_{train} – параметр алгоритма;

Для выбранных вокселей вычисляется центр масс (mx_i, my_i, mz_i) точек внутри вокселя. Полученные точки образуют множество P_{train};

Применяется регрессия на основе гауссовских процессов для оценки высоты $\widehat{z}_l$, в точках $(x_i, y_i); \forall p_i = (x_i, y_i, z_i) \in P$, используя в качестве обучающей выборки $((x_i, y_i), z_i); \forall p_i = (x_i, y_i, z_i) \in P_{\text{train}}$;

$P_g = \{\}$;

foreach $p_i = (x_i, y_i, z_i) \in P$ **do**

    $S(p_i)$ – сектор, в котором находится точка p_i;

    if $z_i < \widehat{z}_i + z_{thr}(S(p_i))$ **then**

        `//` $z_{thr}(S(p_i))$ `-- массив порогов по высоте для каждого сектора`

        $P_g += p_i$;

    end

end

$P_o = P \setminus P_g$;

return P_g, P_o;

Алгоритм 2: Алгоритм фильтрации точек земли

2.4.1 Описание эксперимента

Эксперимент выполнялся на том же наборе данных, что и сравнение в разделе 2.2. Как и прежде, эталонными положениями объектов служили проекции аннотированных параллелепипедов на вид сверху, а для сравнения алгоритмов измерялся коэффициент Жаккара для сопоставленных вычисленных и эталонных повёрнутых ограничивающих рамок на виде сверху.

Существенное отличие от эксперимента раздела 2.2 состоит во входных данных алгоритмов Rotating PCA, Diagonal PCA, Longest Diagonal и Rotated Triangle. Там они применялись к проекциям нижнего контура объектов, найденных на изображении, поскольку исследовались алгоритмы, использующие исключительно визуальные данные. Здесь же комплексирование даёт лидарные кластеры (шаг 6 алгоритма 1), для работы с которыми эти алгоритмы и разрабатывались [97], поэтому они применялись к выпуклой оболочке кластеров. Отсюда следует, что поэтапное сравнение с разделом 2.2 некорректно: у одного и того же алгоритма вписывания здесь и там различается вход. Сквозное сравнение при этом корректно, поскольку все рассматриваемые конвейеры оцениваются целиком – от сырых данных кадра до ориентированной ограничивающей рамки, – на одном и том же наборе эталонных данных, по одной и той же метрике и при одном и том же способе сопоставления вычисленных и эталонных рамок венгерским алгоритмом. Именно в этом смысле приводимые далее значения сопоставляются с результатами раздела 2.2.

Как и ранее для сравниваемых алгоритмов было проведено исследование зависимости среднего коэффициента Жаккара от максимальной дистанции до детектируемых объектов.

Сравниваемые алгоритмы реализованы на языке Python. Помимо этого, предложенный алгоритм реализован на языке C++ в виде узла ROS (англ. Robot Operating System) [113] и интегрирован в систему управления БТС, что позволило проверить его применимость в режиме реального времени.

2.4.2 Результаты экспериментов

В таблице 5 приведены статистики измеренных коэффициентов Жаккара. Предложенный алгоритм комплексирования показывает средний коэффициент Жаккара $0{,}439$ против $0{,}337$ у L-Shape алгоритма, использующего только визуальные данные, что соответствует росту на $30{,}3$ %. Лучший из подходов, использующих исключительно лидарные данные, – Diagonal PCA – достигает $0{,}399$, то есть предложенный алгоритм превосходит и его примерно на 10 %.

Характеристика	**Rotating PCA**	**Diagonal PCA**	**Longest Diagonal**	**Rotated Triangle**	**Предло-женный**	**L-Shape**
Используемые данные	лидар	лидар	лидар	лидар	лидар и камера	камера
Среднее	0,396	0,399	0,335	0,339	**0,439**	0,337
Среднеквадратичное отклонение	0,222	0,222	0,188	0,215	0,256	**0,141**
Минимум	0,008	0,007	0,004	0,009	0,004	**0,078**
25 %	0,212	0,209	0,187	0,175	0,219	**0,230**
50 %	0,400	0,399	0,331	0,330	**0,423**	0,328
75 %	0,562	0,566	0,457	0,439	**0,652**	0,451
Максимум	0,872	0,871	0,734	0,915	**0,925**	0,713

Таблица 5 — Статистики полученных коэффициентов Жаккара для предложенного алгоритма и алгоритмов, не использующих комплексирование. Наилучшие значения выделены полужирным

Вместе с тем по минимальному значению и по нижнему квартилю предложенный алгоритм уступает L-Shape алгоритму: $0{,}004$ против $0{,}078$ и $0{,}219$ против $0{,}230$ соответственно. Причина в том, что качество комплексирования зависит от количества лидарных точек, попавших на объект. Для удалённых либо существенно заслонённых объектов кластер вырождается в несколько точек, и построенная по нему рамка может грубо расходиться с эталоном. В то же время L-Shape алгоритм строит рамку по проекции нижней границы объекта на плоскость движения с фиксированным количеством точек, что даёт устойчивую, хотя и невысокую в среднем, оценку.

Зависимость среднего коэффициента Жаккара от ограничения дальности до детектируемых препятствий представлена на рисунке 2.12. Предложенный алгоритм превосходит остальные во всём диапазоне рассмотренных дальностей.

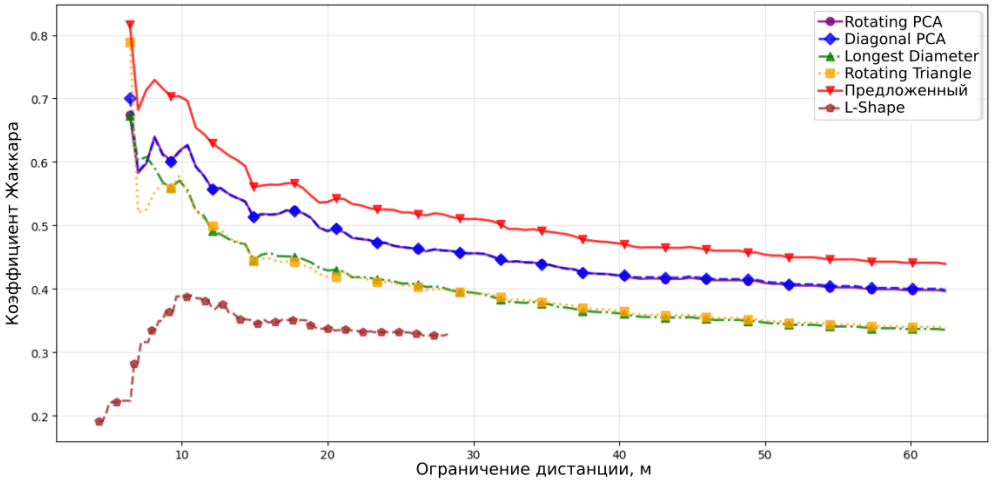


Рисунок 2.12 — График зависимости среднего коэффициента Жаккара от ограничения дистанции до детектируемых препятствий для алгоритма комплексирования и алгоритмов, не использующих комплексирование

2.4.3 Вычислительная эффективность

Измерение времени работы производилось для реализации предложенного алгоритма на языке C++ на вычислительной платформе с процессором AMD Ryzen 5 5600X, ОЗУ объёмом 32 ГБ и ГПУ Nvidia GeForce RTX 3070. Среднее время обработки одного кадра составило $66{,}7$ мс, что удовлетворяет условию 1 раздела 1.7.2: при типичной частоте регистрации данных сенсорами БТС 10 Гц кадр обрабатывается до поступления следующего. Объём занимаемой памяти составил $178{,}8$ МБ ОЗУ и 156 МБ памяти ГПУ.

Время отдельных этапов алгоритма, измеренное с помощью библиотеки инструментального профилирования Tracy [114], приведено на рисунке 2.13. Основную долю времени занимают шаги 4 и 5 алгоритма 1, в которых в текущей реализации дважды выполняется проецирование точек кластеров на плоскость изображения с коррекцией дисторсии. Эта операция выполняется на ЦПУ, хотя, как отмечено в разделе 2.3, хорошо поддаётся распараллеливанию на ГПУ, что оставляет запас для дальнейшей оптимизации.

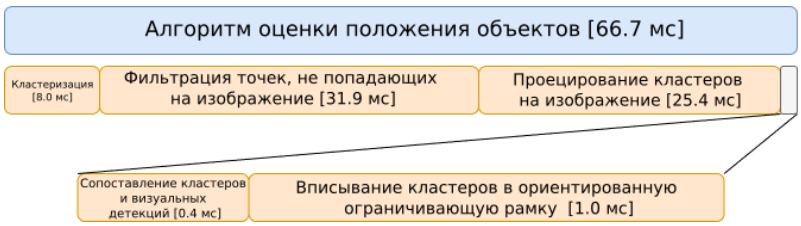
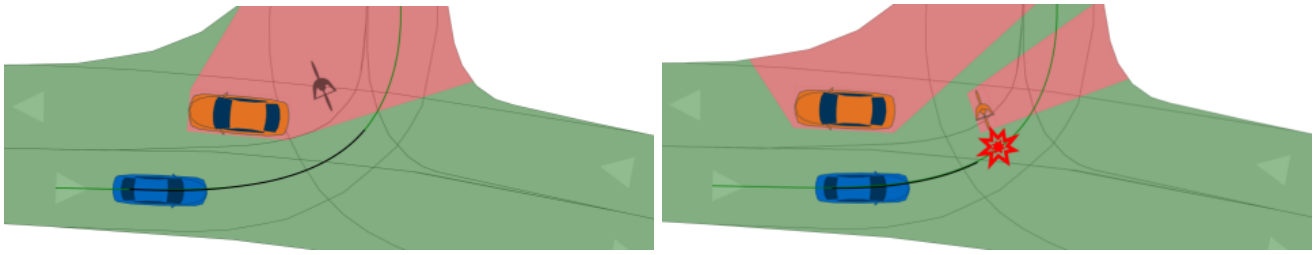


Рисунок 2.13 — Среднее время выполнения этапов алгоритма 1

2.5 Алгоритм проекции сегментации проезжей части на изображении на плоскость движения БТС

Помимо определения положения препятствий в окружающей БТС сцене важную роль в осуществлении безопасного решения задач планирования пути играет включение в модель описания областей, свободных для перемещения БТС. Это позволяет алгоритмам планирования пути отличать области, для которых достоверно известно отсутствие в них каких-либо объектов, от областей, для которых отсутствует информация, позволяющая судить об их проходимости ввиду окклюзии либо недостоверности регистрируемых данных. На рисунке 2.14 приведён пример, демонстрирующий важность этого различия. Семантическая сегментация проезжей части на изображении может быть получена моделью YOLOP либо другой моделью, осуществляющей семантическую сегментацию двумерных изображений. Однако аналогично задаче проецирования препятствий в трёхмерную систему координат БТС O_l, описанной в разделе 2.1, возникает необходимость спроецировать полученную сегментацию на вид сверху. В данном разделе предложен алгоритм, осуществляющий данное проецирование на основе комплексирования данных с камеры и лидаров, установленных на БТС.

В отличие от типичных препятствий в окружающей БТС сцене: людей, автомобилей; размеры и положение которых с высокой точностью описываются ориентированными ограничивающими рамками, проезжая часть после проекции имеет сложную геометрию (рисунок 2.15). Из-за этого векторное представление проезжей части на виде сверху может быть неудобным для построения. Вместо него используется представление в виде сетки фиксированного разрешения, где каждому элементу будет сопоставлен класс «свободно» – в случае если в соответствующей области БТС может безопасно проезжать, или «неизвестно» – в случае если информации об области недостаточно (а далее и дополнительный класс «занято» для областей, где обнаружен объект).



а) БТС (синий) не видит велосипедиста позади автомобиля. Зелёным выделена территория в поле видимости датчиков БТС, красным – области, о которых нет информации ввиду их окклюзии

б) При продолжении движения велосипедист детектируется датчиками БТС, но из-за близости объекта, у систем управления остаётся мало времени для реагирования.

Рисунок 2.14 — Демонстрация влияния окклюзии на безопасность движения БТС [115]



Рисунок 2.15 — Пример семантической сегментации проезжей части моделью YOLOP на изображении с камеры БТС

Прежде чем приступить к описанию алгоритма опишем интуицию, лежащую в его основе, без использования информации с лидаров. Воспользуемся предположением в наивном подходе проецирования препятствий из раздела 2.1 о совпадении поверхности проезжей части с плоскостью $z_l = 0$. В такой модели оценка проходимости элемента сводится к проекции центров сетки на плоскость изображения (как и прежде, внутренняя и внешняя калибровки камеры считаются известными): если при проецировании центр элемента попал на пиксель, классифицированный как проезжая часть, соответствующий элемент сетки классифицируется как «свободный», в противном случае – как «неизвестный» (рисунок 2.17 а). Отметим, что в последнем случае нельзя утверждать, что область не свободна для проезда, т.к. она может быть заслонена другим объектом – см. рисунок 2.16).

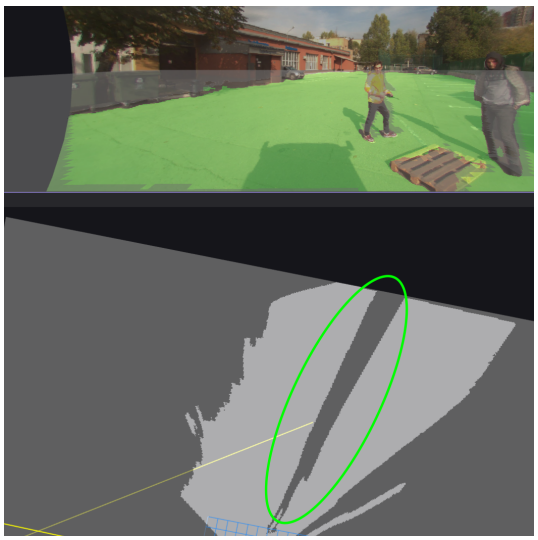


Рисунок 2.16 — Пример заслонённой области свободной для проезда

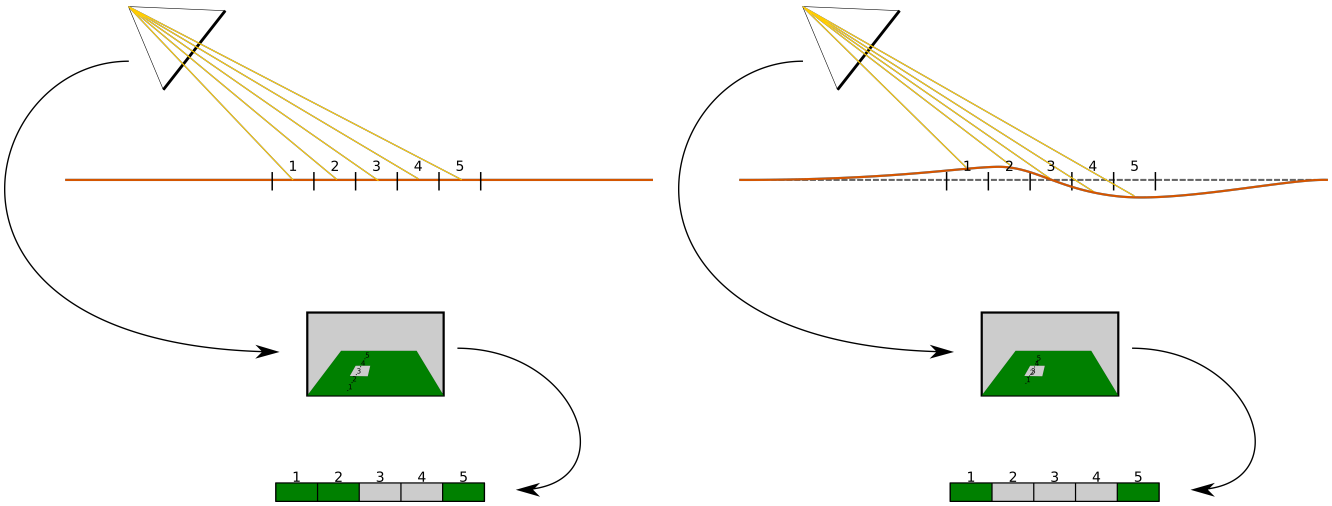
Однако выше уже показано низкое качество моделей, использующих предположение о совпадении поверхности проезжей части с плоскостью. Ослабим это предположение и обобщим алгоритм на этот случай. Будем считать, что поверхность описывается некоторой известной функцией $z_l = z(x_l, y_l)$: для этого достаточно вычислять эту координату перед проецированием центра элемента на изображение, а описанный выше случай плоскости отвечает $z \equiv 0$. Обобщённый алгоритм приведён в листинге 3, его иллюстрация – на рисунке 2.17 б. Алгоритм опирается на следующее условие: более близкие к БТС участки проезжей части не должны перекрывать обзор камеры для более удалённых участков. В терминах проецирования это означает, что проекции центров элементов сетки на плоскость изображения должны располагаться монотонно по дальности: если элемент c_1 находится ближе к БТС, чем элемент c_2, то проекция центра c_1 обязана лежать на изображении ниже проекции центра c_2. Нарушение монотонности означает, что участок поверхности заслонён более близким к БТС возвышением, и значение сегментации, взятое в точке проекции, относится не к тому элементу сетки. Условие выполняется тем надёжнее, чем выше установлена камера и чем меньше перепады высот поверхности на масштабе элемента сетки.

Input: S_I – семантическая сегментация проезжей части на изображении I; res – разрешение сетки [элементов на метр], на которую производится проецирование; w – ширина и высота сетки [м]; $z(x,y)$ – высота поверхности движения БТС

Output: S_{BEV} – проекция сегментации проезжей части на вид сверху

$s \leftarrow \frac{1}{\text{res}}$;
$L \leftarrow w \cdot \text{res}$;
$S_{BEV} \leftarrow$ матрица размера $L \times L$;
for $i \leftarrow 0$ **to** L **do**
  for $j \leftarrow 0$ **to** L **do**
    $x \leftarrow -\frac{w}{2} + js$;
    $y \leftarrow -\frac{w}{2} + is$;
    $(u; v) \leftarrow$ проекция $(x,y,z(x,y))$ на изображение (с учётом дисторсии);
    if (u,v) *попадает на изображение AND* $S_I[u][v] ==$ *«свободно»* **then**
      $S_{BEV}[i][j] \leftarrow$ «свободно»;
    else
      $S_{BEV}[i][j] \leftarrow$ «неизвестно»;
    end
  end
end
return S_{BEV};

Алгоритм 3: Алгоритм проецирования сегментации проезжей части на вид сверху



а) Проезжая часть лежит в известной плоскости $z_l = 0$.

б) Поверхность проезжей части удовлетворяет уравнению $z = z(x,y)$.

Рисунок 2.17 — Иллюстрация алгоритма проецирования сегментации проезжей части на вид сверху. Для облегчения восприятия проецируемая сетка представлена одним рядом

2.5.1 Вычисление функции $z(x,y)$

Опишем алгоритм оценки значений $z(x,y)$ в центрах сетки, на которую производится проекция сегментации. Для этого аналогично разделу 2.3 используется комплексирование данных, зарегистрированных лидарами. Как и прежде, облака точек с лидаров считаются прошедшими предварительную обработку и представлены единым облаком точек с компенсированным движением БТС во время регистрации облаков.

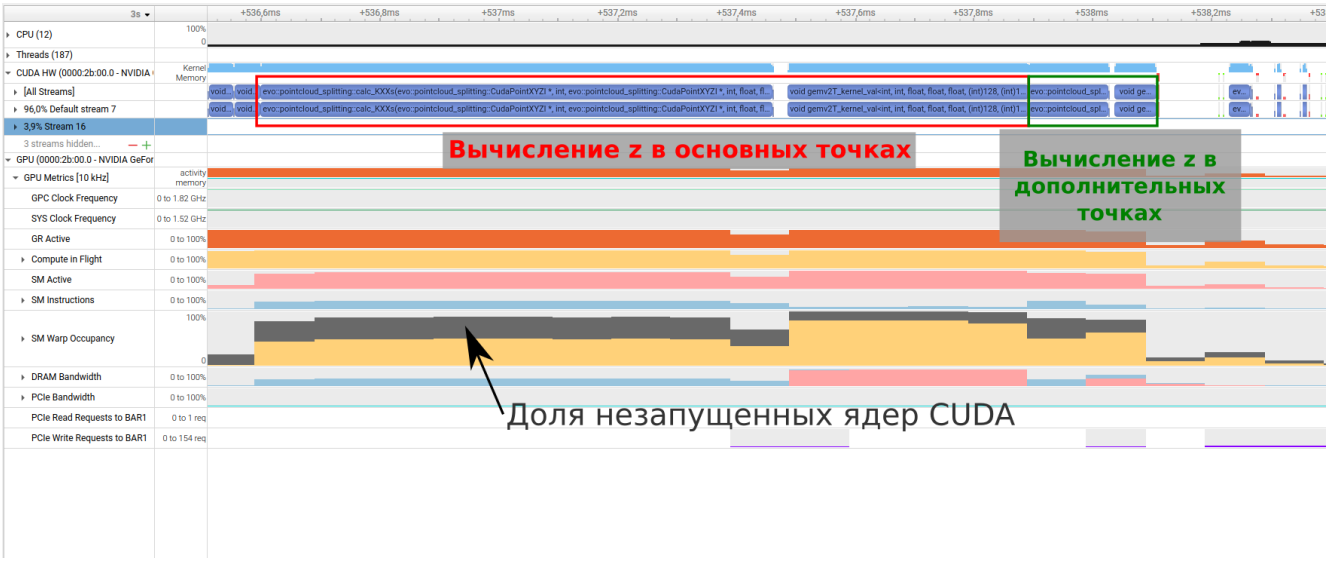
Чтобы оценить $z(x,y)$ в центрах сетки, в описанном ранее алгоритме 2 проводится отдельная итерация регрессии (шаг 14). В качестве обучающих точек используются те же точки, что и раньше, а в качестве тестовых – (x, y) центров сетки. Недостатком такого подхода станет увеличение времени работы алгоритма 2, что замедлит работу алгоритмов, использующих его результаты, в частности алгоритма 1. Однако при наличии вычислительных ресурсов на ГПУ вычисление регрессии можно распараллелить с использованием механизма мультипоточности CUDA Streams [116]. В этом случае в регрессии можно переиспользовать матрицу K из уравнения (2.4). Затем вычислить ковариационные матрицы $\widetilde{K_*}$, $\widetilde{K_{**}}$ для новых тестовых точек, и аналогично (2.5) вычислить аппроксимацию z в тестовых точках. На рисунке 2.18 представлены результаты профилирования ре-

сурсов на ГПУ с помощью Nsight Systems [117] изменившейся части программы, реализующей алгоритм 2 при последовательном и параллельном решении задач регрессии. Программа реализована на языках C++ и CUDA C++. Профилирование происходило на ПК с процессором AMD Ryzen 5 5600X с 6 ядрами и тактовой частотой 4,6 ГГц, ОЗУ объёмом 32 ГБ, а также ГПУ Nvidia GeForce RTX 3070 с графической памятью 8 ГБ и тактовой частотой 1,5 ГГц. Напомним, что обучающей выборкой регрессии служит подвыборка из нескольких сотен усреднённых по вокселям точек земли (раздел 2.3.1), а тестовыми точками – точки, в которых предсказывается высота поверхности. Здесь тестовые точки двух видов: точки входного облака, число которых составило порядка 100 000, и добавляемые к ним центры элементов сетки, число которых составило 10 000. На рисунке 2.19 представлен график сравнения распределения времени работы алгоритма разделения при последовательном и параллельном вычислениях.

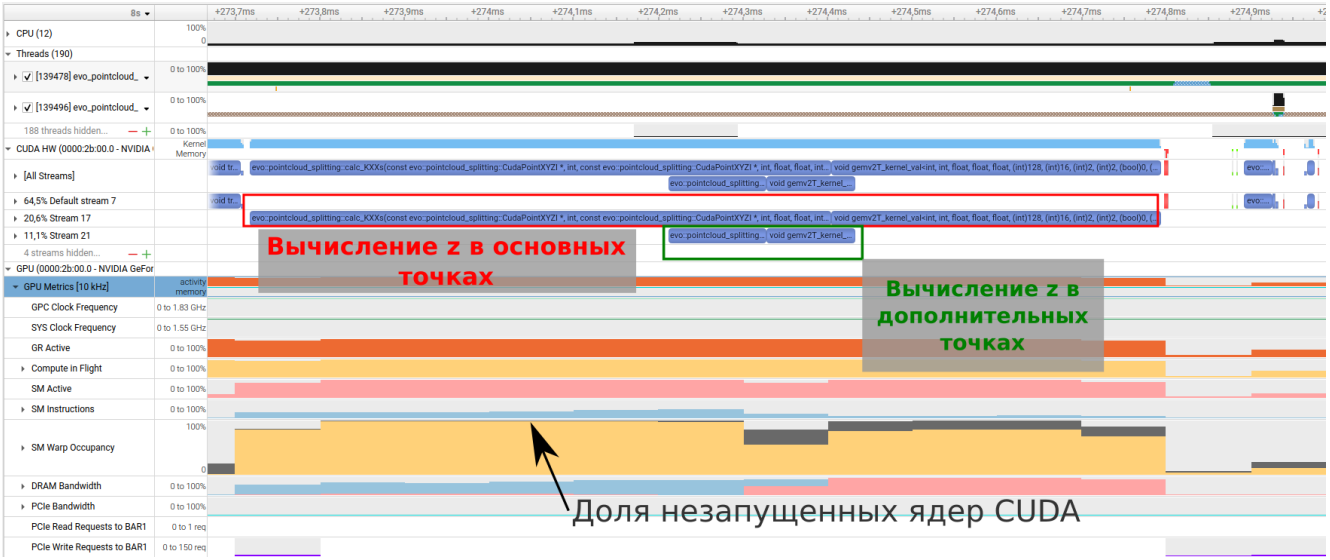
2.5.2 Комплексирование полученной проекции с базовой картой проходимости пространства

Описанный алгоритм строит фрагмент карты проходимости пространства, опираясь на семантическую сегментацию изображения. Он не является самостоятельным способом построения модели окружения: сведения о препятствиях, не относящихся к проезжей части, в нём отсутствуют. Поэтому полученная проекция должна быть объединена с картой, построенной по лидарным данным.

В качестве базового примем алгоритм построения карты проходимости пространства по объединённому облаку точек, восходящий к работам Х. Моравека и А. Элфеса [36; 38] описанным в разделе 1.3. В нём из облака выделяются точки, лежащие выше поверхности земли и не превышающие габаритов БТС. Элементам сетки, в которые проецируются эти точки, присваивается отрицательный вклад l^b_-, а элементам, лежащим на отрезке между положением лидара и зарегистрированной точкой, – положительный вклад l^b_+. Области, расположенные за наблюдаемыми препятствиями, считаются заслонёнными и сохраняют состояние «неизвестно». Учёт вкладов производится обновлением по формуле (1.2). Наконец, все элементы карты квантуются в три значения в зависимости от их



а) Без использования CUDA Streams



б) С исправленной конфигурацией запуска ядер CUDA и использованием CUDA Streams

Красной рамкой выделены вычисления аппроксимации в тестовых точках входного облака, зелёной – в добавленных тестовых точках, отвечающих центрам элементов сетки. Во вкладке «SM Warp Occupancy» жёлтым показана доля вычисляемых ядер относительно максимальной теоретической пропускной способности архитектуры в данный момент времени, серым – доля ядер, которым не хватило ресурсов для вычисления и в результате простаивающих в ожидании их появления

Рисунок 2.18 — Профилирование в Nsight Systems изменившейся части программы, реализующей алгоритм 2, при добавлении тестовых точек, отвечающих центрам элементов сетки

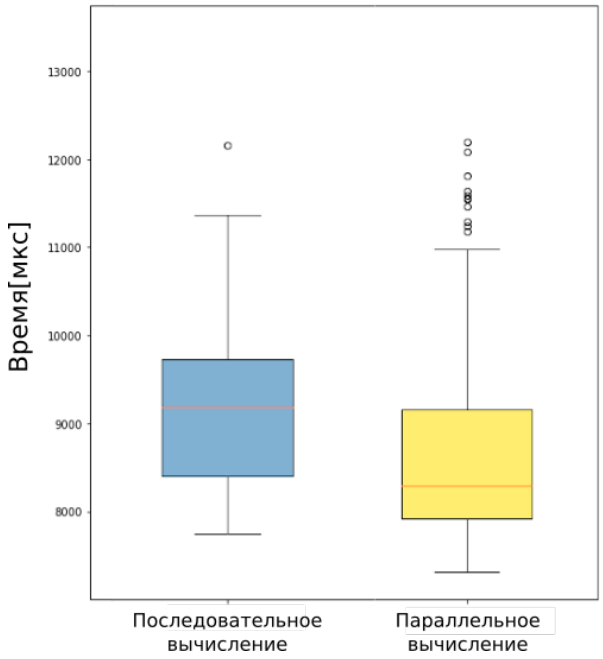


Рисунок 2.19 — График полного времени работы алгоритма 2 с дополнительными тестовыми точками при последовательном и параллельном вычислении

положения относительно интервала $(-l_{thresh}, l_{thresh})$, где l_{thresh} – некоторый фиксированный порог квантования.

Отметим, что базовый алгоритм предполагает статичность наблюдаемой сцены: вклады, полученные в разные моменты времени, накапливаются в элементе карты без ограничений. Для сцен, в которых работает БТС, это предположение не выполняется – в них присутствуют люди, автомобили и иные подвижные объекты, а также перемещается само БТС. Чтобы это учесть, к карте применяется механизм затухания, возвращающий давно не наблюдавшиеся элементы в состояние «неизвестно»; подробно этот механизм рассматривается в главе 4.

Объединение проекции сегментации с картой базового алгоритма выполняется следующим образом. Элементам сетки, определённым как «свободно», присваивается значение l^c_+. Тем элементам, которые были определены как «занято» или «неизвестно» и которые попадают на изображение с камеры – l^c_-. Остальные элементы заполняются 0. Полученная таким образом карта проходимости пространства затем объединяется с картой базового алгоритма по формуле (1.2). Заметим, что все элементы в поле зрения камеры получают ненулевые значения, даже если алгоритм 3 определил их как «неизвестно». Это сделано намеренно, чтобы перевести в состояние «неизвестно» те области карты, которые базовый алгоритм выделил как «свободно», а предложенный алгоритм не смог

это подтвердить. Для достижения этого необходимо выбрать параметры l_-^c и l_+^b таким образом, чтобы выполнялось неравенство:

$$l_-^c + l_+^b \in (-l_{thresh}, l_{thresh}). \tag{2.6}$$

В терминах классификации, приведённой в разделе 1.4, предложенный алгоритм задействует два типа комплексирования на разных своих этапах. На этапе проецирования применяется *кооперативное* комплексирование: лидар предоставляет геометрию поверхности движения, камера – семантическую разметку, и ни один из этих датчиков по отдельности не способен разместить семантическую метку в трёхмерном пространстве. На этапе объединения полученной проекции с картой базового алгоритма применяется *конкурирующее* комплексирование: оба источника оценивают одно и то же свойство – проходимость элемента сетки, – и их расхождение переводит элемент в состояние неопределённости.

2.6 Исследование точности алгоритма проекции сегментации проезжей части на основе комплексирования визуальных и лидарных данных

Для оценки предложенного алгоритма было проведено сравнение качества создаваемой им карты проходимости пространства с картой базового лидарного алгоритма, не использующего комплексирование с визуальными данными. Помимо этого, чтобы раздельно оценить вклад двух составляющих предложенного алгоритма – учёта рельефа поверхности при проецировании и конкурирующего комплексирования при объединении карт, – было выполнено абляционное исследование.

2.6.1 Тестовый набор данных

Для проведения эксперимента использовались 5 записей показаний датчиков БТС, каждая из которых содержит объединённое облако точек с двух лидаров и изображения с трёх бортовых камер. Каждой записи соответствует аннотированный фрагмент длительностью от $4{,}9$ до $9{,}9$ секунд; фрагменты подобраны так,

чтобы охватывать различные дорожные ситуации, характерные для эксплуатации БТС (таблица 6).

№	Описание сценария	Длительность, с	Число кадров
1	Движение по прямой с разворачивающимся впереди автомобилем	9,9	91
2	Начало движения при находящейся впереди группе пешеходов	9,9	90
3	Поворот и движение вдоль парковки автомобилей	9,9	93
4	Подъезд к перекрёстку	4,9	48
5	Объезд препятствий	9,9	87

Таблица 6 — Сценарии, представленные в наборе тестовых данных

Каждый кадр аннотированных данных включает объединённое облако точек, в котором каждой точке присвоена метка принадлежности препятствию, множество спроецированных на плоскость движения БТС детекций людей и автомобилей, а также положение БТС в глобальной системе координат. По этим данным для каждого кадра формировалась эталонная локальная карта проходимости пространства размером 600×600 элементов с разрешением $res = 10$ элементов на метр. Помимо аннотаций кадра, в эталонную карту переносился соответствующий участок размеченной вручную глобальной карты, содержащей области, доступные для проезда БТС, и области статических препятствий. Участкам, выходящим за пределы размеченной глобальной карты, присваивалось состояние «неизвестно». Полученные эталонные карты для всех пяти сценариев приведены в левом столбце рисунка 2.20.

2.6.2 Сравниваемые алгоритмы и метрики

Для используемых данных запись велась с трёх синхронизированных бортовых камер БТС, проецирование сегментации выполнялось для изображения каждой камеры, а полученные фрагменты карты складывались с учётом взаимного расположения камер, после чего результат объединялся с картой базового алгоритма. В качестве источника семантической сегментации использовалась

нейросетевая модель YOLOP [105], дообученная на данных, собранных с реальных БТС. Оценка высоты поверхности земли в центрах элементов сетки производилась добавлением этих центров к тестовым точкам алгоритма 2, как описано в разделе 2.5.1.

Сравнивались четыре алгоритма:

– **Б** – базовый лидарный алгоритм. Комплексирование с визуальными данными отсутствует.
– **П** – предложенный алгоритм. Проекция сегментации выполняется с учётом рельефа поверхности дороги, оценённого по лидарным данным; объединение карт производится конкурирующим комплексированием ($l^c_- \neq 0$).
– **А1** – вариант предложенного алгоритма без учёта рельефа. Проецирование выполняется на фиксированную плоскость $z_l = 0$.
– **А2** – вариант предложенного алгоритма, в котором конкурирующее комплексирование заменено дополняющим. Карта базового алгоритма не содержит свободных зон ($l^b_+ = 0$), а проекция сегментации до сложения с ней не содержит зон препятствий ($l^c_- = 0$). Тем самым лидар отвечает только за препятствия, а камера – только за проезжую часть, и объединяемые карты дополняют, а не уточняют друг друга.

Алгоритмы А1 и А2 добавлены для абляционного исследования предложенного алгоритма, для оценки эффекта отдельных этапов работы алгоритма.

Значения отличающихся параметров, использованных для каждого из алгоритмов, приведены в левой части таблицы 8. Остальные параметры были одинаковы: $z_{max} = 3{,}0$ м, $d_{max} = 40$ м, $res^b = 10$, $res^c = 2{,}5$, $l^b_- = -4{,}701$, $l_{thresh} = 5$. Механизм затухания также применялся одинаково во всех вариантах.

Значения вкладов l^b_+, l^b_-, l^c_+ и l^c_- не подбирались под настоящий эксперимент. Они взяты из конфигурации эксплуатируемой системы восприятия БТС. Тем самым использованный набор параметров не настраивался по тестовой выборке, на которой измеряются приводимые далее метрики.

Качество построенных карт оценивалось тремя метриками, введёнными в разделе 1.5: среднеквадратичной ошибкой относительно эталонной карты (1.7), усреднённым по классам коэффициентом Жаккара (1.6) и метрикой PFC-MSE [77]. При вычислении mJ в качестве классов элементов карты рассматривались три возможных состояния: «занято», «свободно» и «неизвестно». Как уже обсуждалось в главе 1, в качестве основной метрики для сравнения была

выбрана PFC-MSE, поскольку она наиболее показательна для решения задачи планирования движения БТС.

2.6.3 Результаты экспериментов

Значения PFC-MSE по каждому из тестовых сценариев приведены в таблице 7. Предложенный алгоритм превосходит базовый лидарный во всех пяти сценариях; в среднем PFC-MSE снижается с $1019{,}3$ до $924{,}4$, то есть на $9{,}3$ %.

	Алгоритм	**Сценарий**					**Среднее**
		1	2	3	4	5	
Б	Базовый лидарный алгоритм	1218,3	1353,0	1087,5	932,0	505,9	1019,3
П	**Предложенный алгоритм**	**1107,5**	**1213,9**	**949,0**	**850,9**	**500,5**	**924,4**

Таблица 7 — Значения метрики PFC-MSE (меньше – лучше) для предложенного и базового лидарного алгоритмов. Наилучшие значения выделены полужирным

Улучшение наблюдается и по двум другим метрикам: среднеквадратичная ошибка снижается с $0{,}231$ до $0{,}218$, то есть на $5{,}6$ %, а усреднённый по классам коэффициент Жаккара возрастает с $0{,}225$ до $0{,}238$. Таким образом, учёт визуальных данных с камер БТС при построении карты проходимости пространства повышает её качество по сравнению с картой, строящейся исключительно по лидарным данным, причём как по метрикам поэлементного сходства, так и по метрике пригодности карты для планирования маршрута.

Результаты абляционного исследования приведены в таблице 8.

Вклад учёта рельефа поверхности дороги. Алгоритм A1 отличается от предложенного тем, что положение поверхности движения задаётся плоскостью, а не оценивается по лидарным данным. Учёт рельефа даёт преимущество по всем трём метрикам. Более того, по среднеквадратичной ошибке и по основной метрике алгоритм A1 уступает даже базовому лидарному алгоритму, вовсе не использующему визуальные данные. Причина в том, что в процессе эксплуатации БТС внешняя калибровка камер изменяется, а дорожное полотно отклоняется от плоскости, что приводит к неверному сопоставлению точки плоскости с данными семантической сегментации. Тем самым эксперимент подтверждает на реальных

	Параметры				Метрики		
	l^b_+	l^c_-	l^c_+	**рельеф**	**СКО** $\downarrow$	$\boldsymbol{mJ}$ $\uparrow$	**PFC-MSE** $\downarrow$
Б	0,490	–	–	–	0,231	0,225	1019,3
П	0,490	−0,080	1,099	да	0,218	**0,238**	**924,4**
А1	0,490	−0,080	1,099	нет	0,236	0,230	1059,4
А2	0	0	1,099	да	**0,183**	0,205	1130,8

Таблица 8 — Параметры сравниваемых алгоритмов Б, П, А1, А2 (обозначения введены выше) и усреднённые по пяти сценариям значения метрик. Стрелка $\downarrow$ означает, что меньшие значения метрики соответствуют лучшему качеству, $\uparrow$ – большие. Наилучшие значения по каждой метрике выделены полужирным

данных ограничение предположения о плоскости поверхности дороги, отмеченное в разделе 2.3.

Сравнение конкурирующего комплексирования с дополняющим. Алгоритм А2 отличается от предложенного тем, что комплексируемые карты перестают содержать оценки одного и того же свойства: свободные зоны формируются исключительно по данным камер, а лидар отвечает только за препятствия. По основной метрике такая замена приводит к ухудшению результата: PFC-MSE составляет $1130{,}8$ против $924{,}4$ у предложенного алгоритма.

Отдельно отметим, что алгоритм А2 показал наилучшую среди всех вариантов среднеквадратичную ошибку ($0{,}183$), одновременно оказавшись худшим по mJ ($0{,}205$) и по PFC-MSE ($1130{,}8$). Это наблюдение показывает, что среднеквадратичная ошибка и PFC-MSE оценивают карту по-разному: близость карты к эталонной по поэлементному сходству не влечёт её пригодности для планирования маршрута, поскольку связность свободных областей при этом может отличаться существенно. Примеры построенных карт проходимости пространства всеми сравниваемыми алгоритмами приведены на рисунке 2.20.

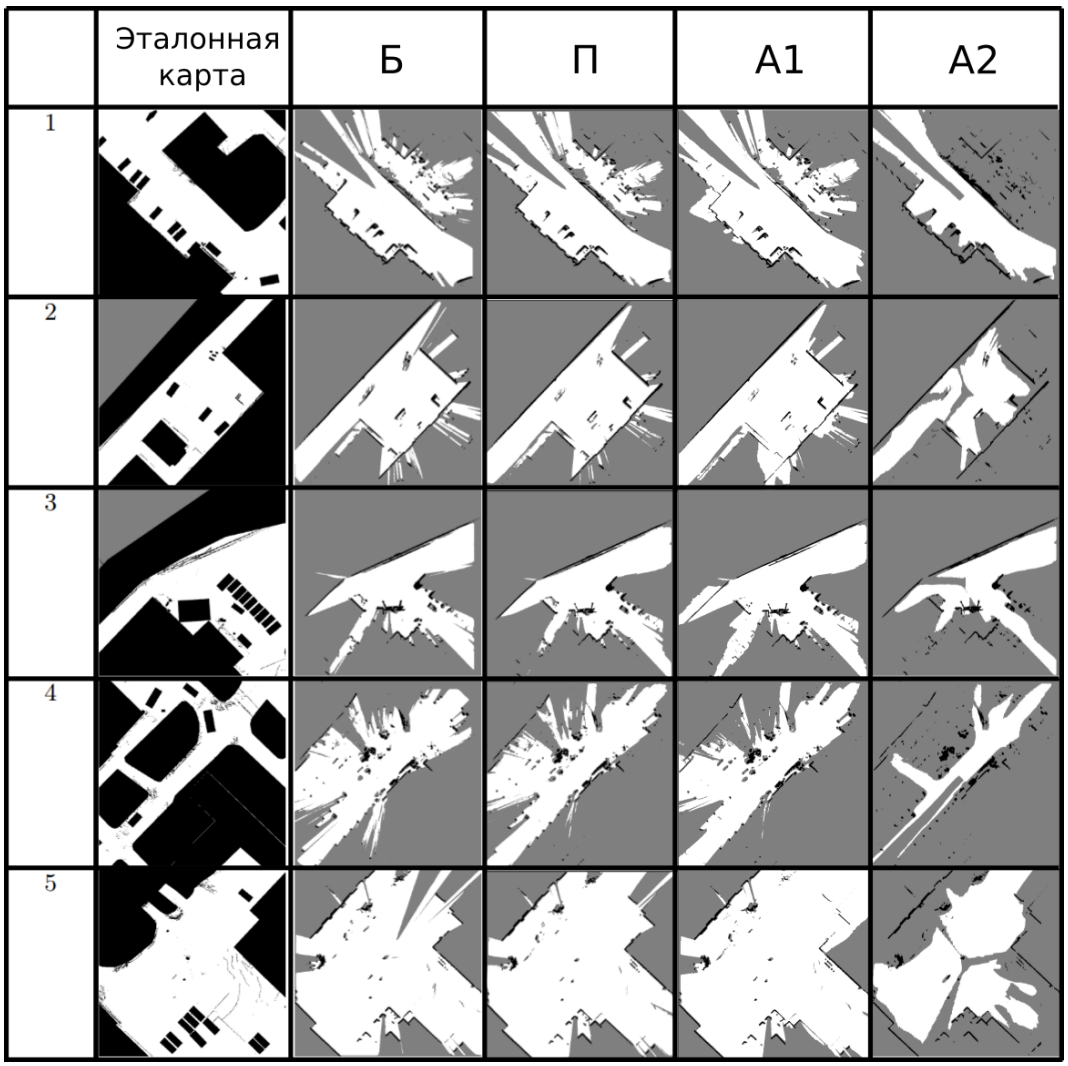


Рисунок 2.20 — Эталонные карты проходимости пространства и карты, построенные сравниваемыми алгоритмами на тестовых данных

2.6.4 Вычислительная эффективность

Измерение времени работы производилось на персональном компьютере с процессором Intel Core i7-13650HX, ОЗУ объёмом 32 ГБ и ГПУ Nvidia GeForce RTX 4070 с графической памятью 8 ГБ. Среднее время одной итерации построения карты проходимости пространства предложенным алгоритмом составило $1{,}63$ мс. Указанное время относится к проецированию результатов семантической сегментации и обновлению значений элементов карты и не включает время инференса модели семантической сегментации. Полученное значение удовлетворяет обоим условиям на задержку, сформулированным в разделе 1.7.2.

2.7 Заключение к главе 2

В главе получены следующие основные результаты:

1. Предложен алгоритм проекции визуальных детекций с изображения на плоскость движения БТС с использованием модели L-shape;
2. Проведено сравнение алгоритма L-Shape с альтернативными алгоритмами. Результаты экспериментов демонстрируют, что:
    а) на реальных данных L-Shape алгоритм показал наивысший среди рассмотренных алгоритмов вписывания ограничивающей рамки средний коэффициент Жаккара ($0{,}337$ против $0{,}328$ у лучшего из остальных);
    б) превосходство L-Shape алгоритма наблюдается не только по среднему, но и по всем квартилям распределения коэффициента Жаккара: $0{,}230$ против $0{,}219$ по нижнему квартилю, $0{,}328$ против $0{,}316$ по медиане, $0{,}451$ против $0{,}448$ по верхнему квартилю и $0{,}713$ против $0{,}651$ по максимуму (по минимуму достигается паритет);
    в) первенство L-Shape алгоритма сохраняется при всех рассмотренных значениях порога дальности, при этом наилучшее абсолютное качество все алгоритмы показывают при ограничении дальности проецируемых препятствий до 10 метров;
3. Предложен алгоритм оценки положения окружающих ТС на основе комплексирования визуальных и лидарных данных, выделяющий точки поверхности движения оценкой её формы регрессией на гауссовских процессах;
4. Проведено сравнение предложенного алгоритма комплексирования с алгоритмами, не использующими комплексирование и обучаемые преобразования в вид сверху. Результаты экспериментов демонстрируют, что:
    а) на реальных данных алгоритм показал средний коэффициент Жаккара $0{,}439$ против $0{,}399$ у лучшего лидарного и $0{,}337$ у визуального алгоритмов. Превосходство сохраняется во всём диапазоне рассмотренных дальностей;

б) среднее время обработки одного кадра составляет $66{,}7$ мс, что удовлетворяет условию 1 на задержку, сформулированному в разделе 1.7.2;

5. Предложен алгоритм проецирования семантической сегментации проезжей части с изображения на оценённую по лидарным данным поверхность движения, а также способ его объединения с базовым алгоритмом построения карты проходимости пространства, использующий конкурирующее комплексирование;
6. Проведено сравнение полученной карты проходимости пространства с картой базового лидарного алгоритма. Результаты экспериментов демонстрируют, что:
   а) учёт визуальных данных снижает PFC-MSE с $1019{,}3$ до $924{,}4$ в пяти рассмотренных сценариях, а среднеквадратичную ошибку относительно эталонной карты – с $0{,}231$ до $0{,}218$;
   б) согласно абляционному исследованию, отказ от лидарной оценки рельефа дорожного покрытия в пользу приближения плоскостью ухудшает все три рассматриваемые метрики, а замена конкурирующего комплексирования дополняющим ухудшает основную метрику;
   в) среднее время одной итерации построения карты составляет $1{,}63$ мс, что удовлетворяет обоим условиям на задержку, сформулированным в разделе 1.7.2.

Перечисленные основные результаты и другие авторские материалы главы опубликованы в работах [129; 126; 130; 127]. Для разработанных алгоритмов было получено свидетельство о регистрации программы для ЭВМ: № 2025617212 Программное обеспечение «Система восприятия N1 V3».

Глава 3. Программный комплекс построения карты проходимости пространства на основе комплексирования разнородных данных

В предыдущей главе 2 были предложены алгоритмы извлечения информации о проходимости пространства вокруг БТС. Результаты их работы имеют разный формат представления, что может усложнять их учёт последующими алгоритмами планирования маршрута БТС. Кроме того, не определены правила, как разрешать противоречия о проходимости пространства в полученных результатах алгоритмов.

В данной главе описан программный комплекс построения карты проходимости пространства – модели, описанной в главе 1. Данный комплекс реализует сбор разнородных данных с датчиков или их обработчиков, преобразует данные в единый формат – фрагменты карты проходимости пространства и производит их комплексирование в единую карту проходимости пространства. Глава содержит подробное описание разработанного программного комплекса: структуру, функциональные возможности, формат входных и выходных данных.

Из требований, сформулированных в разделе 1.8, к программному комплексу непосредственно относятся два: масштабируемость по числу используемых датчиков и расширяемость под новые типы датчиков (требование 4), а также возможность одновременного обновления модели несколькими источниками данных (требование 5). Задача, решаемая в настоящей главе, состоит в том, чтобы найти архитектурные решения, удовлетворяющие этим требованиям, при сохранении работы комплекса в режиме реального времени на бортовом вычислителе БТС (требование 3).

Принятые решения и их связь с требованиями сведены в таблице 9; далее в главе каждое из них описано подробно.

3.1 Общие сведения

Полное наименование комплекса программ – «Программный комплекс построения карты проходимости в окрестности высокоавтоматизированного беспилотного грузового транспортного средства». Условное обозначение

Требование	Архитектурное решение	Чем требование удовлетворено
Расширяемость под новые типы датчиков	Единый интерфейс поставщика фрагментов «IMapPatchProvider», параметризуемый типом входных данных	Подключение источника нового типа сводится к реализации единственного метода, преобразующего данные во фрагмент карты. Код построения карты не изменяется
Масштабируемость по числу датчиков	Инверсия зависимостей: класс, строящий карту, не знает о конкретных поставщиках и получает фрагменты через общую очередь	Число одновременно подключённых поставщиков не фиксировано в коде и задаётся конфигурацией
Одновременное обновление несколькими источниками	Обработка каждого источника в отдельном потоке выполнения. В общую карту попадают уже готовые фрагменты	Извлечение признаков и построение фрагментов выполняются параллельно. Последовательным остаётся только сведение фрагментов, которому посвящена глава 4

Таблица 9 — Требования к программному комплексу и реализующие их архитектурные решения

– OccupancyMapFusion. Программный комплекс реализован на языках C++ и CUDA C++.

3.2 Назначение и функциональные возможности

Программный комплекс Occupancy Map Fusion предназначен для построения карты проходимости пространства – двумерной сетки фиксированного разрешения, элементы которой описывают возможность проезда БТС в пространстве, ограниченном границами данного элемента. Построение производится на основе сырых показаний датчиков, а также признаков, извлечённых из них.

Программный комплекс предоставляет следующие функциональные возможности:

1. многопоточное обновление карты проходимости пространства на основе различных данных:
    а) облака точек, принадлежащих препятствиям;
    б) показаний сонара;

в) комплексирования двумерных детекций на RGB-изображении с камеры и облака точек с лидара, принадлежащих препятствиям;
г) комплексирования семантической сегментации проезжей части на RGB-изображении с камеры и облака точек с лидара, принадлежащих земле;
д) трёхмерных детекций препятствий;

2. расширение поддерживаемых источников данных о проходимости окружающего пространства путём самостоятельной генерации пользователем фрагментов карты проходимости пространства;
3. публикация генерируемой карты проходимости пространства в виде ROS-сообщений (Robot Operating System) [113];
4. опциональное удаление из карты проходимости пространства информации об областях, занимаемых динамическими препятствиями;
5. отключение и подключение отдельных источников информации, учитываемой в карте проходимости пространства, по ходу выполнения программы;

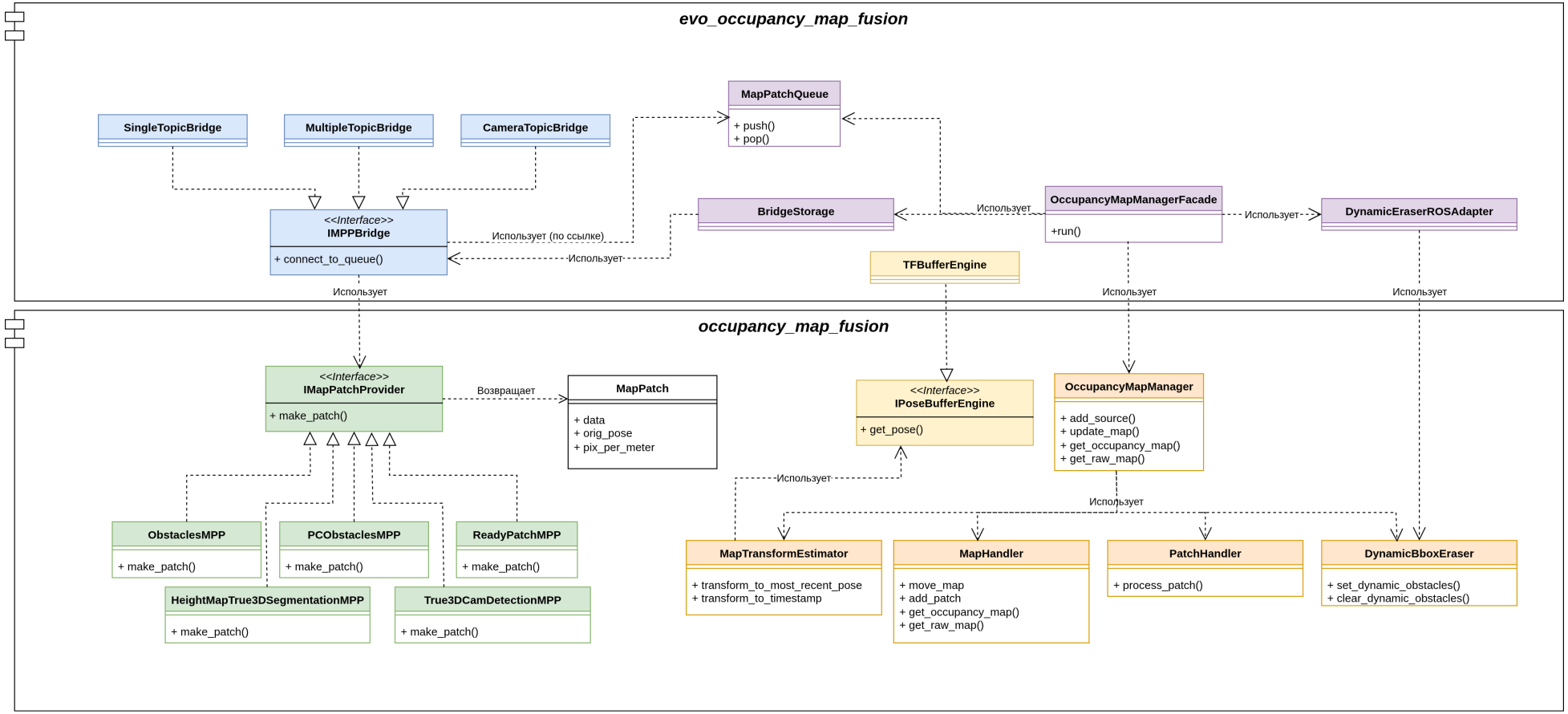
3.3 Структура программного комплекса

На рисунке 3.1 показана диаграмма основных компонентов описываемого программного комплекса. На рисунке 3.2 схематично изображена последовательность шагов при обработке данных приходящих от источников данных (т.е. датчиков либо их предварительных обработчиков, извлекающих полезные признаки). Комплекс состоит из двух крупных компонентов:

1. occupancy_map_fusion – библиотека, реализующая логику работы с картой проходимости пространства: добавление дополнительных источников данных, актуализация положения карты, обработка и учёт входных данных от источников. Компонент реализован на языках C++ и CUDA C++
2. evo_occupancy_map_fusion – программа, интегрирующая библиотеку occupancy_map_fusion в систему ROS: производит соединение с остальными компонентами систем БТС через механизмы ROS-топиков и ROS-сервисов, преобразует входные данные в формат, соответству-

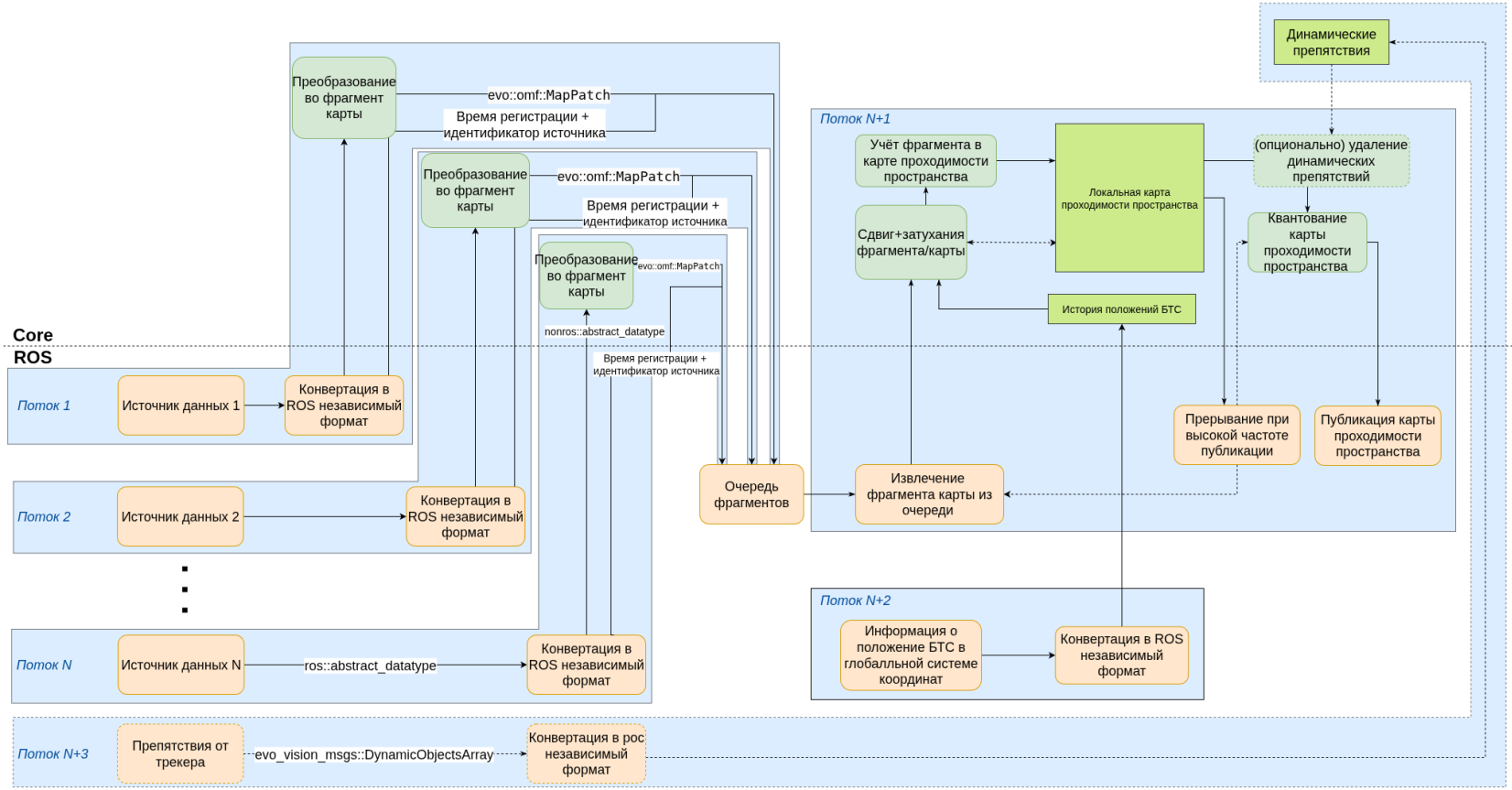
ющий интерфейсам библиотеки, и преобразует генерируемую карту проходимости в ROS сообщение, передаваемое последующим компонентам ПО БТС, в частности компоненту планирования пути БТС. Компонент реализован на языке C++.

Далее в разделе представлено подробное описание структуры данных компонентов, их назначение и форматы входных и выходных данных.



Цветом выделены «идейно связанные» компоненты: базовый класс и его наследники, либо класс и классы хранимых в нём (путём композиции) объектов

Рисунок 3.1 — Диаграмма компонентов программного комплекса Occupancy Map Fusion на языке UML(Unified Modeling Language)



Зелёным цветом выделены этапы обработки данных компонентами независящими от ROS(Robot Operating System), оранжевым – компонентами, зависящими от ROS

Рисунок 3.2 — Диаграмма деятельности программного комплекса Occupancy Map Fusion

3.3.1 Библиотека «occupancy_map_fusion»

Библиотека «occupancy_map_fusion» реализует функционал для построения карты проходимости пространства на основе обратной модели наблюдения. Учёт новых данных в карте происходит в два этапа (рисунок 3.2):

1. Построение фрагмента карты проходимости по данным от одного из источников данных.
2. Добавление построенного фрагмента на общую карту проходимости пространства с учётом:
    а) положения системы координат фрагмента относительно системы координат карты,
    б) изменения положения БТС за время между временем зарегистрированных данных и последнего актуального положения карты в глобальной системе координат,
    в) времени прошедшего с момента регистрации данных.

Первый шаг реализуется через интерфейс «IMapPatchProvider» и наследующие его классы-реализации. Второй – классом «OccupancyMapManager». Опишем их подробнее.

Интерфейс «IMapPatchProvider»

«IMapPatchProvider» – шаблонный класс, параметризуемый типом входных данных $InputT$. Интерфейс задаёт единственный абстрактный метод make_patch, реализующий преобразование входных данных во фрагмент карты проходимости пространства.

Инициализация включает параметры, используемые всеми наследуемыми классами:

1. pix_per_meter – разрешение генерируемого фрагмента,
2. $p(TP) \in (0,1)$ – вероятность того, что реализация верно определит наличие препятствия в клетке генерируемого фрагмента,
3. $p(FN) \in (0,1)$ – вероятность того, что реализация ошибочно проигнорирует препятствие в клетке генерируемого фрагмента.

Входом метода «make_patch» являются входные данные источника типа $InputT$.

Выходом метода «make_patch» является фрагмент карты проходимости пространства. Фрагментом является сетка фиксированного разрешения pix_per_meter. Те элементы сетки, которые, согласно реализуемому алгоритму, заняты препятствиями, равны логарифму шансов вероятности корректной работы алгоритма:

$$l_+ = \ln \frac{p(TP)}{1 - p(TP)}. \tag{3.1}$$

Те элементы сетки, которые, согласно реализуемому алгоритму, не заняты какими-либо препятствиями, равны логарифму шансов вероятности ошибки алгоритма:

$$l_- = \ln \frac{p(FN)}{1 - p(FN)}. \tag{3.2}$$

Все остальные значения сетки заполнены 0, что соответствует вероятности занятости пространства $0{,}5$:

$$p(occ) = \frac{\exp(l)}{1 + \exp(l)} \overset{l=0}{=} \frac{1}{2}. \tag{3.3}$$

Помимо самой сетки результирующий фрагмент хранит её разрешение, а также положение источника данных относительно системы координат генерируемой сетки. Всё это описание хранится в виде структуры «MapPatch»:

```
struct MapPatch{
  cv::Mat data;
  float res;
  cv::Point2f origin;
};
```

Здесь сетка хранится в виде структуры «cv::Mat» из библиотеки Open Source Computer Vision Library (OpenCV) [118], это позволяет упростить дальнейшие манипуляции с фрагментом при добавлении на общую карту проходимости: такие как поворот и квантование. Такая форма представления может быть полезна и для классов наследников при построении фрагмента, например для добавления на фрагмент геометрических примитивов.

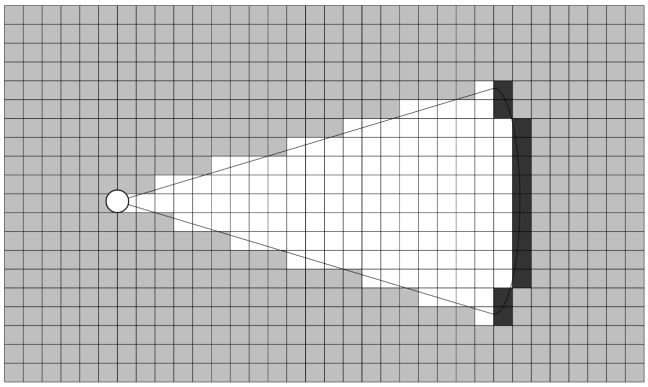


Рисунок 3.3 — Иллюстрация поля зрения, занимаемого препятствием

Класс «ObstaclesMPP»

Класс «ObstaclesMPP» реализует интерфейс «IMapPatchProvider». Входным типом для класса является набор полигонов, ограничивающих области, занятые препятствиями. Такие данные могут приходить от сонаров или алгоритмов, представленных в разделе 2.1.

Инициализация класса полностью совпадает с инициализацией «IMapPatchProvider».

Входом метода «make_patch» являются полигоны, ограничивающие области, занимаемые препятствиями. Координаты вершин полигонов описаны в системе координат источника данных.

```
using InputT = std::vector<std::vector<cv::Point2f>>;
```

Выходом метода «make_patch» является фрагмент карты проходимости пространства в формате «MapPatch». Области ограниченные входными полигонами заполнены значениями l_+. Дополнительно метод может (если задать $p(FN) \neq 0{,}5$) заполнить поле зрения, занимаемое входными полигонами, значениями l_- (рисунок 3.3). Это повышает качество генерируемой карты проходимости пространства, если наблюдаемые препятствия не могут заслонять друг друга и, следовательно, можно судить об отсутствии препятствий в пространстве между источником данных и препятствием.

Класс «True3DCamDetectionMPP»

Класс «True3DCamDetectionMPP» реализует интерфейс «IMapPatchProvider». Входной тип «InputT» – облако точек препятствий с лидара, полученное алгоритмом разделения облака точек (раздел 2.3.1) и двумерные детекции с RGB-камеры БТС.

```
struct BBox2D{
  int min_x = 0;
  int min_y = 0;
  int max_x = 0;
  int max_y = 0;
};
using InputT = std::pair<pcl::PointCloud<PointT>, std::vector<BBox2D>>;
```

Здесь облако точек хранится в формате «pcl::PointCloud<PointT»> библиотеки Point Cloud Library (PCL) [119]. Данная библиотека широко распространена и предлагает функционал по вычислительно эффективной обработке облаков точек. Также её преимуществом являются готовые библиотеки, предоставляющие возможность конвертации классов библиотеки PCL (в частности, облаков точек) в ROS сообщения и обратно, например, pcl_conversions [120]. В качестве структуры точки, хранимой в облаке точек «PointT» может выступать любая структура с обязательными полями «x», «y» и «z», которые являются числами с плавающей точкой «float».

Метод «make_patch» реализует алгоритм представленный в разделе 2.3 проекции двумерных детекций с камеры БТС в систему координат БТС O_l с использованием комплексирования данных с лидара с последующей растеризацией полученных ограничивающих рамок. В случае отсутствия кластера облака точек препятствий, соответствующего двумерной детекции (см. шаг 6 алгоритма 1), метод опционально может спроецировать детекцию в систему координат, используя наивный алгоритм из раздела 2.1.1, чтобы хоть и с меньшей точностью, но учесть объект в генерируемом фрагменте карты проходимости.

Инициализация класса помимо стандартных параметров «IMapPatchProvider» включает следующие параметры:

1. параметры кластеризации облака точек:
   a) min_size, max_size – минимальное и максимальное число точек кластеров (кластеры за пределами этих границ игнорируются);

б) $tolerance$ – максимальное расстояние Евклида между точками, при котором они считаются точками одного кластера;

в) $distance_clip$ – максимальное расстояние Евклида от центра системы координат O_l до кластеризуемых точек (точки, расположенные далее удаляются из облака);

г) $z_squeeze$ – коэффициент сжатия координат вдоль оси Oz_l (ввиду принципа формирования облака точек точки располагаются реже вдоль этой оси, чем в плоскости Oxy_l);

2. параметры метода RANSAC для вписывания кластера в ограничивающую рамку:
   а) $ransac_tolerance$ – максимальное расстояние Евклида от линии-гипотезы RANSAC до точек кластера, чтобы считать их соответствующими гипотезе;
   б) $iterations_num$ – число итераций метода RANSAC;
3. настройки наивного алгоритма проекции детекций в случае отсутствия сопоставленного кластера:
   а) $enable$ – булев тип для включения алгоритма;
   б) bev_depth – глубина прямоугольника, строящегося на отрезке после проекции нижней границы препятствия;

Входом метода «make_patch» является пара: облако точек препятствий в формате «pcl::PointCloud» и двумерные детекции с камеры в формате «std::vector<BBox2D»>.

Выходом метода «make_patch» является фрагмент карты проходимости в формате «MapPatch», на котором области внутри полученных алгоритмами из разделов 2.3 и 2.1.1 (опционально) ограничивающими рамками заполнены l_+, остальные элементы фрагмента заполнены 0.

Класс «HeightMapTrue3dSegmentationMPP»

Класс «HeightMapTrue3dSegmentationMPP» реализует интерфейс «IMapPatchProvider». Метод «make_patch» реализует алгоритмы из раздела 2.5 проецирования семантической сегментации на вид сверху в окрестности БТС.

Инициализация класса помимо стандартных параметров «IMapPatchProvider» включает следующие параметры:

1. параметры построения карты высот:
    a) $height_map_creator_type$ – параметр задающий способ построения карты высот над плоскостью движения БТС. Доступны варианты:
        1) $ortho_projection$ – использование готовой карты высоты, которая прямоугольно проецируется на плоскость движения БТС;
        2) $delaunay$ – карта высот инициализирована частично, неизвестные значения высоты линейно интерполируются согласно триангуляции Делоне;
2. use_gpu – булев тип, указывающий использовать ли алгоритм проекции с использованием параллельных вычислений на архитектуре CUDA.

Входом метода «make_patch» является пара: облако точек земли в формате «pcl::PointCloud» и двумерная семантическая сегментация проезжей части с камеры в формате одноканального изображения «cv::Mat».

```
using InputT = std::pair<pcl::PointCloud<PointT>, cv::Mat>;
```

Выходом метода «make_patch» является фрагмент карты проходимости «MapPatch», построенный алгоритмом 3. Области, согласно алгоритму соответствующие свободному для проезда БТС пространству, заполнены l_-, занятые препятствиями – l_+, остальные – 0.

Класс «PCObstaclesMPP»

Класс «PCObstaclesMPP» реализует интерфейс «IMapPatchProvider». Метод «make_patch» производит прямоугольную проекцию облака точек препятствий на плоскость карты проходимости пространства.

Инициализация класса помимо стандартных параметров «IMapPatchProvider» включает следующие параметры:

1. параметры отбора точек:
    a) min_z, max_z – интервал координаты z точек, попадающих на фрагмент карты проходимости пространства(необходим для

фильтрации точек, находящихся выше БТС и не препятствующих его движению);

б) $inplane_max_dist$ – максимальное расстояние Евклида в плоскости Oxy_l от центра координат до точек, попадающих на итоговый фрагмент.

Входом метода «make_patch» является: облако препятствий в формате «pcl::PointCloud».

```
using InputT = pcl::PointCloud<PointT>;
```

Выходом метода «make_patch» является фрагмент карты проходимости пространства «MapPatch», элементы фрагмента, в которые попали проекции точек инициализированы l_+, остальные – 0.

Класс «ReadyPatchMPP»

Класс «ReadyPatchMPP» реализует интерфейс «IMapPatchProvider». Класс позволяет расширить количество источников информации, с которыми может взаимодействовать библиотека без изменения содержащегося в ней кода. Это упрощает для пользователей возможность экспериментировать с дополнительными алгоритмами, учитываемыми в карте проходимости пространства БТС.

Инициализация класса полностью совпадает с инициализацией «IMapPatchProvider».

Входом метода «make_patch» является готовый фрагмент карты проходимости пространства «MapPatch». Элементы фрагмента заполнены значениями в интервале $[-1, 1]$, значения в интервале $[-1, 0)$ интерпретируются как области, свободные для проезда БТС, в интервале $(0, 1]$ – занятые препятствиями.

```
using InputT = MapPatch;
```

Выходом метода «make_patch» является фрагмент карты проходимости пространства «MapPatch» того же размера и разрешения, что и входной фрагмент. Значения элементов определяются по следующей формуле:

$$l_{out} = \begin{cases} l_- & \text{, если } l_{in} < 0 \\ 0 & \text{, если } l_{in} = 0 \\ l_+ & \text{, если } l_{in} > 0 \end{cases}, \tag{3.4}$$

где l_{out} – итоговое значение итогового фрагмента, l_{in} – оригинальное значение входного фрагмента.

Класс «DynamicEraser»

В некоторых задачах планирования пути БТС важно различать статические и динамические препятствия (люди, машины, велосипедисты и пр.), окружающие БТС и обрабатывать их разными алгоритмами. Чтобы сделать это возможным в рамках библиотеки «occupancy_map_fusion» был создан класс «DynamicEraser», который осуществляет удаление информации об областях, занимаемых динамическими препятствиями.

Инициализация класса осуществляется одним параметром:

$bbox_expand_factor$ – параметр, задающий степень увеличения входящих динамических препятствий, чтобы учесть погрешность в определении их положения.

Класс обладает двумя методами:

1. «set_dynamic_obstacles» – сохраняет в объекте класса наблюдаемые на данный момент динамические препятствия в окрестности БТС. Препятствия описываются повёрнутыми ограничивающими рамками на плоскости карты проходимости пространства:

```
struct DynamicObstacle{
  struct {
    float x;
    float y;
  } center;
  struct {
    float x;
    float y;
  } dims;
  float heading;
};
```

2. «clear_dynamic_obstacles» – осуществляет удаление динамических препятствий из карты проходимости пространства. На вход алгоритму подаётся текущая карта проходимости пространства: для всех областей,

занимаемых сохранёнными динамическими препятствиями, соответствующие значения элементов карты становятся равными 0.

Класс «OccupancyMapManager»

Класс «OccupancyMapManager» осуществляет хранение и обработку итоговой карты проходимости пространства вокруг БТС на основе фрагментов, генерируемых реализациями «IMapPatchProvider», и информации о перемещении БТС в глобальной системе координат.

Класс имеет следующие основные методы:

1. **«add_source».** Метод, регистрирующий новый источник информации, учитываемый в карте проходимости пространства. Аргументом функции является положение поля генерируемых источником фрагментов «MapPatch». Положение описывается структурой:

```
struct Pose2f{
  float x;
  float y;
  float heading;
};
```

   x, y – положение $origin$ генерируемых «MapPatch» в системе координат O_l БТС. $heading$ – угол поворота генерируемых фрагментов в той же системе координат.

   Метод возвращает уникальный идентификатор источника, по которому в дальнейшем будет производиться преобразование фрагмента в общую систему координат перед тем, как он будет добавлен на карту проходимости пространства.

2. **«map_update».** Существует две перегрузки данного метода. Первая – без каких-либо аргументов. При вызове этой перегрузки, метод находит последнее известное положение БТС в глобальной системе координат и сдвигает карту с учётом положения, в котором в последний раз была сохранена карта. Вторая перегрузка принимает на вход три параметра: фрагмент карты проходимости пространства «MapPatch», идентификатор источника данных, который сгенерировал данный фрагмент, и время,

которому этот фрагмент соответствует. При вызове этой перегрузки метод корректирует положение карты проходимости (если время регистрации фрагмента позднее времени последнего сохранения карты), осуществляет сдвиг и поворот фрагмента, чтобы сопоставить его с картой проходимости пространства и добавляет его путём суммирования элементов фрагмента с элементами карты проходимости (см. «Поток N+1» на рисунке 3.2). Дополнительно после этого шага может быть произведено удаление динамических препятствий с карты проходимости. Также в обеих перегрузках данного метода происходит экспоненциальное затухание элементов карты проходимости или фрагмента, а также их отсечение, чтобы уменьшить количество артефактов, полученных в результате ошибок источников информации. Подробнее об этих двух операциях будет рассказано в главе 4.

3. **«get_raw_map».** Метод не принимает никаких параметров и возвращает последнюю сохранённую карту проходимости пространства без квантования значений элементов карты.
4. **«get_occupancy_map».** Метод не принимает никаких параметров и возвращает последнюю сохранённую карту проходимости пространства, полученную после квантования:

$$l_{quant} = \begin{cases} -1 & \text{, если } l_{raw} < -l_{thresh} \\ 1 & \text{, если } l_{raw} > l_{thresh} \\ 0 & \text{, в остальных случаях} \end{cases}, \tag{3.5}$$

где l_{quant} итоговое значение элемента после квантования, l_{raw} – оригинальное значение элемента, $l_{thresh} > 0$ – порог квантования, задаваемый параметром класса.

Для реализации описанного функционала класс хранит путём композиции несколько специализированных классов:

1. **«MapTransformEstimator».** Класс отвечает за сбор, хранение и вычисление положений БТС в глобальной системе координат в течение времени работы класса. Часть функционала класса делегирована интерфейсу «IPoseBufferEngine» для реализации принципа инверсии зависимостей [121], поскольку в системе ROS уже существует механизм для оценки относительных перемещений систем координат, используемых в описании движения БТС [122].

2. **«MapHandler».** Класс отвечает за хранение, модификацию и извлечение карты проходимости пространства. К модификациям карты относятся: сдвиг, экспоненциальное затухание, клиппинг и добавление фрагментов приведённых в систему координат карты.
3. **«PatchHandler».** Класс отвечает за преобразование фрагментов карты (сдвиг, поворот, экспоненциальное затухание фрагментов) в систему координат карты.
4. **«DynamicEraser».** Опциональный класс, реализующий удаление информации об участках, занятых динамическими препятствиями, как описано в предыдущем разделе.

3.3.2 Программа «evo_occupancy_map_fusion»

Программа «evo_occupancy_map_fusion» осуществляет интеграцию функционала в систему ROS, предоставляя классы осуществляющие преобразование ROS-сообщений в классы и структуры, используемые библиотекой «occupancy_map_fusion» (тем самым реализуя структурный шаблон проектирования «Адаптер» [123]). Программа включает следующие основные классы:

1. **«IMPPBridge».** Класс «Адаптер» класса «IMapPatchProvider». Реализации этого класса в отдельном программном потоке подписываются на один («SingleTopicBridge») или несколько («MultipleTopicBridge») ROS-топиков, в которые поступают сообщения на обработку программой, преобразует их в формат библиотеки «occupancy_map_fusion» и передаёт адаптируемому объекту класса «IMapPatchProvider», к сформированному фрагменту добавляется метаинформация: идентификатор источника данных и время регистрации отражаемых им данных, после чего структура добавляется в очередь «MapPatchQueue». Помимо этого класс предоставляет возможность временно отключить учёт, соответствующего ему источника данных в карте проходимости пространства через ROS-сервис – модель коммуникации компонентов ROS путём двунаправленной синхронной связи между клиентом и сервером.

2. **«MapPatchQueue».** Класс, реализующий очередь и гарантирующий потоковую безопасность при добавлении и извлечении фрагментов с метаинформацией.
3. **«DynamicEraserROSAdapter»**. Класс, аналогично «IMPPBridge» подписывающийся на сообщения с динамическими препятствиями, конвертирующий их в «std::vector<DynamicObstacle»> и передающий результат в хранимый внутри класса объект «DynamicEraser».
4. **«TFBufferEngine».** Класс реализует функционал, необходимый для работы «MapTransformEstimator» адаптируемой библиотеки.
5. **«OccupancyMapManagerFacade».** Класс, исполняющий основной цикл обновления и публикации карты проходимости пространства вокруг БТС («Поток N+1» на рисунке 3.2). Метод «run» в цикле пытается извлекать новые фрагменты из очереди «MapPatchQueue», при успехе фрагмент и метаинформация передаются в метод «map_update» хранимого в классе объекта «OccupancyMapManager», после чего, если класс «DynamicEraserROSAdapter» инициализирован, происходит удаление областей, занимаемых динамическими препятствиями. Далее, если с момента последней публикации карты проходимости пространства прошло больше заданного в параметрах класса времени происходит извлечение карты проходимости через метод «get_occupancy_map» класса «OccupancyMapManager», который затем преобразуется в ROS-сообщение типа «nav_msgs/OccupancyGrid» и публикуется в отдельный топик для сообщений данного типа.

3.4 Заключение к главе 3

В данной главе разработан программный комплекс построения карты проходимости пространства на основе комплексирования разнородных данных от датчиков, установленных на БТС, и их обработчиков. Комплекс реализует алгоритмы, описанные в главе 2, а также позволяет пользователям интегрировать собственные алгоритмы построения фрагментов карты проходимости пространства. Подключение нового источника данных, в том числе датчика нового типа, сводится к реализации интерфейса «IMapPatchProvider» и не требует измене-

ния кода построения карты, а число одновременно подключаемых источников не ограничено архитектурой комплекса. На представленный программный комплекс были получены патент № 2853062 «Способ построения карты проходимости в окрестности высокоавтоматизированного беспилотного грузового транспортного средства» и свидетельство о регистрации программы для ЭВМ № 2025669744 Программное обеспечение «Способ построения карты проходимости в окрестности высокоавтоматизированного беспилотного грузового транспортного средства».

Глава 4. Параллельный алгоритм построения карты проходимости пространства на основе комплексирования разнородных данных

В главе 3 описан программный комплекс, реализующий построение карты проходимости пространства на основе множества показаний с датчиков, установленных на БТС. Несмотря на показанную работоспособность для реальных БТС, данный комплекс может задерживать добавление сформированных фрагментов на карту в случае большого количества учитываемых в модели источников данных. Данное свойство комплекса ограничивает число датчиков, которые могут участвовать в уточнении карты проходимости пространства, и, следовательно, ограничивает надёжность строящейся карты.

В данной главе предложены модификации алгоритма построения карты проходимости пространства, реализованного в программном комплексе из главы 3, позволяющие снизить время обработки информации об окружающем пространстве при добавлении её на карту проходимости пространства, а также избавиться от необходимости последовательного добавления фрагментов на карту, что позволяет увеличить количество комплексируемых источников данных с сохранением возможности работы в режиме реального времени.

4.1 Производительность алгоритма построения карты проходимости пространства

Представленный в главе 3 алгоритм построения карты проходимости пространства можно разделить на два этапа:

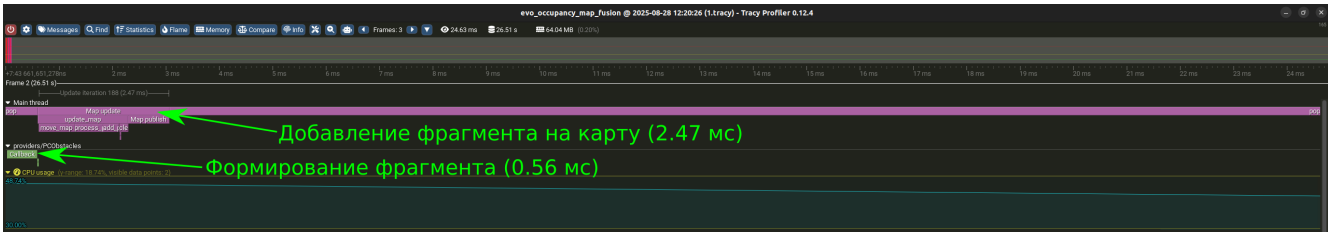
1. Преобразование входных данных к единому формату представления данных о проходимости пространства. Как правило, это фрагмент карты проходимости, содержащий вероятности занятости клеток, основываясь на последних показаниях данного источника данных.
2. Комплексирование полученных фрагментов с раннее построенной картой проходимости пространства. Для обратной модели наблюдения этот шаг заключается в простом суммировании полученного фрагмента с соответствующей областью карты.

Первый шаг допускает параллельное выполнение: для построения фрагмента используются данные только одного источника, и синхронизация с другими источниками и подсистемами не требуется. Его вычислительная стоимость определяется сложностью конкретного алгоритма, однако далее в главе предполагается, что эти алгоритмы уже реализованы оптимальным для целевой системы образом, и потому ускорению не подлежат. Второй шаг, напротив, является блокирующим: созданный фрагмент попадает в очередь и включается в карту лишь после всех фрагментов, стоящих перед ним. Задержка возникает даже тогда, когда добавляемые фрагменты не пересекаются и параллельное обновление теоретически возможно, и растёт с числом источников данных, участвующих в формировании карты.

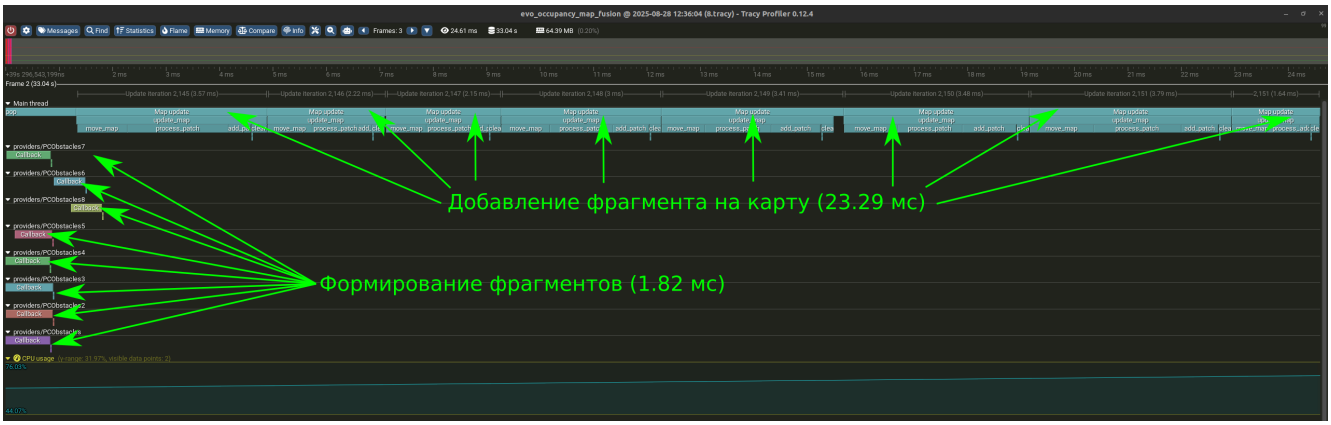
Покажем это на практике: на рисунке 4.1 представлены результаты профилирования «evo_occupancy_map_fusion» при обработке одного и восьми источников данных. В версии с восемью источниками все провайдеры фрагментов были дубликатами провайдера из запуска с одним источником. Следовательно, все провайдеры обрабатывали одни и те же данные, зарегистрированные в один момент времени. Можно видеть, что из-за последовательного добавления фрагментов на карту проходимости пространства задержки обновления карты проходимости доходят до более чем семикратного замедления.

Сопоставим полученные значения с условиями на задержку, сформулированными в разделе 1.7.2. Условие 1 – учёт данных до поступления следующих – при частоте регистрации 10 Гц отводит на обновление карты 100 мс и выполняется в обоих запусках. Условие 2 – содержательность разрешения карты – при размере элемента $0{,}1$ м и относительной скорости 40 км/ч отводит около 9 мс: оно выполняется при одном источнике данных ($3{,}03$ мс), но при восьми ($22{,}78$ мс) нарушается примерно в $2{,}5$ раза. Иными словами, при штатном для рассматриваемых БТС составе датчиков последовательное добавление фрагментов не теряет данные, однако строит карту, разрешение которой заведомо не соответствует моменту её актуальности. Это и определяет предмет настоящей главы.

Далее в главе предложены модификации базового алгоритма построения карты проходимости, направленные на ускорение второго этапа обновления карты проходимости пространства, а также делающие этот шаг асинхронным для каждого источника данных. Перед тем как приступить к описанию модификаций опишем формально задачу, решаемую алгоритмом.



а) Один источник данных. Время обновления карты – $3{,}03$ мс.



б) 8 источников данных. Самое долгое время обновления карты – $22{,}78$ мс (замедление на 652 %).

Результаты профилирования получены в программе инструментального профилирования «Tracy» [114]. Приведённые снимки экрана приведены к одному масштабу временной шкалы

Рисунок 4.1 — Наблюдаемые задержки при обновлении карты проходимости пространства

4.2 Постановка задачи построения карты проходимости пространства

Зафиксируем неподвижную относительно земли глобальную систему координат, расположенную так, что движение БТС происходит в плоскости $z = 0$. Положение БТС в ней описывается тройкой $\mathbf{x} = x, y, \theta$: координатами x, y фиксированной точки БТС и углом θ поворота БТС от оси Ox вокруг вертикальной оси (рисунок 4.2). С той же фиксированной точкой свяжем локальную систему координат БТС, ось Ox которой направлена вдоль оси его движения; координаты в ней обозначаются x_l, y_l.

Пусть на БТС имеется S источников данных, каждый со своей системой координат, положение которой известно и неизменно относительно локальной. Независимо от остальных источник s порождает сведения о занятом и/или свободном пространстве в окрестности БТС в виде наборов $(P_{s,k}, \mathbf{x}_{s,k}, t_{s,k})$, $s = \overline{1,S}$, $k \in \mathbb{N}$. Здесь $P_{s,k}$ – фрагмент карты проходимости, то есть двумерный массив

логарифмов шансов занятости пространства с фиксированным разрешением res_s (pixel per meter, элементов на метр), оси которого ориентированы вдоль осей системы координат источника s; $\mathbf{x}_{s,k}$ задаёт положение начала системы координат датчика относительно левого верхнего угла массива; $t_{s,k}$ – время регистрации данных, по которым построен фрагмент; k – порядковый индекс набора.

По этим данным требуется сформировать локальную карту проходимости пространства в виде набора $(M_i, \mathbf{x}_i, t_i)$, $i \in \mathbb{N}$, где M_i – двумерный массив логарифмов шансов занятости пространства с фиксированным разрешением res, соответствующий моменту времени t_i, а $\mathbf{x}_i$ – координаты левого верхнего угла карты. Координаты $\mathbf{x}_i$ выбираются так, чтобы начало локальной системы координат в момент t_i приходилось на центр карты M_i, при этом оси карты остаются параллельными осям глобальной системы координат. Такая ориентация выбрана не случайно: при ориентации карты вдоль осей локальной системы координат каждое изменение курса БТС требовало бы поворота двумерного массива на малый угол, и накопление подобных поворотов вносило бы погрешность в оценку положения свободного и занятого пространства.

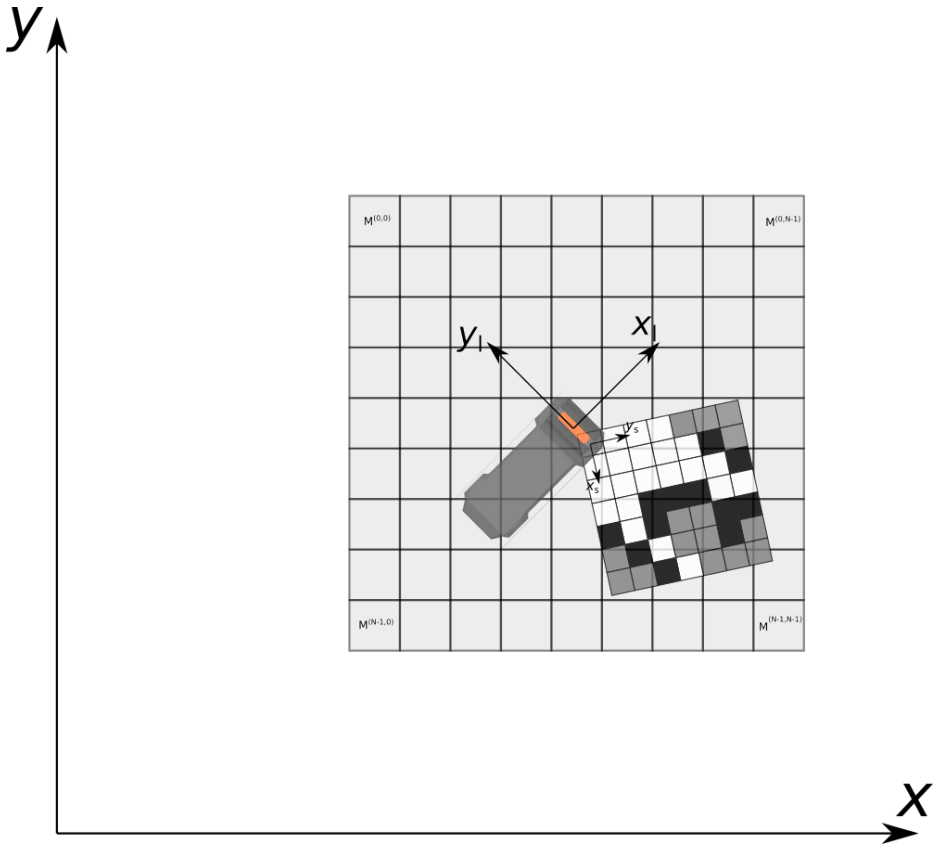


Рисунок 4.2 — Расположение локальной карты проходимости и её фрагмента от одного из источников данных относительно БТС и глобальной системы координат

Генерируемые фрагменты $P_{s,k}$ поступают в общую очередь по мере готовности, а локальная карта обновляется последовательным извлечением фрагментов из этой очереди. Порядок извлечения определяется моментом готовности фрагмента, а не моментом регистрации данных, поэтому последовательность времён $t_{s,k}$ в общем случае не монотонна (рисунок 4.3).

Одна итерация обновления приведена в листинге 4 и состоит из трёх действий. Во-первых, если извлечённый фрагмент новее карты, карта актуализируется: её положение и время заменяются положением БТС и временем регистрации фрагмента, а содержимое сдвигается и экспоненциально затухает (назначение затухания описано далее в этом разделе). Во-вторых, фрагмент приводится к системе координат карты: «доворачивается» до её ориентации, масштабируется до её разрешения, и вычисляется смещение $patch_shift$ его левого верхнего угла относительно угла карты; если фрагмент старше карты, затухание применяется к нему, а не к ней. В-третьих, преобразованный фрагмент поэлементно суммируется с соответствующей ему областью карты (рисунок 4.4).

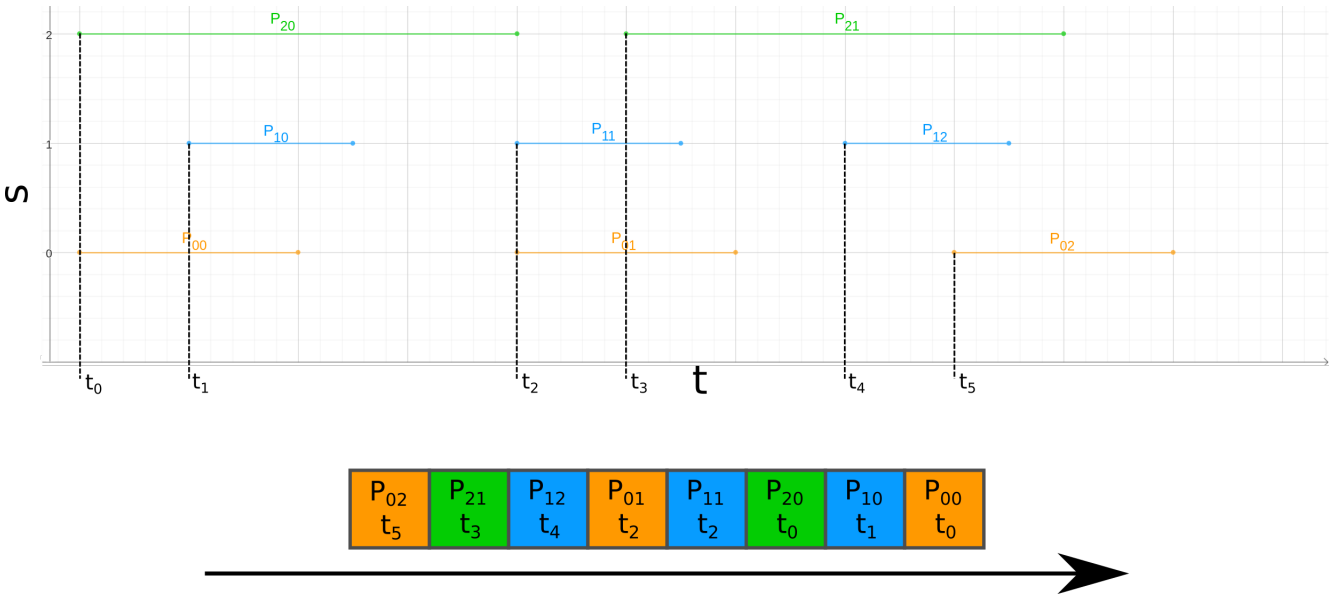


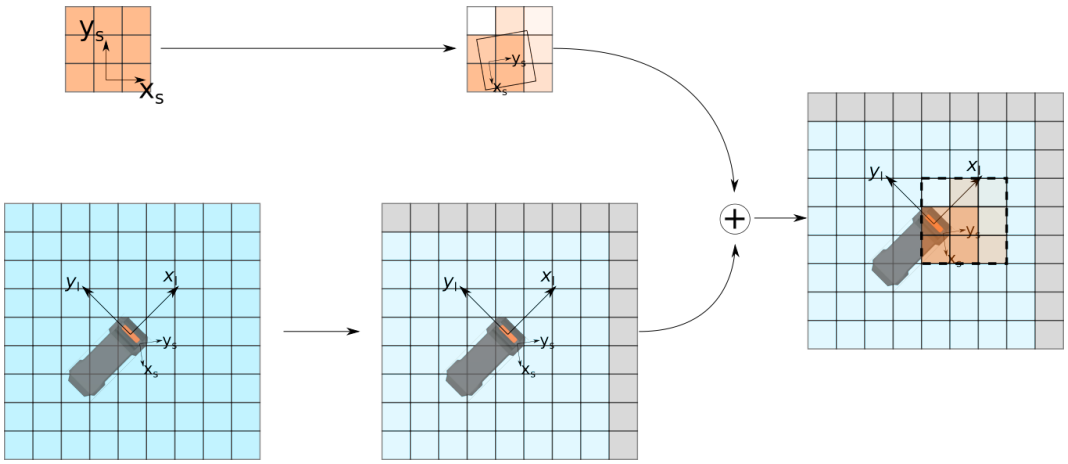
Рисунок 4.3 — Демонстрация формирования очереди фрагментов. Горизонтальные отрезки обозначают время формирования фрагмента каждым источником: начало отрезка соответствует времени регистрации данных $t_{s,k}$, конец отрезка соответствует времени завершения формирования фрагмента и моменту добавления его в очередь фрагментов. Внизу иллюстрации продемонстрирован итоговый порядок фрагментов в очереди: фрагменты извлекаются из очереди в порядке справа налево

Input: $P_{s,k}$ – фрагмент извлечённый из очереди;
$x_{s,k}$ – положение извлечённого фрагмента;
$t_{s,k}$ – время регистрации извлечённого фрагмента;
x_{i-1}, t_{i-1} – положение БТС и время при последнем обновлении карты;
M_{i-1} – карта проходимости пространства после последнего обновления;
res – разрешение карты проходимости;
res_s – разрешение извлечённого фрагмента;
Output: M_i – карта после добавления фрагмента

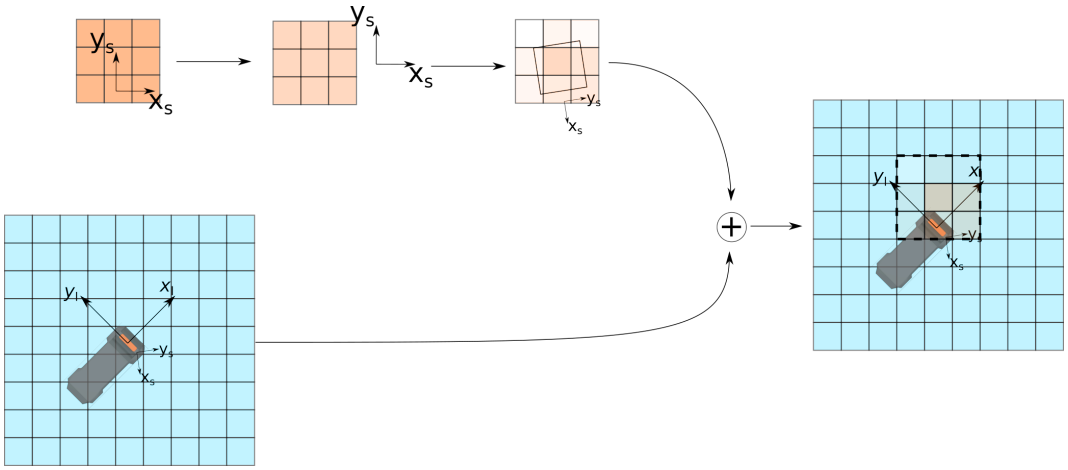
if $t_{s,k} > t_{i-1}$ **then**
  $x_i \leftarrow get_map_pose(t_{s,k})$;
  $t_i \leftarrow t_{s,k}$;
  $M_{actual} \leftarrow shift_map(M_{i-1}, x_i - x_{i-1})$;
  $M_{actual} \leftarrow dim(M_{actual}, t_i - t_{i-1})$;
else
  $x_i \leftarrow x_{i-1}$;
  $t_i \leftarrow t_{i-1}$;
  $M_{actual} \leftarrow M_{i-1}$;
end
$P_{aligned}, patch_shift \leftarrow align_patch(P_{s,k}, t_{s,k}, t_i, res/res_s)$;
$P_{aligned} \leftarrow dim(P_{aligned}, t_i - t_{s,k})$;
$M_i \leftarrow add_patch(M_{actual}, P_{aligned}, patch_shift)$;
return M_i

Алгоритм 4: Добавление фрагмента на карту: $update_map$

Недостатком обратной модели наблюдения является предположение о статичности сцены: в случае наличия динамических объектов в сцене, такие объекты оставляют за собой «след», который не удаляется с локальной карты проходимости пространства, если нет источника данных, способного отмечать области, в которых препятствия отсутствуют. Даже при наличии такого источника, «след» может попасть в его слепую зону и остаться на карте. Для решения данной проблемы применяется экспоненциальное затухание значений в клетках карты, постепенно возвращающее их в состояние «неизвестно» без повторных обновлений от источников данных. Операция затухания $dim(\widehat{M}, \Delta t)$, применяемая в листинге 4 как к карте, так и к фрагменту, поэлементно пересчитывает их значения:



а) Время регистрации фрагмента новее последнего времени обновления карты



б) Время регистрации фрагмента старше последнего времени обновления карты

Рисунок 4.4 — Иллюстрация алгоритма добавления фрагмента от источника данных s на локальную карту проходимости пространства

$$\widehat{M}_i^{j,k} = \exp(-a\,\Delta t)\;\widehat{M}_{i-1}^{j,k}, \tag{4.1}$$

где $\Delta t \geqslant 0$ – разница между текущим временем и временем регистрации $\widehat{M}$, а a – гиперпараметр, задающий скорость затухания.

Одного затухания, однако, недостаточно. Долго стоявшее неподвижно динамическое препятствие успевает накопить в занимаемых им клетках большой по модулю логарифм шансов ($|l_t| \gg 0$), и после того как препятствие сдвинулось, освободившееся пространство ещё долго квантуется как занятое: возврат таких клеток в состояние «неизвестно» или «свободно» за счёт одного лишь затухания занимает продолжительное время. Чтобы исключить такую «перезарядку», на шаге добавления фрагмента абсолютная величина значения клетки ограничивается сверху:

$$l_t = \max\left(\min\left[\text{inv_obs_model}(t), l_{thresh}\right], -l_{thresh}\right), \tag{4.2}$$

где inv_obs_model(t) – значение логарифма шансов, полученное по формуле (1.2), l_{thresh} – пороговое значение логарифма шансов по абсолютному значению.

4.3 Модификации базовой реализации

В данном разделе предлагаются модификации базовой реализации для снижения задержки между появлением информации о проходимости у одного из источников данных и её добавлением на итоговую карту проходимости пространства. Чтобы упростить изложение, пространство далее считается одномерным: карта в таком случае представляет собой одномерный массив M размера N, а движение возможно только влево или вправо. Размер одной клетки также принимается равным 1 метру, чтобы опустить операции с учётом разрешения карты res. В предыдущем разделе показано, что карта должна поддерживать операции сдвига и добавления фрагмента. Помимо них к описанию реализаций отдельно добавлена операция получения карты, поскольку в модификациях данная операция перестаёт быть тривиальным копированием массива и требует отдельного обсуждения.

4.3.1 Базовая реализация

Опишем исходную реализацию в одномерном случае. Карта представляет собой массив M размера N, текущую координату центра x_i и время t_i, которому это состояние карты соответствует (рисунок 4.5).

Сдвиг. При переходе в новую точку x_{i+1} в момент времени t_{i+1} массив со значениями логарифма шансов проходимости сдвигается на $\lfloor x_{i+1} - x_i \rceil$ клеток, а клетки, для которых прежних значений не существует, инициализируются нулём. Операция затрагивает весь массив, поэтому её вычислительная сложность $\Theta(N)$.

Добавление фрагмента. Операция повторяет листинг 4 с двумя упрощениями, следующими из одномерности задачи: «доворачивание» фрагмента не требуется, а из преобразований его координат остаётся лишь учёт сдвига карты

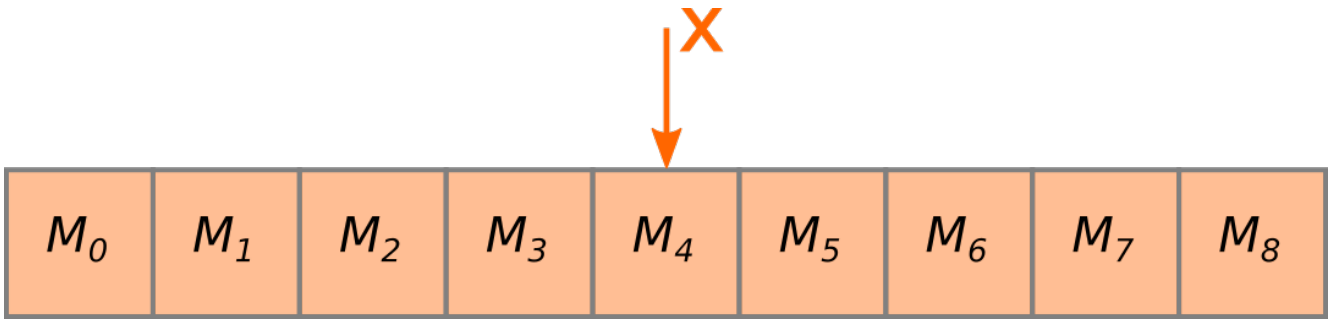


Рисунок 4.5 — Оригинальная реализация локальной карты проходимости пространства

за промежуток между моментом регистрации фрагмента и итоговым временем карты. Далее размер фрагмента P обозначается N_P.

Часть операции, предшествующая поэлементному суммированию, имеет сложность $\Theta(1)$, если $t_{s,k} \leqslant t_{i-1}$, и $\Theta(N)$ в противном случае, поскольку требует сдвига и затухания всей карты. Само суммирование выполняет $\Theta(N_P)$ операций, если фрагмент целиком помещается на карту, $\Theta(N)$ операций, если фрагмент больше карты и полностью её покрывает, и не выполняется вовсе, если фрагмент с картой не пересекается. Тем самым сложность суммирования составляет $O(\min(N_P, N))$, а всей операции – $O(N + \min(N_P, N))$.

Получение карты. Возвращается копия текущего массива, что требует $\Theta(N)$ операций. Копия необходима, чтобы на время передачи карты потребителям не блокировать последующие операции сдвига и добавления фрагментов.

4.3.2 Расширенная карта проходимости пространства

Первая модификация локальной карты проходимости пространства направлена на снижение стоимости сдвига. Массив для хранения карты увеличивается до размера $\widehat{N} > N$, чтобы описывать область больше отображаемой, а к координате x отображаемой части добавляется координата $\widehat{x}$ некоторой фиксированной точки «полного» массива; в качестве такой точки возьмём его центр (рисунок 4.6).

Сдвиг в общем случае сводится к обновлению значения x и потому требует $\Theta(1)$ операций. Содержимое массива сдвигается, как в базовой реализации, только когда отображаемая часть карты выходит за границы полного массива, то есть когда $\widehat{x} - x_{i+1} \notin [N/2;\ \widehat{N} - N/2)$; тогда содержимое сдвигается так, чтобы новый

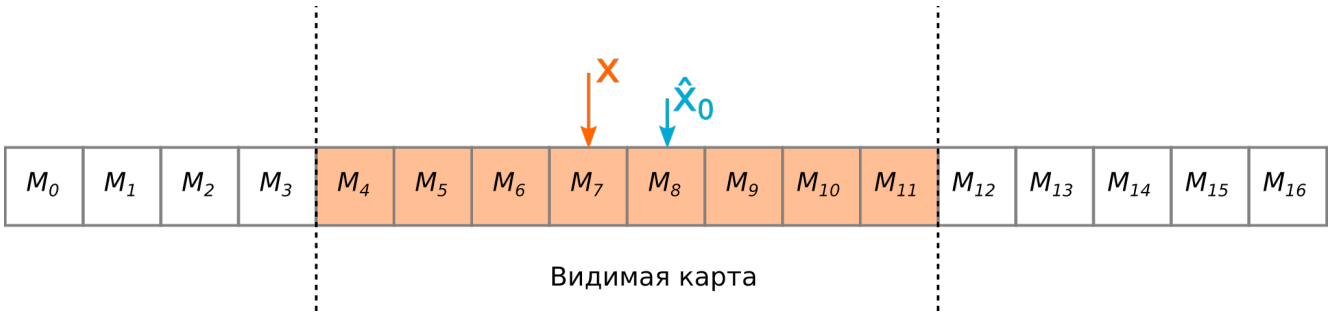


Рисунок 4.6 — Карта с предподготовленными окрестностями

$\widehat{x}$ совпал с актуальным x, что стоит $\Theta(\widehat{N})$ операций. При правильно выбранном размере массива амортизированная сложность операции составляет $\Theta(1)$.

Добавление фрагмента в точности повторяет одноимённую операцию базовой реализации. Её сложность – $O(\min(N_P, N))$, если затухание карты не выполняется, и $O(\widehat{N} + \min(N_P, N))$ в противном случае: затуханию подлежат и клетки за границами видимой области, поскольку они могли быть обновлены ранее и вернуться в неё при последующих сдвигах.

Получение карты заключается в копировании подмассива, соответствующего видимой части карты, и требует, как и прежде, $O(N)$ операций. Все операции по-прежнему остаются блокирующими.

4.3.3 Карта с разделяемыми блокировками

С ростом числа источников данных узким местом алгоритма становятся блокировки: источник ждёт своей очереди на доступ к карте даже тогда, когда добавляемые фрагменты между собой не пересекаются. Вторая модификация снижает эту задержку, изменяя структуру самой клетки карты. Вместо одного числа с плавающей точкой клетка хранит атомарную пару «значение логарифма шансов занятости – время, которому это значение соответствует». Требование, чтобы все клетки массива относятся к одному моменту времени, тем самым снимается. Размер атомарной структуры данных, то есть структуры, не использующей при чтении и записи примитивы блокировки (мьютексы, семафоры, сигналы и т.п.), должен быть согласован с возможностями аппаратного обеспечения. Здесь структура состоит из двух 32-битных полей, то есть занимает 64 бита,

а атомарные операции с данными такого размера поддерживаются инструкциями большинства современных архитектур [124].

Сдвиг не претерпел значительных изменений в сравнении с предыдущей реализацией: при копировании клеток во время «полноценного» сдвига копируется не только значение логарифма шансов, но и время, а новые клетки требуют инициализации времени (используется значение 0, поскольку оно не влияет на последующие обновления клетки). Сложность операции остаётся амортизированной $\Theta(1)$ и $\Theta(\widehat{N})$ в худшем случае, однако реальная константа увеличивается: копирование и инициализация атомарных структур заметно продолжительнее, чем у неатомарных аналогов.

Добавление фрагмента во многом повторяет одноимённую операцию предыдущей реализации. Сначала проверяется, требуется ли сдвиг карты. Если не требуется, операция выполняется одновременно с другими такими же операциями; если требуется, она дожидается исключительных прав на модификацию карты и производит сдвиг.

Затем корректируются координаты области, в которую добавляется фрагмент, и обновляются её клетки. Значение очередной клетки считывается атомарно, по нему с учётом затухания вычисляется новое значение (формула (4.1)), и результат записывается обратно операцией сравнения с обменом (Compare and Swap, CAS) – то есть только при условии, что за время вычисления значение клетки не изменилось. Если условие не выполнено, новое значение пересчитывается уже от обновившегося состояния клетки, и попытка повторяется до успешной записи. Описанный алгоритм приведён в листинге 5.

Теоретически данная операция может выполняться неограниченно долго, если между попытками сравнения с обменом происходит обновление значения клетки в другой операции. На практике незавершаемость операции практически невозможна ввиду в первую очередь ограниченного числа потоков, работающих одновременно (в рассматриваемом случае их 12, на современных вычислителях, используемых в робототехнике их число варьируется от 8 до 24). Если пренебречь единичными перерасчётами обновляемых значений, сложность такой операции становится $O(\min(N_P, N))$ ($O(\widehat{N} + \min(N_P, N))$, если во время выполнения операции произошёл «полноценный» сдвиг карты). Однако константа в этом случае будет заметно больше, чем в предыдущих вариантах из-за использования а) атомарных операций, которые в несколько раз вычислительно дороже неатомарных аналогов и б) shared_mutex, который имеет значительно большее время блоки-

Input: M – карта проходимости пространства;
P_s – добавляемый фрагмент;
$x_{t_{s,k}}$ – положение карты M в момент $t_{s,k}$;
$x_{s,k}$ – положение источника данных s в момент регистрации фрагмента;
Output: M после добавления фрагмента

```
// Область с возможностью одновременного добавления фрагментов
shared_lock(map_access_mutex);
x_{t_{s,k}} ← local occupancy map pose at t_{s,k};
if (Нужен сдвиг M) then
    // Начало критической секции
    unique_lock(map_access_mutex);
    if Сдвиг по-прежнему нужен и не произошёл в другом потоке до входа в
      критическую секцию then
        M ← shift_map(M, x_{t_{s,k}})
    else
        x_{s,k} ← x_{s,k} − (x_{t_i} − x_{t_{s,k}});
    end
    // Конец критической секции
else
    x_{s,k} ← x_{s,k} − (x_{t_i} − x_{t_{s,k}});
end
Δx_s ← round(x_{s,k} − x_i);
for j ← max(0, Δx_s) to min(N_P; N + Δx_s) do
    t_prev ← 0;
    val_prev ← 0;
    t_new ← t_{s,k};
    val_new ← P_s[j];
    while При CAS(M[j − Δx_s], {val_prev, t_prev}, {val_new, t_new}) обмен не произошёл do
        // Значение {val_prev, t_prev} актуализируются в случае неудачной
           операции сравнения с обменом
        if t_prev < t_{s,k} then
            t_new ← t_{s,k};
            val_new ← val_prev exp(−a(t_{s,k} − t_prev)) + P_s[j];
        else
            t_new ← t_prev;
            val_new ← val_prev + P_s[j] exp(−a(t_prev − t_{s,k}));
        end
    end
end
```

Алгоритм 5: Добавление фрагмента (модификация с разделяемыми блокировками): $update_map(P_s, t_{s,k}, x_{s,k})$

ровки в сравнении с обычным mutex-ом. Поэтому эта реализация становится выгоднее предыдущих вариантов при условии достаточного количества источников, производящих одновременное добавление фрагментов на карту.

Получение карты. Чтобы исключить состояние гонки с другими операциями, клетки видимой части карты копируются целиком, и одновременно среди них отыскивается наиболее позднее время t_{newest}. По копии строится карта «старого» формата – массив чисел с плавающей точкой, значения которого приведены к моменту t_{newest}. Сложность операции $O(N)$, но, как и при добавлении фрагмента, скрытая за O константа превышает прежнюю. Эта операция допускает одновременное выполнение с добавлением фрагментов, если частично обновлённое состояние карты приемлемо. Рассмотрим, например, такой сценарий:

1. Операция получения карты считала значение $M[0]$
2. Источник данных обновил значения клеток $M[0]$, $M[1]$
3. Операция получения карты считала значение $M[1]$

В этом случае только в $M[1]$ будут учтены показания источника данных. Естественно, в последующих вызовах эти показания будут учтены в обеих клетках. Если такая несогласованность данных считается недопустимой, операция получения карты должна быть блокирующей аналогично сдвигу.

4.3.4 Карта проходимости пространства с полным обновлением без блокировок

Последняя модификация снимает оставшееся ограничение и делает неблокирующей также операцию сдвига. Карта разбивается на два равных примыкающих друг к другу фрагмента размера $\widehat{N}$ каждый; к фрагменту добавляется координата его положения (как и ранее – центр), а сами фрагменты хранятся в разделяемых умных указателях pM_1, pM_2 (рисунок 4.7). Реализация опирается на возможность работать с такими указателями атомарно: для стандартной C++ реализации std::shared_ptr атомарные операции поддерживаются большинством современных архитектур.

Сдвиг. Прежде чем приступить к описанию операции, важно сделать следующее замечание: предлагаемая реализация сдвига является неблокирующей

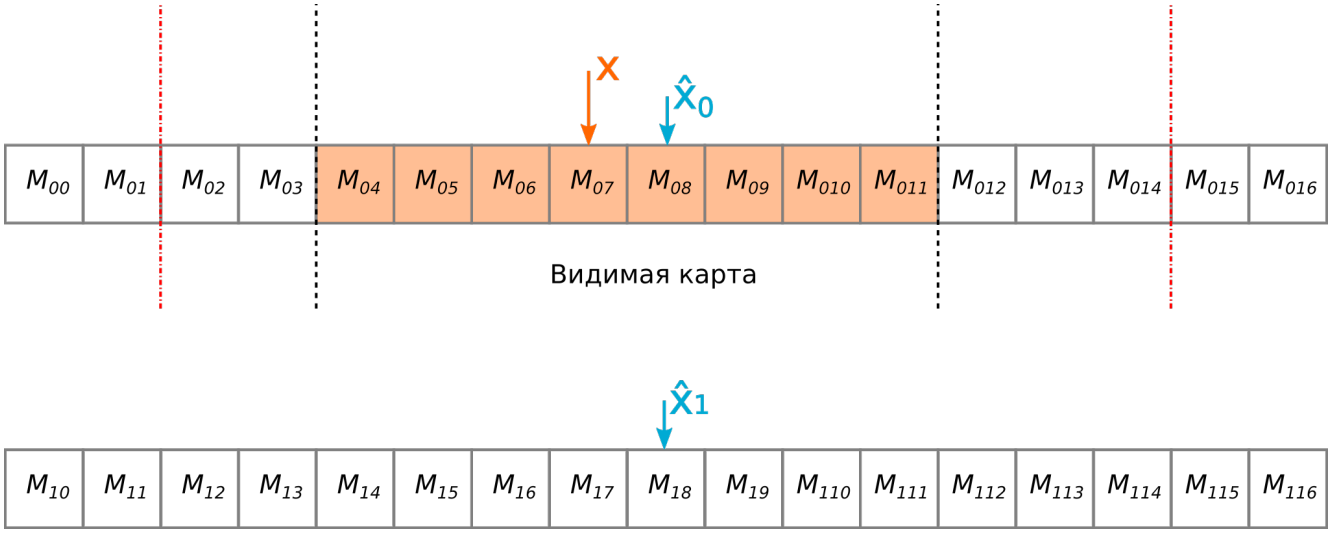


Рисунок 4.7 — Карта с полным обновлением без блокировок

относительно добавления фрагмента и получения карты, но в то же время, эта версия не может выполняться одновременно с другими операциями сдвига.

При сдвиге определяется положение отображаемой карты после сдвига. Если отображаемая часть карты присутствует целиком на одном фрагменте, и в результате сдвига отдаляется от его центра более чем на заданный порог Δ_{lim} и при этом так же отдаляется и от второго фрагмента (проще говоря, если дальнейшее движение карты в том же направлении приведет к выходу за границы всех фрагментов), то тот фрагмент, на котором сейчас не находится карта, заменяется на новый: все значения клеток (как и ранее это атомарные пары значения логарифма шансов и времени) инициализируются $\{0,0\}$, а его координата меняется таким образом, чтобы при дальнейшем движении в том же направлении отображаемая часть карты «перешла» на этот фрагмент (см. рисунок 4.8). В противном случае, как и ранее, положение отображаемой части карты просто обновляется (см. листинг 6). В данном алгоритме важно правильно подобрать Δ_{lim} таким образом, чтобы отображаемая часть карты не могла выйти за границы фрагмента, когда производится замена второго фрагмента (в противном случае это может повлиять на операцию получения карты, выполняемой одновременно со сдвигом).

Вычислительная сложность, как и ранее, $\Theta(1)$, в общем случае, когда обновляется только координата отображаемой части локальной карты проходимости пространства. $\Theta(\widehat{N})$, если происходит замена второго фрагмента.

Добавление фрагмента происходит аналогично предыдущим реализациям за исключением следующих моментов:

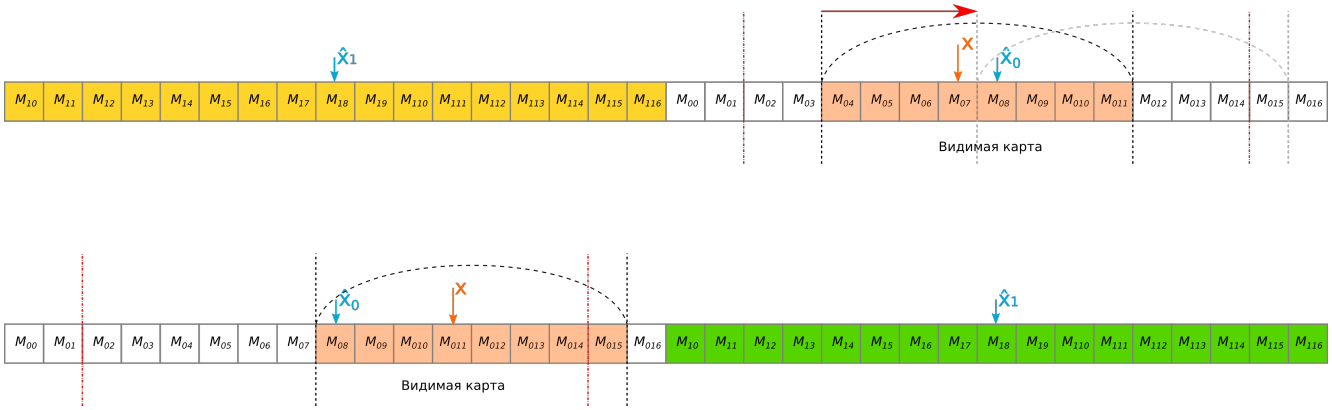


Рисунок 4.8 — Сдвиг карты проходимости пространства с полным обновлением без блокировок

Input: M – карта до сдвига;
x_i – положение M до сдвига;
x_{i+1} – новое положение карты
Output: M после сдвига

if x_{i+1} *расположен между центрами* M_1 *и* M_2 **then**
  return M
end
$M_c \leftarrow$ фрагмент (M_1 или M_2), на котором находится отображаемая карта;
$M_o \leftarrow$ оставшийся фрагмент;
$\widehat{x} \leftarrow$ центр M_c;
if $|\widehat{x} - x_{i+1}| > \Delta_{lim}$ **then**
  $M_{new} \leftarrow$ новый фрагмент размера $\widehat{N}$;
  for $i \leftarrow 0$ **to** $\widehat{N}$ **do**
    $M_{new}[i] \leftarrow \{0, 0\}$;
  end
  $M_o \leftarrow M_{new}$;
end
return M

Алгоритм 6: Сдвиг карты (реализация полностью без блокировок): $shift_map(M, x_{i+1})$

1. В самом начале операции происходит копирование указателей на фрагменты M_1, M_2, чтобы гарантировать их сохранение во время выполнения операции (на случай если одновременно с этой операцией выполняется сдвиг, заменяющий один из фрагментов).
2. Т.к. операция сдвига не может выполняться одновременно с другими операциями сдвига, в добавлении фрагмента эта операция более не происходит. Вместо этого она либо выполняется в отдельном потоке с определённой частотой (гарантирующей, что отображаемая часть карты всегда содержится на одном или обоих фрагментах карты), либо, например, происходит в операции получения локальной карты проходимости пространства (если гарантируется, что эта операция не выполняется одновременно с такой же)
3. Операция применяется последовательно к обоим фрагментам карты

Таким образом вычислительная сложность остаётся $O(\min(N_P, N))$. Реальная константа будет незначительно больше, чем в предыдущей реализации из-за атомарного копирования указателей и применения на двух фрагментах (заметим что последнее может быть выполнено одновременно, но асимптотику операции в общем случае это не изменит).

Получение карты также почти не отличается от предыдущей реализации: первым действием операция копирует указатели на фрагменты, чтобы гарантировать сохранность данных во время работы. Вычислительная сложность, как и раньше, $O(N)$.

Сводка вычислительной сложности операций для всех четырёх рассмотренных реализаций приведена в таблице 10.

4.4 Сравнение задержки обновления локальной карты проходимости пространства

Чтобы сравнить представленные модификации между собой, были подготовлены их реализации на C++ и синтетические тесты для измерения времени работы. В рамках этих тестов симулируются следующие сценарии:

– Сдвиг карты на 20 клеток.

Реализация	**Сдвиг**	**Добавление фрагмента**	**Получение карты**
Базовая	$\Theta(N)$	$O(N + \min(N_P,N))$	$\Theta(N)$
Расширенная карта	$\Theta(1)$ аморт.	$O(\min(N_P,N))$; $O(\widehat{N} + \min(N_P,N))$ при затухании карты	$O(N)$
С разделяемыми блокировками	$\Theta(1)$ аморт.	$O(\min(N_P,N))$, допускает одновременное выполнение	$O(N)$
С полным обновлением без блокировок	$\Theta(1)$	$O(\min(N_P,N))$, допускает одновременное выполнение	$O(N)$

Таблица 10 — Вычислительная сложность операций над локальной картой проходимости пространства для базовой реализации и трёх предложенных модификаций. Для операций, допускающих одновременное выполнение несколькими потоками, сложность указана для одного вызова

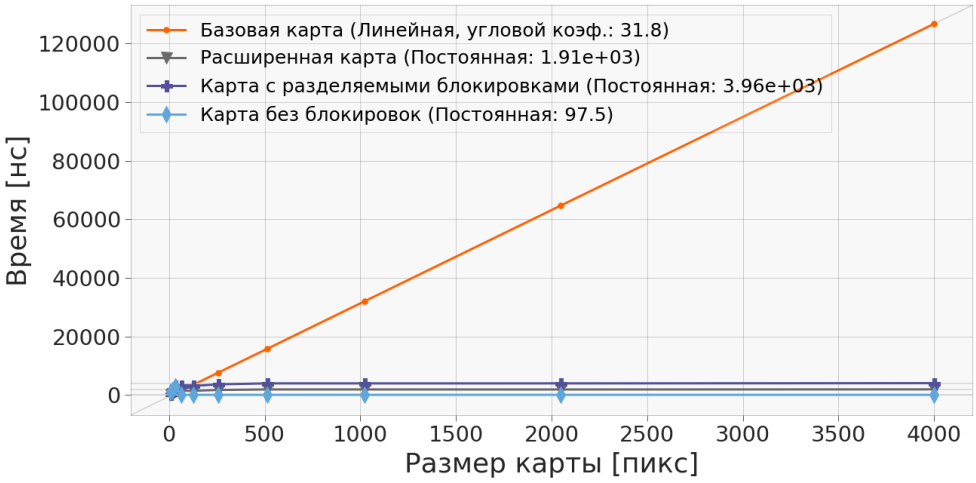
– Добавление фрагмента на карту в одном потоке. Размер фрагмента фиксирован и равен 100. Положение фрагмента меняется и берётся из циклической очереди: $-100, -50, -25, -1, 0, 1, 5, 10, 25, 50, 100$. Таким образом моделируются сценарии в которых только часть фрагмента попадает на локальную карту проходимости пространства.
– Добавление фрагмента на карту в нескольких потоках. Тот же сценарий, что и в предыдущем пункте, но запускается в нескольких потоках. Протестированы варианты с разным числом потоков: $1, 5, 9, 13$.
– Получение локальной карты проходимости пространства.
– Стандартный цикл «публикации» локальной карты проходимости пространства: В 8 потоках производится сдвиг карты на 10 клеток и добавления фрагмента размера 100, после завершения работы всех потоков, вызывается операция получения карты, имитирующая передачу карты потребителям.

Все эксперименты повторялись для «видимых областей» разного размера N: $10, 16, 32, 64, 128, 256, 512, 1024, 2048, 4000$. Для модификаций принималось значение $\widehat{N} = 2N$ (в случае с последней модификации с полным обновлением без блокировок полный размер карты с учётом неотображаемых областей составил $4N$). Результаты представленных измерений были получены на ПК с процессором AMD Ryzen 5 5600X с 6 ядрами и тактовой частотой 4,6 ГГц, ОЗУ объёмом 32 Гб.

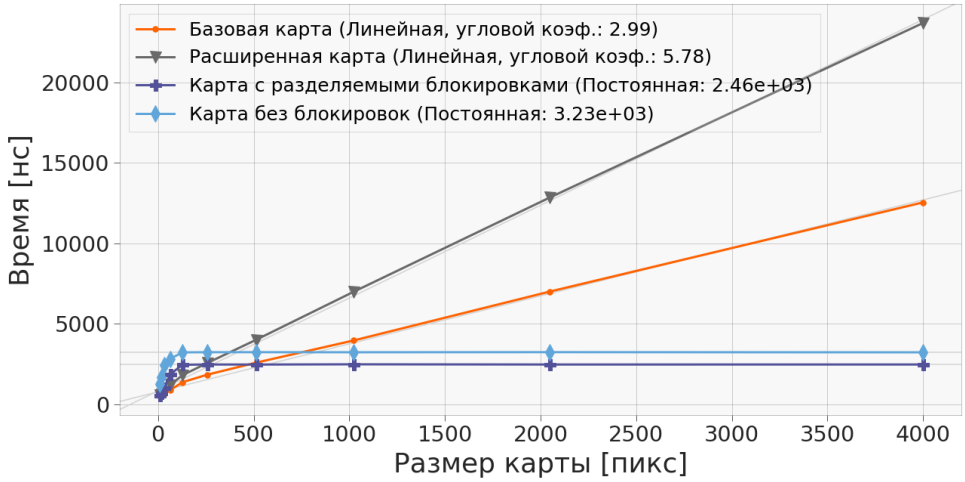
Замеры выполнялись средствами библиотеки Google Benchmark [125]. Число итераций для каждой точки подбирается библиотекой автоматически, исходя из требуемой длительности замера, и достигает $1{,}5 \cdot 10^6$. Результатом одного замера служит среднее время итерации. Каждый замер повторялся 10 раз. На графиках приведены средние значения, коэффициент вариации по повторам не превышает $2{,}5$ %. На графиках отложено астрономическое время выполнения, поскольку предметом сравнения является задержка обновления карты. Под коэффициентом линейной аппроксимации далее понимается угловой коэффициент прямой, построенной методом наименьших квадратов по всем десяти размерам карты. Из условий проведения замеров отметим, что масштабирование тактовой частоты процессора не отключалось.

Сдвиг карты. Результаты замеров приведены на рисунке 4.9 а. Исходная реализация растёт линейно, как и было показано при обсуждении её сложности. Остальные реализации имеют в среднем константное время выполнения, поскольку в общем случае требуют лишь обновления координаты x, однако их константы различаются. Быстрее всех оказывается реализация с полным обновлением без блокировок, не использующая мьютексы для создания критических секций; далее следует расширенная карта, использующая стандартный std::mutex; худшее время среди модификаций показывает карта с разделяемыми блокировками, поскольку манипуляции с std::shared_mutex вычислительно более интенсивны. Отметим также, что на малых размерах карты исходная реализация эффективнее всех модификаций.

Добавление фрагмента в одном потоке. Результаты приведены на рисунке 4.9 б. Как и в предыдущем сценарии, исходная реализация имеет линейную сложность; ту же сложность показывает и расширенная карта, что объясняется необходимостью приводить все клетки массива к одному моменту времени. Этим же объясняется и большее время расширенной карты в сравнении с исходной: ей требуется обновить большее число клеток. Оставшиеся две модификации с ростом размера карты выходят на постоянное число операций, так как обновляют не всю карту, а только клетки, пересекающиеся с фрагментом. При этом реализация без блокировок уступает реализации с разделяемыми блокировками: вероятная причина – атомарное копирование указателей на фрагменты, обращение к элементам через указатель и повторение операции на двух массивах; последние два фактора снижают локальность данных и увеличивают число промахов кэша.



а) Сдвиг карты.



б) Добавление фрагмента в одном потоке.

Рисунок 4.9 — Графики зависимости времени выполнения операций над локальной картой проходимости пространства от размера карты

Добавление фрагмента в нескольких потоках. Результаты замеров приведены на рисунке 4.10. Здесь применимы рассуждения предыдущего сценария; сценарий с одним потоком отличается от предыдущего замера только наличием обвязки для запуска в пуле потоков. Можно отметить, что с ростом числа потоков уменьшается размер карты, начиная с которого модификации с частичным или полным отказом от блокировок становятся вычислительно эффективнее исходной реализации.

Получение карты. Результаты приведены на рисунке 4.11 а и заметно отличаются от предыдущих сценариев. Время получения карты у исходной реализации и у расширенной карты с ростом размера карты возрастает настолько

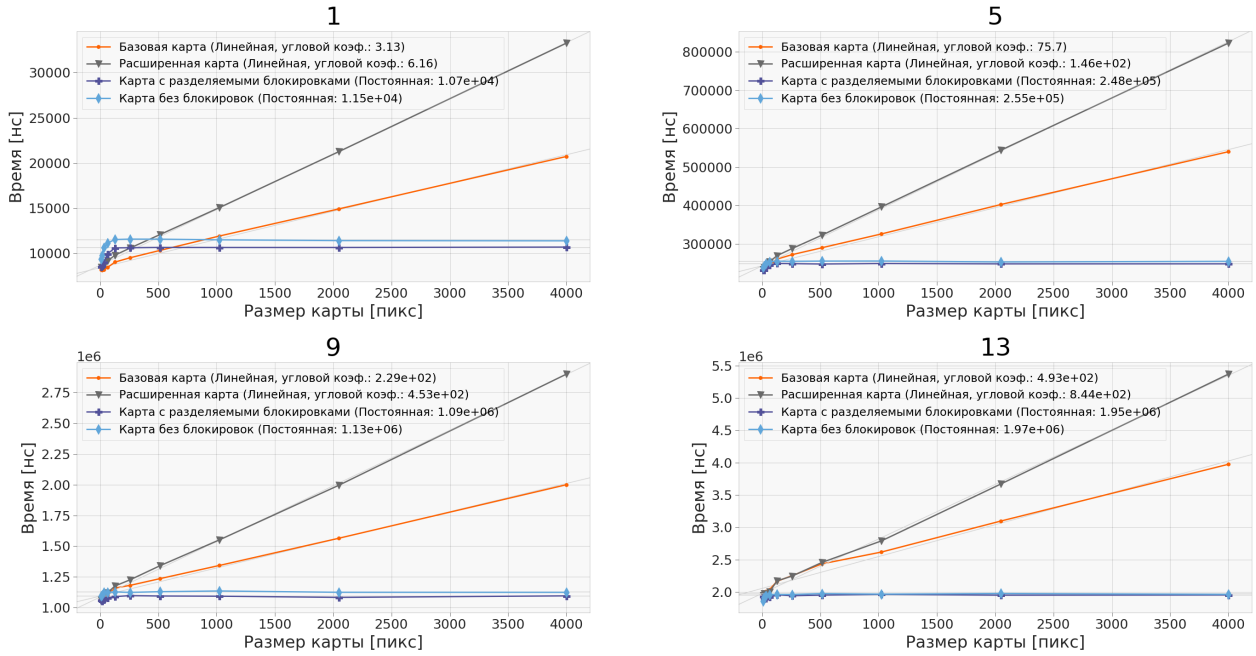
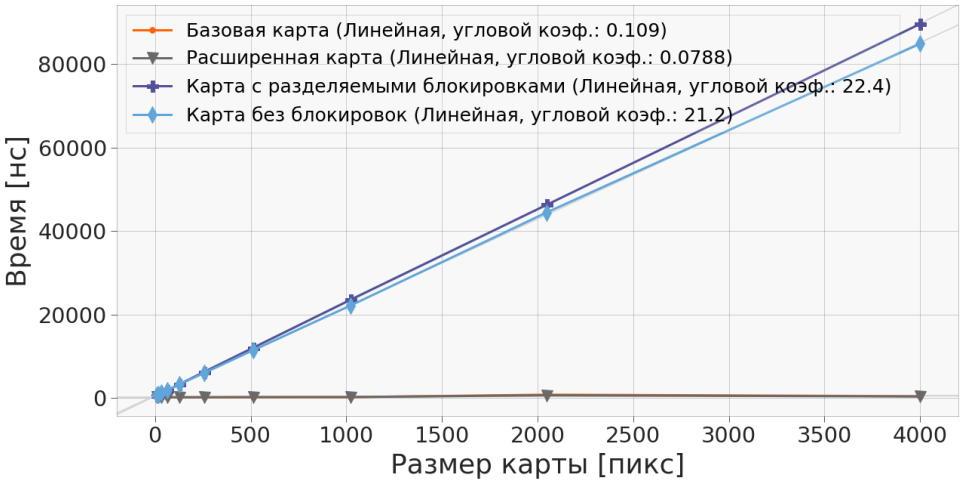


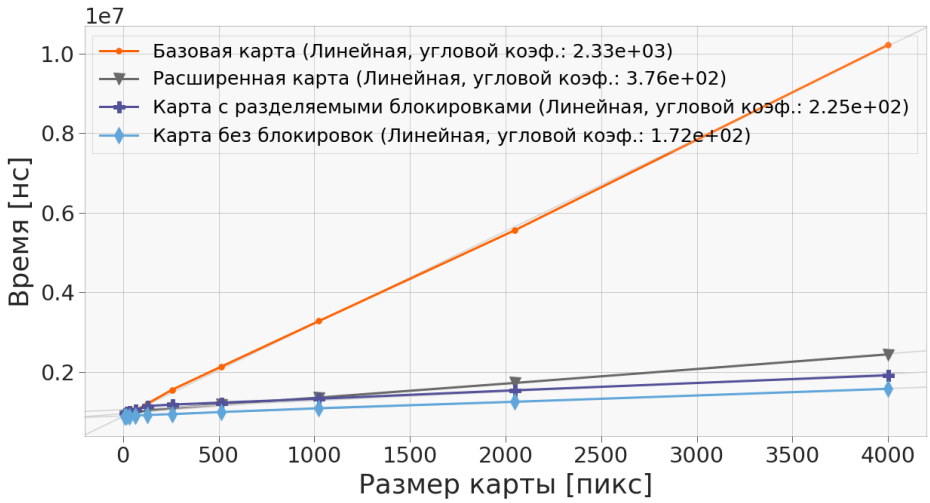
Рисунок 4.10 — Графики зависимости времени добавления фрагмента на карту от размера карты при 1, 5, 9 и 13 потоках

медленно, что на фоне реализаций с частичным или полным отказом от блокировок кажется константным, хотя и остаётся линейным. У последних же из-за необходимости атомарно считывать значения клеток и приводить их к общему моменту времени время работы увеличивается значительно быстрее.

Стандартный цикл «публикации» карты. Это наиболее показательный сценарий: он определяет, какая реализация окажется лучше в реальном режиме работы. Результаты приведены на рисунке 4.11 б. Все реализации имеют линейную вычислительную сложность относительно размера карты, и все предложенные модификации превосходят исходную реализацию, причём каждая следующая модификация показывает себя лучше предыдущей – по крайней мере по мере увеличения размера карты. Лучшую производительность показала реализация с полным обновлением без блокировок: экспериментально установленный для неё коэффициент линейной аппроксимации зависимости времени одной итерации от размера карты в $13{,}5$ раза меньше, чем у исходной реализации. Далее следует модификация с разделяемыми блокировками (в $10{,}3$ раза быстрее исходной реализации). Наконец, расширенная карта при всей своей тривиальности дала прирост производительности в $6{,}2$ раза.



а) Получение карты. График базовой реализации практически совпадает с расширенной картой и на таком масштабе от неё неотличим.



б) Одна итерация «публикации» карты.

Рисунок 4.11 — Графики зависимости времени получения карты и времени одной итерации «публикации» карты от размера карты

4.5 Реализация финальной модификации для двумерной карты проходимости пространства

Показав работоспособность и лучшую производительность модификации локальной карты проходимости пространства с полным обновлением без блокировок в одномерном случае, представим её расширение для двумерного случая. Как и ранее, полная карта окружения разбивается на несколько, теперь уже двумерных, массивов, состоящих из уже описанных ранее атомарных пар логарифма шансов и времени. Для движения в плоскости потребуется 9 массивов (рисунок 4.12). Если видимая область карты пересекает границы, отмеченные штрих-пунктирной линией, создаются новые массивы и заменяют часть предыдущих таким образом, чтобы текущий массив, в котором находится видимая область карты, стал центральным. Важно отметить, что отступ этих линий от краёв массива должен быть подобран так, чтобы за одну итерацию сдвига, видимая область не могла преодолеть расстояние между линией и границей массива. В противном случае если параллельно будет происходить получение карты, то массив, в который «переходит» видимая область, может быть ещё не создан, что приведёт к ошибке выполнения.

Плата за такую реализацию двояка. Во-первых, она значительно увеличивает объём памяти, требуемой для хранения структуры данных. Во-вторых, как и в одномерном случае, операции получения карты и добавления фрагмента применяются ко всем 9 массивам, что увеличивает коэффициент линейной аппроксимации времени их выполнения. Определение условий, при которых такая реализация вычислительно эффективнее исходной, требует дополнительных экспериментов: результат зависит от средней скорости движения БТС, среднего размера добавляемых фрагментов, числа источников данных и частоты их работы, а также от масштаба локальной карты проходимости пространства. Тем самым установленный в разделе 4.4 выигрыш относится к модельной одномерной задаче; перенос двумерной реализации на бортовой вычислитель БТС и измерение задержки обновления карты в тех же условиях, в которых выполнено профилирование раздела 4.1, составляют предмет дальнейшей работы.

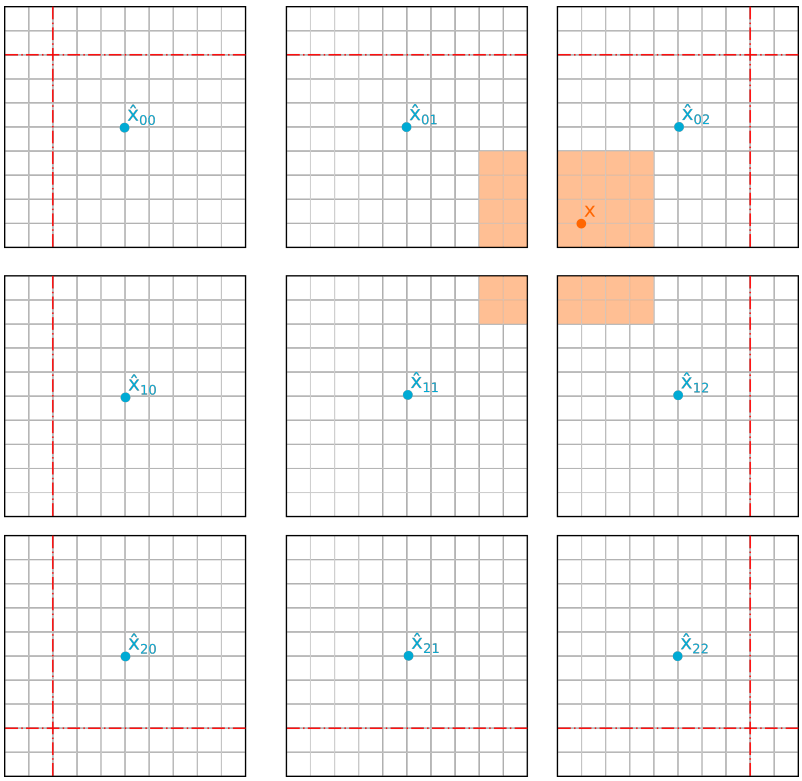


Рисунок 4.12 — Двумерная реализация локальной карты проходимости пространства с полным обновлением без блокировок

4.6 Заключение к главе 4

В главе получены следующие результаты:

1. выполнено профилирование программного комплекса, представленного в главе 3, при одном и восьми одновременно подключённых источниках данных. Время обновления карты составило $3{,}03$ и $22{,}78$ мс соответственно. В обоих случаях удовлетворяется условие 1 раздела 1.7.2 – учёт данных до поступления следующих (100 мс при частоте регистрации 10 Гц), – однако при восьми источниках допустимая по условию 2 задержка 9 мс, при которой разрешение карты $0{,}1$ м остаётся содержательным, превышена примерно в $2{,}5$ раза. Рост задержки более чем в семь раз при переходе от одного источника к восьми показывает, что последовательное добавление фрагментов ограничивает дальнейшее наращивание их числа;
2. предложены три последовательные модификации структуры данных для представления карты проходимости пространства, направленные на снижение вычислительной сложности производимых операций;

3. произведена оценка вычислительной сложности предложенных алгоритмов;
4. алгоритмы реализованы в виде программного кода на языке C++ и протестированы для упрощённого сценария одномерной карты проходимости пространства. Полученные при тестировании результаты:
    а) теоретические оценки сложности алгоритмов подтверждены экспериментально;
    б) лучшую скорость добавления фрагмента на карту в одном и нескольких потоках показала вторая модификация – карта с разделяемыми блокировками;
    в) лучшую скорость получения карты проходимости показала оригинальная реализация карты;
    г) лучшую скорость сдвига карты показала последняя модификация – карта проходимости пространства с полным обновлением без блокировок;
    д) в симуляции одного цикла публикации карты проходимости пространства лучшая производительность продемонстрирована последней модификацией: коэффициент линейной аппроксимации зависимости времени итерации от размера карты уменьшился в $13{,}5$ раза относительно базовой реализации;
5. предложено расширение последней модификации карты проходимости пространства для двумерного пространства.

Отметим, что достигнутый выигрыш представляет собой размен, а не улучшение по всем показателям сразу. Он получен на составном цикле публикации карты и оплачен замедлением операции получения карты: коэффициент линейной аппроксимации для неё возрастает с $0{,}109$ до $21{,}2$ нс на элемент, то есть почти на два порядка. К этому добавляется рост потребления памяти – в одномерном случае вчетверо относительно отображаемой области. Выигрыш достигается за счёт того, что в цикле публикации операции добавления фрагмента и сдвига карты выполняются многократно чаще операции её получения.

Перечисленные основные результаты и другие авторские материалы главы опубликованы в работе [128].

Заключение

Основные результаты работы заключаются в следующем:

1. Выполнен анализ моделей окружающего пространства БТС, алгоритмов их построения и способов комплексирования разнородных данных. На его основе обоснован выбор карты проходимости пространства в качестве модели, описывающей проходимость окружающего БТС пространства, байесовского вывода с использованием обратной модели наблюдения в качестве алгоритма её построения, а также коэффициента Жаккара и метрики PFC-MSE в качестве метрик оценки её качества. Сформулированы требования к разрабатываемым модели и алгоритмам её построения.
2. Предложен алгоритм оценки положения окружающих ТС на основе комплексирования визуальных и лидарных данных, выделяющий точки поверхности движения оценкой её формы регрессией на гауссовских процессах. На реальных данных алгоритм показал средний коэффициент Жаккара $0{,}439$ против $0{,}399$ у лучшего лидарного и $0{,}337$ у визуального алгоритмов, не использующих обучаемых преобразований в вид сверху. Алгоритм сохраняет превосходство во всём диапазоне рассмотренных дальностей.
3. Предложен алгоритм проецирования семантической сегментации проезжей части с изображения на оценённую по лидарным данным поверхность движения, а также способ его объединения с базовым алгоритмом построения карты проходимости пространства, использующий конкурирующее комплексирование. Результаты экспериментов показали, что полученное объединение алгоритмов снижает среднеквадратичную ошибку стоимости поиска пути (PathFinding Cost Mean Squared Error, PFC-MSE) с $1019{,}3$ до $924{,}4$ в пяти рассмотренных сценариях, а среднеквадратичную ошибку относительно эталонной карты – с $0{,}231$ до $0{,}218$. Проецирование той же сегментации на фиксированную плоскость увеличивает PFC-MSE до $1059{,}4$.
4. Разработан программный комплекс построения карты проходимости пространства на основе комплексирования разнородных данных от датчиков, установленных на БТС, и их обработчиков. Архитектура

программного комплекса позволяет осуществлять параллельную подготовку фрагментов карты по показаниям независимых источников данных, оставляя последовательным только этап добавления фрагментов в общую карту.

5. Реализованы три последовательные модификации структуры данных для представления карты проходимости пространства, направленные на снижение вычислительной сложности производимых операций. Тестирование на синтетических данных, моделирующих одномерную карту проходимости пространства, показало, что наибольший выигрыш для одного цикла публикации карты даёт финальная модификация – карта проходимости пространства с полным обновлением без блокировок: коэффициент линейной аппроксимации зависимости времени итерации от размера карты уменьшается в $13{,}5$ раза относительно исходной реализации.
6. Предложено расширение последней модификации карты проходимости пространства для двумерного пространства.

По результатам работы могут быть даны следующие рекомендации. При построении карты проходимости пространства на БТС, оснащённых камерами и лидарами, проецирование результатов обработки изображения целесообразно выполнять на оценённую по дальномерным данным поверхность движения, а не на фиксированную плоскость: как показано во второй главе, отказ от предположения о плоской поверхности улучшает описание сцены даже по сравнению с алгоритмом, вовсе не использующим визуальные данные. Объединение фрагментов карты, построенных по разным источникам данных, следует выполнять конкурирующим комплексированием, переводя элементы карты, о проходимости которых источники дают противоречивые сведения, в состояние «неизвестно». Наконец, при проектировании программного обеспечения построения карты источники данных целесообразно подключать через единый интерфейс поставщика фрагментов, что позволяет наращивать их число без изменения кода построения карты.

Перспективы дальнейшей разработки темы состоят в следующем. Двумерная реализация карты проходимости пространства с полным обновлением без блокировок требует переноса и валидации на реальной платформе, а также измерения задержки обновления карты при различном числе источников данных в тех же условиях, в которых выполнено профилирование программного комплекса.

Это позволит определить условия, при которых она вычислительно эффективнее исходной реализации. Разработанный программный комплекс допускает подключение поставщиков фрагментов карты по данным радаров и сонаров, однако качество получаемой при этом модели в настоящей работе не исследовалось. Отдельным направлением является переход к динамической карте проходимости пространства, хранящей вместе с занятостью элемента оценку скорости наблюдаемого препятствия, что позволило бы отказаться от механизма затухания как основного способа учёта подвижных объектов. Отдельного внимания заслуживает используемая в работе метрика PFC-MSE: стоимости путей в ней вычисляются без учёта кинематической модели движения и габаритов БТС, из-за чего области, недостижимые для БТС, дают вклад в итоговое значение метрики наравне с достижимыми. Учёт кинематической модели движения и габаритов БТС при построении графа сделал бы оценку влияния ошибок карты на алгоритмы планирования пути ближе к тому, как карта используется в действительности. Наконец, представляет интерес оценка предложенных алгоритмов на публичных наборах данных: их применение потребует либо подбора наборов с сопоставимой конфигурацией датчиков, либо построения эталонной разметки проходимости пространства по имеющимся в них аннотациям.

Автор выражает благодарность своему научному руководителю О. С. Шипитько за помощь в постановке задач и обсуждении результатов.

В заключение благодарю коллектив компании «ЭвоКарго». Отдельно хочу поблагодарить К. В. Садовскова, А. П. Рудоманенко, С. А. Скрипицина, П. А. Комиссарову, А. Д. Воронкова за дискуссии, обсуждение результатов и помощь в проведении экспериментов. Благодарю авторов шаблона *Russian-Phd-LaTeX-Dissertation-Template* за помощь в оформлении работы.

Список сокращений и условных обозначений

БТС	Беспилотное транспортное средство
ГПУ	Графическое процессорное устройство
МНК	Метод наименьших квадратов
ОЗУ	Оперативное запоминающее устройство
ПК	Персональный компьютер
ПО	Программное обеспечение
ТС	Транспортное средство
ЦПУ	Центральное процессорное устройство
BEV	Bird’s-eye view
CAS	Compare and Swap
CUDA	Compute Unified Device Architecture
FN	False Negative
FP	False Positive
GPR	Gaussian Process Regression
LiDAR	Light Detection and Ranging
PCL	Point Cloud Library
PFC-MSE	PathFinding Cost Mean Squared Error
RADAR	Radio Detection and Ranging
RANSAC	Random Sample Consensus
RGB	Red, Green, Blue
ROS	Robot Operating System
RSS	Residual Sum of Squares
TP	True Positive
UML	Unified Modeling Language

Список литературы

1. *SAE On-Road Automated Vehicle Standards Committee*. Taxonomy and definitions for terms related to on-road motor vehicle automated driving systems [Текст] : Recommended Practice / SAE On-Road Automated Vehicle Standards Committee ; SAE International. — 2014. — J3016. — действующая редакция J3016_202104.
2. Lane departure warning systems and lane line detection methods based on image processing and semantic segmentation: A review [Текст] / W. Chen [и др.] // Journal of traffic and transportation engineering (English edition). — 2020. — Т. 7, № 6. — С. 748—774.
3. Blind spot monitoring in light vehicles—System performance [Текст] : тех. отч. / G. Forkenbrock [и др.] ; United States. National Highway Traffic Safety Administration. — 2014.
4. Autonomous emergency braking test results [Текст] / W. Hulshof [и др.] // Proceedings of the 23rd International Technical Conference on the Enhanced Safety of Vehicles (ESV). — National Highway Traffic Safety Administration Washington, DC, USA. 2013. — С. 1—13.
5. *Pananurak, W.* Adaptive cruise control for an intelligent vehicle [Текст] / W. Pananurak, S. Thanok, M. Parnichkun // 2008 IEEE International Conference on Robotics and Biomimetics. — IEEE. 2009. — С. 1794—1799.
6. A situation-adaptive lane-keeping support system: Overview of the safelane approach [Текст] / A. Amditis [и др.] // IEEE Transactions on Intelligent Transportation Systems. — 2010. — Т. 11, № 3. — С. 617—629.
7. Research on automatic parking systems based on parking scene recognition [Текст] / S. Ma [и др.] // IEEE Access. — 2017. — Т. 5. — С. 21901—21917.
8. Test procedures traffic jam assist test development considerations [Текст] : тех. отч. / S. J. Rao, G. J. Forkenbrock [и др.] ; United States. Department of Transportation. National Highway Traffic Safety … — 2019.
9. Waymo public road safety performance data [Электронный ресурс] / M. Schwall [и др.] // arXiv preprint arXiv:2011.00038. — 2020. — URL: https://arxiv.org/abs/2011.00038 (дата обр. 08.09.2026).

10. *Ingle, S.* Tesla autopilot: semi autonomous driving, an uptick for future autonomy [Текст] / S. Ingle, M. Phute // International Research Journal of Engineering and Technology. — 2016. — Т. 3, № 9. — С. 369—372.

11. *Чистяков, А.* Как Яндекс делает обычные автомобили беспилотными [Электронный ресурс] / А. Чистяков. — 11.2021. — URL: https://habr.com/ru/companies/yandex/articles/585444/ (дата обр. 30.08.2025).

12. Baidu apollo em motion planner [Электронный ресурс] / H. Fan [и др.] // arXiv preprint arXiv:1807.08048. — 2018. — URL: https://arxiv.org/abs/1807.08048 (дата обр. 08.09.2026).

13. *Freedman, I. G.* Autonomous vehicles are cost-effective when used as taxis [Текст] / I. G. Freedman, E. Kim, P. A. Muennig // Injury epidemiology. — 2018. — Т. 5, № 1. — С. 24.

14. Level-5 autonomous driving—Are we there yet? A review of research literature [Текст] / M. A. Khan [и др.] // ACM Computing Surveys (CSUR). — 2022. — Т. 55, № 2. — С. 1—38.

15. An overview of autonomous vehicles sensors and their vulnerability to weather conditions [Текст] / J. Vargas [и др.] // Sensors. — 2021. — Т. 21, № 16. — С. 5397.

16. *Сысоева, С.* Актуальные технологии и применения датчиков автомобильных систем активной безопасности. Часть 6. Радары [Текст] / С. Сысоева // Компоненты и технологии. — 2007. — № 68. — С. 67—76.

17. *Wandinger, U.* Introduction to lidar [Текст] / U. Wandinger // Lidar: range-resolved optical remote sensing of the atmosphere. — Springer, 2005. — С. 1—18.

18. An overview of sensors in Autonomous Vehicles [Текст] / H. A. Ignatious, M. Khan [и др.] // Procedia Computer Science. — 2022. — Т. 198. — С. 736—741.

19. *Kleeman, L.* Sonar sensing [Текст] / L. Kleeman, R. Kuc // Springer handbook of robotics. — Springer, 2016. — С. 753—782.

20. Навигация беспилотных автономных аппаратов на основе регистрации и передачи наблюдений окружающего ландшафта [Текст] / А. Б. Миллер [и др.] // Известия Иркутского государственного университета. Серия: Математика. — 2025. — Т. 53. — С. 18—34. — URL: http://dx.doi.org/10.26516/1997-7670.2025.53.18.

21. *Yudin, D.* M3DMap: Object-aware Multimodal 3D Mapping for Dynamic Environments [Электронный ресурс] / D. Yudin. — 2025. — arXiv: 2508.17044 [cs.CV]. — URL: https://arxiv.org/abs/2508.17044 (дата обр. 08.09.2026).

22. Scene representations for autonomous driving: an approach based on polygonal primitives [Текст] / M. Oliveira [и др.] // Robot 2015: Second Iberian Robotics Conference: Advances in Robotics, Volume 1. — Springer. 2015. — С. 503—515.

23. *Dryanovski, I.* Multi-volume occupancy grids: An efficient probabilistic 3D mapping model for micro aerial vehicles [Текст] / I. Dryanovski, W. Morris, J. Xiao // 2010 IEEE/RSJ International Conference on Intelligent Robots and Systems. — IEEE. 2010. — С. 1553—1559.

24. *Feng, T.* A survey of world models for autonomous driving [Электронный ресурс] / T. Feng, W. Wang, Y. Yang // arXiv preprint arXiv:2501.11260. — 2025. — URL: https://arxiv.org/abs/2501.11260 (дата обр. 08.09.2026).

25. 3D object detection for autonomous driving: A comprehensive survey [Текст] / J. Mao [и др.] // International Journal of Computer Vision. — 2023. — Т. 131, № 8. — С. 1909—1963.

26. nuscenes: A multimodal dataset for autonomous driving [Текст] / H. Caesar [и др.] // Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. — 2020. — С. 11621—11631.

27. Multi-Object Tracking with Camera-LiDAR Fusion for Autonomous Driving [Электронный ресурс] / R. Pieroni [и др.]. — 2024. — arXiv: 2403.04112 [cs.RO]. — URL: https://arxiv.org/abs/2403.04112 (дата обр. 08.09.2026).

28. MCTrack: A Unified 3D Multi-Object Tracking Framework for Autonomous Driving [Электронный ресурс] / X. Wang [и др.]. — 2024. — arXiv: 2409.16149 [cs.CV]. — URL: https://arxiv.org/abs/2409.16149 (дата обр. 08.09.2026).

29. *Altché, F.* An LSTM network for highway trajectory prediction [Текст] / F. Altché, A. de La Fortelle // 2017 IEEE 20th international conference on intelligent transportation systems (ITSC). — IEEE. 2017. — С. 353—359.

30. Trajectron++: Dynamically-Feasible Trajectory Forecasting with Heterogeneous Data [Текст] / T. Salzmann [и др.] // Computer Vision – ECCV 2020. — Springer International Publishing, 2020. — С. 683—700. — URL: http://dx.doi.org/10.1007/978-3-030-58523-5_40.

31. *Thrun, S.* Learning occupancy grid maps with forward sensor models [Текст] / S. Thrun // Autonomous robots. — 2003. — Т. 15, № 2. — С. 111—127.

32. A random finite set approach for dynamic occupancy grid maps with real-time application [Текст] / D. Nuss [и др.] // The International Journal of Robotics Research. — 2018. — Июль. — Т. 37, № 8. — С. 841—866. — URL: http://dx.doi.org/10.1177/0278364918775523.

33. *Souza, A.* Occupancy-elevation grid: an alternative approach for robotic mapping and navigation [Текст] / A. Souza, L. M. G. Gonçalves // Robotica. — 2015. — Апр. — Т. 34, № 11. — С. 2592—2609. — URL: http://dx.doi.org/10.1017/S0263574715000235.

34. *Paskin, M.* Robotic mapping with polygonal random fields [Электронный ресурс] / M. Paskin, S. Thrun // arXiv preprint arXiv:1207.1399. — 2012. — URL: https://arxiv.org/abs/1207.1399 (дата обр. 08.09.2026).

35. Generating evidential BEV maps in continuous driving space [Текст] / Y. Yuan [и др.] // ISPRS Journal of Photogrammetry and Remote Sensing. — 2023. — Т. 204. — С. 27—41.

36. *Moravec, H.* High resolution maps from wide angle sonar [Текст] / H. Moravec, A. Elfes // Proceedings. 1985 IEEE international conference on robotics and automation. Т. 2. — IEEE. 1985. — С. 116—121.

37. *Elfes, A.* Occupancy grids: a probabilistic framework for mobile robot perception and navigation [Текст] : дис. … канд. / Elfes Alberto. — Ph. D. Thesis. Department of electrical, computer engineering. Carnegie …, 1989.

38. *Moravec, H. P.* Sensor Fusion in Certainty Grids for Mobile Robots [Текст] / H. P. Moravec // AI Magazine. — 1988. — Т. 9, № 2. — С. 61. — URL: https://ojs.aaai.org/aimagazine/index.php/aimagazine/article/view/676.

39. *Thrun, S.* Probabilistic Robotics [Текст] / S. Thrun, W. Burgard, D. Fox. — Cambridge, MA : MIT Press, 2005.

40. A single LiDAR-based feature fusion indoor localization algorithm [Текст] / Y.-T. Wang [и др.] // Sensors. — 2018. — Т. 18, № 4. — С. 1294.

41. *Luo, Z.* A probability occupancy grid based approach for real-time LiDAR ground segmentation [Текст] / Z. Luo, M. V. Mohrenschildt, S. Habibi // IEEE Transactions on Intelligent Transportation Systems. — 2019. — Т. 21, № 3. — С. 998—1010.

42. Data driven prediction architecture for autonomous driving and its application on apollo platform [Текст] / K. Xu [и др.] // 2020 IEEE Intelligent Vehicles Symposium (IV). — IEEE. 2020. — С. 175—181.

43. *Rosenband, D. L.* Inside Waymo's self-driving car: My favorite transistors [Текст] / D. L. Rosenband // 2017 Symposium on VLSI Circuits. — IEEE. 2017. — С. C20—C22.

44. Experience of the ARGO autonomous vehicle [Текст] / M. Bertozzi [и др.] // Enhanced and Synthetic Vision 1998. Т. 3364. — SPIE. 1998. — С. 218—229.

45. *Bochare, A.* Camera-Only Bird's Eye View Perception: A Neural Approach to LiDAR-Free Environmental Mapping for Autonomous Vehicles [Электронный ресурс] / A. Bochare. — 2025. — arXiv: 2505.06113 [cs.CV]. — URL: https://arxiv.org/abs/2505.06113 (дата обр. 08.09.2026).

46. Radar/lidar sensor fusion for car-following on highways [Текст] / D. Göhring [и др.] // The 5th International Conference on Automation, Robotics and Applications. — IEEE. 2011. — С. 407—412.

47. Bi-lrfusion: Bi-directional lidar-radar fusion for 3d dynamic object detection [Текст] / Y. Wang [и др.] // Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. — 2023. — С. 13394—13403.

48. *Song, J.* Lirafusion: Deep adaptive lidar-radar fusion for 3d object detection [Текст] / J. Song, L. Zhao, K. A. Skinner // 2024 IEEE International Conference on Robotics and Automation (ICRA). — IEEE. 2024. — С. 18250—18257.

49. Lidar-based glass detection for improved occupancy grid mapping [Текст] / H. Tibebu [и др.] // Sensors. — 2021. — Т. 21, № 7. — С. 2263.

50. LIDAR–camera fusion for road detection using fully convolutional neural networks [Текст] / L. Caltagirone [и др.] // Robotics and Autonomous Systems. — 2019. — Т. 111. — С. 125—131.

51. *Liu, H.* Real time object detection using LiDAR and camera fusion for autonomous driving [Текст] / H. Liu, C. Wu, H. Wang // Scientific Reports. — 2023. — Т. 13, № 1. — С. 8056.

52. Radar-camera fusion for object detection and semantic segmentation in autonomous driving: A comprehensive review [Текст] / S. Yao [и др.] // IEEE Transactions on Intelligent Vehicles. — 2023. — Т. 9, № 1. — С. 2094—2128.

53. *Nabati, R.* Centerfusion: Center-based radar and camera fusion for 3d object detection [Текст] / R. Nabati, H. Qi // Proceedings of the IEEE/CVF winter conference on applications of computer vision. — 2021. — С. 1527—1536.

54. *Hasanujjaman, M.* Sensor Fusion in Autonomous Vehicle with Traffic Surveillance Camera System: Detection, Localization, and AI Networking [Текст] / M. Hasanujjaman, M. Z. Chowdhury, Y. M. Jang // Sensors. — 2023. — Т. 23, № 6. — URL: https://www.mdpi.com/1424-8220/23/6/3335.

55. Sensor and Sensor Fusion Technology in Autonomous Vehicles: A Review [Текст] / D. J. Yeong [и др.] // Sensors. — 2021. — Т. 21, № 6. — URL: https://www.mdpi.com/1424-8220/21/6/2140.

56. Obstacle detection quality as a problem-oriented approach to stereo vision algorithms estimation in road situation analysis [Текст] / A. Smagina [и др.] // Journal of Physics: Conference Series. Т. 1096. — IOP Publishing. 2018. — С. 012035.

57. Deep learning on monocular object pose detection and tracking: A comprehensive overview [Текст] / Z. Fan [и др.] // ACM Computing Surveys. — 2022. — Т. 55, № 4. — С. 1—40.

58. Manipulator grabbing position detection with information fusion of color image and depth image using deep learning [Текст] / D. Jiang [и др.] // Journal of Ambient Intelligence and Humanized Computing. — 2021. — Т. 12, № 12. — С. 10809—10822.

59. Multisensor data fusion: A review of the state-of-the-art [Текст] / B. Khaleghi [и др.] // Information Fusion. — 2013. — Янв. — Т. 14, № 1. — С. 28—44. — URL: http://dx.doi.org/10.1016/j.inffus.2011.08.001.

60. *Sasiadek, J. Z.* Sensor fusion [Текст] / J. Z. Sasiadek // Annual Reviews in Control. — 2002. — Т. 26, № 2. — С. 203—228.

61. *Duong, T.* Autonomous navigation in unknown environments with sparse bayesian kernel-based occupancy mapping [Текст] / T. Duong, M. Yip, N. Atanasov // IEEE Transactions on Robotics. — 2022. — Т. 38, № 6. — С. 3694—3712.

62. A Fusion of Dynamic Occupancy Grid Mapping and Multi-object Tracking Based on Lidar and Camera Sensors [Текст] / Y. Wang [и др.] // 2020 3rd International Conference on Unmanned Systems (ICUS). — 2020. — С. 107—112.

63. *Швец, Е. А.* Восстановление карты проходимости с использованием прямой модели сонаров методом градиентного спуска [Текст] / Е. А. Швец, Д. А. Шепелев, Д. П. Николаев // Информационные процессы. — 2016. — Т. 16, № 1. — С. 61—71.

64. BEVFusion: Multi-Task Multi-Sensor Fusion with Unified Bird's-Eye View Representation [Электронный ресурс] / Z. Liu [и др.]. — 2024. — arXiv: 2205.13542 [cs.CV]. — URL: https://arxiv.org/abs/2205.13542 (дата обр. 08.09.2026).

65. *Durrant-Whyte, H. F.* Sensor models and multisensor integration [Текст] / H. F. Durrant-Whyte // The International Journal of Robotics Research. — 1988. — Т. 7, № 6. — С. 97—113.

66. A real-time photogrammetric algorithm for sensor and synthetic image fusion with application to aviation combined vision [Текст] / M. A. Lebedev [и др.] // The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences. — 2014. — Авг. — Т. XL—3. — С. 171—175. — URL: http://dx.doi.org/10.5194/isprsarchives-XL-3-171-2014.

67. *Jaccard, P.* Distribution de la flore alpine dans le bassin des Dranses et dans quelques régions voisines [Текст] / P. Jaccard // Bulletin de la Société Vaudoise des Sciences Naturelles. — 1901. — Т. 37, № 140. — С. 241—272.

68. The Pascal Visual Object Classes (VOC) challenge [Текст] / M. Everingham [и др.] // International Journal of Computer Vision. — 2010. — Т. 88, № 2. — С. 303—338.

69. *Bernardin, K.* Evaluating multiple object tracking performance: the clear mot metrics [Текст] / K. Bernardin, R. Stiefelhagen // EURASIP Journal on Image and Video Processing. — 2008. — Т. 2008, № 1. — С. 246309.

70. Performance measures and a data set for multi-target, multi-camera tracking [Текст] / E. Ristani [и др.] // European conference on computer vision. — Springer. 2016. — С. 17—35.

71. Hota: A higher order metric for evaluating multi-object tracking [Текст] / J. Luiten [и др.] // International journal of computer vision. — 2021. — Т. 129, № 2. — С. 548—578.

72. Grid-Centric Traffic Scenario Perception for Autonomous Driving: A Comprehensive Review [Текст] / Y. Shi [и др.] // IEEE Transactions on Neural Networks and Learning Systems. — 2025. — Т. 36, № 7. — С. 11814—11834.

73. UniOcc: A Unified Benchmark for Occupancy Forecasting and Prediction in Autonomous Driving [Электронный ресурс] / Y. Wang [и др.]. — 2025. — arXiv: 2503.24381 [cs.CV]. — URL: https://arxiv.org/abs/2503.24381 (дата обр. 08.09.2026).

74. *Matsushita, S.* Machine learning-based object movement prediction method using occupancy grid maps from roadside sensor [Текст] / S. Matsushita, O. Alparslan, K. Sato // IEICE Communications Express. — 2025. — Т. 14, № 5. — С. 189—192.

75. Fiery: Future instance prediction in bird's-eye view from surround monocular cameras [Текст] / A. Hu [и др.] // Proceedings of the IEEE/CVF International Conference on Computer Vision. — 2021. — С. 15273—15282.

76. Video panoptic segmentation [Текст] / D. Kim [и др.] // Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. — 2020. — С. 9859—9868.

77. A navigation-based evaluation metric for probabilistic occupancy grids: Pathfinding cost mean squared error [Текст] / J.-B. Horel [и др.] // 2023 IEEE 26th International Conference on Intelligent Transportation Systems (ITSC). — IEEE. 2023. — С. 3148—3153.

78. *Dijkstra, E. W.* A note on two problems in connexion with graphs [Текст] / E. W. Dijkstra // Numerische Mathematik. — 1959. — Т. 1, № 1. — С. 269—271.

79. *Li, P. Z. X.* GMMap: Memory-Efficient Continuous Occupancy Map Using Gaussian Mixture Model [Текст] / P. Z. X. Li, S. Karaman, V. Sze // IEEE Transactions on Robotics. — 2024. — Т. 40. — С. 1339—1355. — URL: http://dx.doi.org/10.1109/TRO.2023.3348305.

80. *O'Callaghan, S.* Continuous occupancy mapping with integral kernels [Текст] / S. O'Callaghan, F. Ramos // Proceedings of the AAAI conference on artificial intelligence. Т. 25. — 2011. — С. 1494—1500.

81. *Vido, C. E.* From grids to continuous occupancy maps through area kernels [Текст] / C. E. Vido, F. Ramos // 2016 IEEE International Conference on Robotics and Automation (ICRA). — 2016. — С. 1043—1048.

82. *Fischler, M. A.* Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography [Текст] / M. A. Fischler, R. C. Bolles // Commun. ACM. — New York, NY, USA, 1981. — Июнь. — Т. 24, № 6. — С. 381—395. — URL: https://doi.org/10.1145/358669.358692.

83. *Bertozzi, M.* GOLD: a parallel real-time stereo vision system for generic obstacle and lane detection [Текст] / M. Bertozzi, A. Broggi // IEEE Transactions on Image Processing. — 1998. — Т. 7, № 1. — С. 62—81.

84. *СП 34.13330.2021*. Автомобильные дороги. СНиП 2.05.02-85* [Текст]. — Утв. приказом Минстроя России от 09.02.2021 № 53/пр; применяется с 08.04.2021 взамен СП 34.13330.2012; с изм. № 1, утв. приказом Минстроя России от 27.12.2024 № 947/пр. — М. : Минстрой России, 2021. — (Свод правил).

85. *Himmelsbach, M.* Fast segmentation of 3D point clouds for ground vehicles [Текст] / M. Himmelsbach, F. v. Hundelshausen, H.-J. Wuensche // 2010 IEEE Intelligent Vehicles Symposium. — IEEE, 06.2010. — С. 560—565. — URL: http://dx.doi.org/10.1109/IVS.2010.5548059.

86. *Zermas, D.* Fast segmentation of 3D point clouds: A paradigm on LiDAR data for autonomous vehicle applications [Текст] / D. Zermas, I. Izzat, N. Papanikolopoulos // 2017 IEEE International Conference on Robotics and Automation (ICRA). — IEEE, 05.2017. — С. 5067—5073. — URL: http://dx.doi.org/10.1109/ICRA.2017.7989591.

87. *Lim, H.* Patchwork: Concentric Zone-Based Region-Wise Ground Segmentation With Ground Likelihood Estimation Using a 3D LiDAR Sensor [Текст] / H. Lim, M. Oh, H. Myung // IEEE Robotics and Automation Letters. — 2021. — Окт. — Т. 6, № 4. — С. 6458—6465. — URL: http://dx.doi.org/10.1109/LRA.2021.3093009.

88. *Rasmussen, C. E.* Gaussian processes in machine learning [Текст] / C. E. Rasmussen // Summer school on machine learning. — Springer, 2003. — С. 63—71.

89. Gaussian-Process-Based Real-Time Ground Segmentation for Autonomous Land Vehicles [Текст] / T. Chen [и др.] // Journal of Intelligent & Robotic Systems. — 2013. — Т. 76, № 3/4. — С. 563—582. — URL: http://dx.doi.org/10.1007/s10846-013-9889-4.

90. *Philion, J.* Lift, Splat, Shoot: Encoding Images from Arbitrary Camera Rigs by Implicitly Unprojecting to 3D [Текст] / J. Philion, S. Fidler // Computer Vision – ECCV 2020. — Springer International Publishing, 2020. — С. 194—210. — URL: http://dx.doi.org/10.1007/978-3-030-58568-6_12.

91. BEVFormer: Learning Bird's-Eye-View Representation from Multi-camera Images via Spatiotemporal Transformers [Текст] / Z. Li [и др.] // Computer Vision – ECCV 2022. — Springer Nature Switzerland, 2022. — С. 1—18. — URL: http://dx.doi.org/10.1007/978-3-031-20077-9_1.

92. Bevfusion: Multi-task multi-sensor fusion with unified bird's-eye view representation [Текст] / Z. Liu [и др.] // 2023 IEEE international conference on robotics and automation (ICRA). — IEEE. 2023. — С. 2774—2781.

93. Tri-perspective view for vision-based 3D semantic occupancy prediction [Текст] / Y. Huang [и др.] // 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). — 2023. — С. 9223—9232.

94. Scene as occupancy [Текст] / W. Tong [и др.] // 2023 IEEE/CVF International Conference on Computer Vision (ICCV). — 2023. — С. 8372—8381.

95. *Petrovskaya, A.* Model based vehicle tracking for autonomous driving in urban environments [Текст] / A. Petrovskaya, S. Thrun // Proceedings of robotics: science and systems IV, Zurich, Switzerland. — 2008. — Т. 34.

96. *Arnon, D. S.* A linear time algorithm for the minimum area rectangle enclosing a convex polygon [Текст] / D. S. Arnon, J. P. Gieselmann. — 1983.

97. Optimal Vehicle Pose Estimation Network Based on Time Series and Spatial Tightness with 3D LiDARs [Текст] / H. Wang [и др.] // Remote Sensing. — 2021. — Окт. — Т. 13. — С. 4123.

98. Efficient L-shape fitting for vehicle detection using laser scanners [Текст] / X. Zhang [и др.] // 2017 IEEE Intelligent Vehicles Symposium (IV). — IEEE. 2017. — С. 54—59.

99. The Marathon 2: A Navigation System [Текст] / S. Macenski [и др.] // 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). — Las Vegas, NV, USA : IEEE, 10.2020.

100. *Fankhauser, P.* A Universal Grid Map Library: Implementation and Use Case for Rough Terrain Navigation [Текст] / P. Fankhauser, M. Hutter // Robot Operating System (ROS). — Springer International Publishing, 2016. — С. 99—120. — URL: http://dx.doi.org/10.1007/978-3-319-26054-9_5.

101. OctoMap: an efficient probabilistic 3D mapping framework based on octrees [Текст] / A. Hornung [и др.] // Autonomous Robots. — 2013. — Февр. — Т. 34, № 3. — С. 189—206. — URL: http://dx.doi.org/10.1007/s10514-012-9321-0.

102. Efficient occupancy grid computation on the GPU with lidar and radar for road boundary detection [Текст] / F. Homm [и др.] // 2010 IEEE Intelligent Vehicles Symposium. — IEEE, 06.2010. — С. 1006—1013. — URL: http://dx.doi.org/10.1109/IVS.2010.5548091.

103. *Michael, M. M.* Simple, fast, and practical non-blocking and blocking concurrent queue algorithms [Текст] / M. M. Michael, M. L. Scott // Proceedings of the fifteenth annual ACM symposium on Principles of distributed computing - PODC '96. — ACM Press, 1996. — С. 267—275. — (PODC '96). — URL: http://dx.doi.org/10.1145/248052.248106.

104. *Herlihy, M.* The Art of Multiprocessor Programming [Текст] / M. Herlihy, N. Shavit. — Morgan Kaufmann, 04.2008.

105. Yolop: You only look once for panoptic driving perception [Текст] / D. Wu [и др.] // Machine Intelligence Research. — 2022. — Т. 19, № 6. — С. 550—562.

106. Object Detection in Autonomous Driving Scenarios Based on an Improved Faster-RCNN [Текст] / Y. Zhou [и др.] // Applied Sciences. — 2021. — Т. 11, № 24. — URL: https://www.mdpi.com/2076-3417/11/24/11630.

107. *Kuhn, H. W.* The Hungarian method for the assignment problem [Текст] / H. W. Kuhn // Naval research logistics quarterly. — 1955. — Т. 2, № 1/2. — С. 83—97.

108. *ГОСТ 21.701-2013*. Система проектной документации для строительства. Правила выполнения рабочей документации автомобильных дорог [Текст]. — Введ. 01.01.2015. — М. : Стандартинформ, 2014. — 32 с. — (Межгосударственный стандарт).

109. *NVIDIA*. CUDA, release: 10.2.89 [Электронный ресурс] / NVIDIA, P. Vingelmann, F. H. Fitzek. — 2020. — URL: https://developer.nvidia.com/cuda-toolkit (дата обр. 08.09.2026).

110. Fast euclidean cluster extraction using gpus [Текст] / A. Nguyen [и др.] // Journal of Robotics and Mechatronics. — 2020. — Т. 32, № 3. — С. 548—560.

111. *Stein, M. L.* Interpolation of spatial data: some theory for kriging [Текст] / M. L. Stein. — Springer Science & Business Media, 1999.

112. *Jones, A.* The matérn class of covariance functions [Электронный ресурс] / A. Jones. — 07.2021. — URL: https://andrewcharlesjones.github.io/journal/matern-kernels.html (дата обр. 15.04.2025).

113. ROS: an open-source Robot Operating System [Текст] / M. Quigley [и др.] // ICRA workshop on open source software. Т. 3. — Kobe. 2009. — С. 5.

114. *Kyöstilä, S.* Tracy: A debugger and system analyzer for cross-platform graphics development [Текст] / S. Kyöstilä, K. J. Kangas, K. Pulli // Proceedings of the 23rd ACM SIGGRAPH/EUROGRAPHICS symposium on Graphics hardware. — 2008. — С. 1—11.

115. *Moller, K.* Overcoming Blind Spots: Occlusion Considerations for Improved Autonomous Driving Safety [Текст] / K. Moller, R. Trauth, J. Betz // 2024 IEEE Intelligent Vehicles Symposium (IV). — 2024. — С. 819—826.

116. *Rennich, S.* Cuda C/C++ streams and concurrency [Электронный ресурс] / S. Rennich. — URL: https://developer.download.nvidia.com/CUDA/training/StreamsAndConcurrencyWebinar.pdf (дата обр. 17.08.2025).

117. *NVIDIA*. NVIDIA Nsight Systems: User Guide [Электронный ресурс] / NVIDIA. — 08.2025. — URL: https://docs.nvidia.com/nsight-systems/index.html (дата обр. 17.08.2025).

118. *Bradski, G.* The OpenCV Library [Текст] / G. Bradski // Dr. Dobb's Journal of Software Tools. — 2000.

119. *Rusu, R. B.* 3D is here: Point Cloud Library (PCL) [Текст] / R. B. Rusu, S. Cousins // IEEE International Conference on Robotics and Automation (ICRA). — Shanghai, China : IEEE, 05.2011.

120. *Woodall, W.* pcl-conversions [Электронный ресурс] / W. Woodall. — URL: https://wiki.ros.org/pcl_conversions (дата обр. 17.08.2025).

121. *Martin, R. C.* The dependency inversion principle [Текст] / R. C. Martin // C Report. — 1996. — Т. 8, № 6. — С. 61—66.

122. *Marder-Eppstein, E.* tf2-ros [Электронный ресурс] / E. Marder-Eppstein, W. Meeussen. — URL: https://wiki.ros.org/tf2_ros (дата обр. 17.08.2025).

123. Head First Design Patterns: A Brain-Friendly Guide [Текст] / E. Freeman [и др.]. — ” O’Reilly Media, Inc.”, 2004.

124. *Fog, A.* Lists of instruction latencies, throughputs and micro-operation breakdowns for Intel, AMD and VIA CPUs [Текст] / A. Fog // Agner Fog Homepage. — 2014.

125. *Google*. Google Benchmark: a microbenchmark support library [Электронный ресурс] / Google. — 2025. — URL: https://github.com/google/benchmark (дата обр. 08.09.2026).

Список работ, опубликованных автором по теме диссертации

В изданиях из списка ВАК РФ

126. *Чеканов, М. О.* Трехмерная детекция объектов на основе L-shape модели в автономных системах движения [Текст] / М. О. Чеканов, С. В. Кудряшова, О. С. Шипитько // Сенсорные системы. — 2025. — Т. 39, № 1. — С. 66—78.

127. *Воронков, А. Д.* Алгоритм проекции семантической сегментации проезжей части на карту проходимости пространства с использованием лидарной оценки геометрии поверхности [Текст] / А. Д. Воронков, А. П. Рудоманенко, М. О. Чеканов // Информационные процессы. — 2026. — Т. 26, № 2. — С. 511—530.

128. *Чеканов, М. О.* Многопоточные неблокирующие алгоритмы построения карты проходимости пространства [Текст] / М. О. Чеканов // Информационные технологии и вычислительные системы. — 2026. — № 2. — С. 40—51.

В изданиях, индексируемых Web of Science и Scopus

129. *Chekanov, M.* L-shape fitting algorithm for 3D object detection in bird's-eye-view in an autonomous driving system [Текст] / M. Chekanov, O. Shipitko // Sixteenth International Conference on Machine Vision (ICMV 2023). Т. 13072 / под ред. W. Osten. — SPIE, 04.2024. — С. 40. — (Proceedings of SPIE). — URL: http://dx.doi.org/10.1117/12.3023477.

130. *Chekanov, M.* Camera and lidar sensor fusion for projection of 2d detections into 3d coordinate space [Текст] / M. Chekanov, O. Shipitko, D. Nikolaev // ECMS 2026 Proceedings edited by Filippo Sanfilippo, Florenc Demrozi, Fabio Sgarbossa, Mohammad Poursina. — ECMS, 06.2026. — С. 259—266. — URL: http://dx.doi.org/10.7148/2026-0259.

Список рисунков

1.1 Пример эталонной карты и предсказанных карт с одинаковым качеством согласно метрикам, не учитывающим контекст решаемой задачи . . . 30

2.1 Наивный подход к проецированию ограничивающей рамки в 2D на вид сверху . . . 47
2.2 Неверная проекция грузовика. Зона слева от грузовика воспринимается, как часть объекта, и мешает объехать его . . . 47
2.3 Проекция двумерной ограничивающей рамки на вид сверху с учётом сегментации проезжей части . . . 48
2.4 Пример неправильно построенной ограничивающей рамки . . . 51
2.5 Примеры сцен из набора данных, использованного для сравнения алгоритмов . . . 53
2.6 Примеры работы сравниваемых алгоритмов построения ограничивающих рамок . . . 54
2.7 График зависимости среднего коэффициента Жаккара от ограничения дистанции до детектируемых препятствий . . . 55
2.8 Примеры оформления поперечного профиля из ГОСТ 21.701-2013 [108] 56
2.9 Демонстрация работы алгоритма 1 на данных, полученных с БТС . . . 59
2.10 Сравнение регрессии на основе гауссовских процессов с использованием в качестве ядра радиальной базисной функции и ядра Матерна [112] . . . 60
2.11 Пример работы алгоритма разделения облака точек земли и препятствий . . . 61
2.12 График зависимости среднего коэффициента Жаккара от ограничения дистанции до детектируемых препятствий для алгоритма комплексирования и алгоритмов, не использующих комплексирование 65
2.13 Среднее время выполнения этапов алгоритма 1 . . . 66
2.14 Демонстрация влияния окклюзии на безопасность движения БТС [115] 67
2.15 Пример семантической сегментации проезжей части моделью YOLOP на изображении с камеры БТС . . . 67
2.16 Пример заслонённой области свободной для проезда . . . 68

2.17 Иллюстрация алгоритма проецирования сегментации проезжей части на вид сверху. Для облегчения восприятия проецируемая сетка представлена одним рядом 70
2.18 Профилирование в Nsight Systems изменившейся части программы, реализующей алгоритм 2, при добавлении тестовых точек, отвечающих центрам элементов сетки 72
2.19 График полного времени работы алгоритма 2 с дополнительными тестовыми точками при последовательном и параллельном вычислении 73
2.20 Эталонные карты проходимости пространства и карты, построенные сравниваемыми алгоритмами на тестовых данных 79

3.1 Диаграмма компонентов программного комплекса Occupancy Map Fusion на языке UML(Unified Modeling Language) 86
3.2 Диаграмма деятельности программного комплекса Occupancy Map Fusion 87
3.3 Иллюстрация поля зрения, занимаемого препятствием 90

4.1 Наблюдаемые задержки при обновлении карты проходимости пространства 103
4.2 Расположение локальной карты проходимости и её фрагмента от одного из источников данных относительно БТС и глобальной системы координат 104
4.3 Демонстрация формирования очереди фрагментов. Горизонтальные отрезки обозначают время формирования фрагмента каждым источником: начало отрезка соответствует времени регистрации данных $t_{s,k}$, конец отрезка соответствует времени завершения формирования фрагмента и моменту добавления его в очередь фрагментов. Внизу иллюстрации продемонстрирован итоговый порядок фрагментов в очереди: фрагменты извлекаются из очереди в порядке справа налево 105
4.4 Иллюстрация алгоритма добавления фрагмента от источника данных s на локальную карту проходимости пространства 107
4.5 Оригинальная реализация локальной карты проходимости пространства 109
4.6 Карта с предподготовленными окрестностями 110
4.7 Карта с полным обновлением без блокировок 114

4.8 Сдвиг карты проходимости пространства с полным обновлением без блокировок 115
4.9 Графики зависимости времени выполнения операций над локальной картой проходимости пространства от размера карты 119
4.10 Графики зависимости времени добавления фрагмента на карту от размера карты при 1, 5, 9 и 13 потоках 120
4.11 Графики зависимости времени получения карты и времени одной итерации «публикации» карты от размера карты 121
4.12 Двумерная реализация локальной карты проходимости пространства с полным обновлением без блокировок 123

Список таблиц

1 Сравнение характеристик датчиков, используемых в колёсной робототехнике. Приведены типичные для колёсной робототехники значения, обобщающие описание датчиков в разделах 1.2 и источниках [15; 18; 19] 19

2 Изменение предельного числа одновременно учитываемых источников данных с ростом рабочей скорости БТС 42

3 Алгоритмы вписывания полигона в ориентированную ограничивающую рамку из работы Ванга [97], адаптированные под проекции на вид сверху 52

4 Статистики полученных коэффициента Жаккара для сравниваемых алгоритмов. Наилучшие значения выделены полужирным 55

5 Статистики полученных коэффициентов Жаккара для предложенного алгоритма и алгоритмов, не использующих комплексирование. Наилучшие значения выделены полужирным 64

6 Сценарии, представленные в наборе тестовых данных 75

7 Значения метрики PFC-MSE (меньше – лучше) для предложенного и базового лидарного алгоритмов. Наилучшие значения выделены полужирным 77

8 Параметры сравниваемых алгоритмов Б, П, А1, А2 (обозначения введены выше) и усреднённые по пяти сценариям значения метрик. Стрелка ↓ означает, что меньшие значения метрики соответствуют лучшему качеству, ↑ – большие. Наилучшие значения по каждой метрике выделены полужирным 78

9 Требования к программному комплексу и реализующие их архитектурные решения 83

10 Вычислительная сложность операций над локальной картой проходимости пространства для базовой реализации и трёх предложенных модификаций. Для операций, допускающих одновременное выполнение несколькими потоками, сложность указана для одного вызова 117

Приложение А

Вывод решения задачи вписывания точек в ориентированную ограничивающую рамку с использованием L-Shape модели

В разделе 2.1 задача вписывания набора точек $p_i = (x_i, y_i)$, $i \in \{1, \ldots, N_p\}$, в ориентированную ограничивающую рамку сведена к поиску двух перпендикулярных прямых $l : y = kx + d_1$ и $l' : x = -ky + d_2$, минимизирующих сумму квадратов отклонений вписываемых точек от соответствующей прямой. Согласно L-Shape модели, существует номер m, разделяющий точки между прямыми:

$$\begin{aligned} \exists m \in \{1, \ldots, N_p - 1\} \hookrightarrow & \forall i \in \{1, \ldots, m\}, p_i = (x_i, y_i) \in l; \\ & \forall i \in \{m+1, \ldots, N_p\}, p_i = (x_i, y_i) \in l'. \end{aligned} \tag{A.1}$$

При фиксированном m задача минимизации записывается в виде:

$$\underset{k,d_1,d_2}{\operatorname{argmin}} \left(\begin{pmatrix} x_1 & 1 & 0 \\ x_2 & 1 & 0 \\ \vdots & \vdots & \vdots \\ x_m & 1 & 0 \\ -y_{m+1} & 0 & 1 \\ \vdots & \vdots & \vdots \\ -y_{N_p} & 0 & 1 \end{pmatrix} \begin{pmatrix} k \\ d_1 \\ d_2 \end{pmatrix} - \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \\ x_{m+1} \\ \vdots \\ x_{N_p} \end{pmatrix} \right)^2 \tag{A.2}$$

Обозначим через $\widehat{x}$ и $\widetilde{y}$ столбцы координат точек, отнесённых к прямой l, а через $\widehat{y}$ и $\widetilde{x}$ – столбцы координат точек, отнесённых к прямой l'. Тогда выражение переписывается как

$$\underset{a}{\operatorname{argmin}} \left(\begin{pmatrix} \widehat{x} & 1 & 0 \\ -\widehat{y} & 0 & 1 \end{pmatrix} a - \begin{pmatrix} \widetilde{y} \\ \widetilde{x} \end{pmatrix} \right)^2 = \underset{a}{\operatorname{argmin}} \left(Xa - y \right)^2, \tag{A.3}$$

где $a = (k, d_1, d_2)^T$. Решение этой задачи и соответствующее ему значение суммы квадратов отклонений даются формулами (2.1) и (2.2) основного текста.

Покажем, что при переходе от m к $m+1$ входящие в эти формулы матрицы пересчитываются за фиксированное, не зависящее от N_p, число операций. Для удобства чтения введём обозначение $\overline{m} = N_p - m$. Выпишем промежуточные матрицы в явном виде:

$$
\begin{aligned}
X^T X &= \begin{pmatrix} \sum \widehat{x}^2 + \sum \widehat{y}^2 & \sum \widehat{x} & -\sum \widehat{y} \\ \sum \widehat{x} & m & 0 \\ -\sum \widehat{y} & 0 & \overline{m} \end{pmatrix} \\
\Delta_{X^T X} &= m\overline{m}(\sum \widehat{x}^2 + \sum \widehat{y}^2) - m\left(\sum \widehat{y}\right)^2 - \overline{m}\left(\sum \widehat{x}\right)^2 \\
\left(X^T X\right)^{-1} &= \frac{1}{\Delta_{X^T X}} \\
&\begin{pmatrix} m\overline{m} & -\overline{m}\sum \widehat{x} & m\sum \widehat{y} \\ -\overline{m}\sum \widehat{x} & \overline{m}(\sum \widehat{x}^2 + \sum \widehat{y}^2) - (\sum \widehat{y})^2 & -\sum \widehat{x}\sum \widehat{y} \\ m\sum \widehat{y} & -\sum \widehat{x}\sum \widehat{y} & m(\sum \widehat{x}^2 + \sum \widehat{y}^2) - (\sum \widehat{x})^2 \end{pmatrix} \\
X^T y &= \begin{pmatrix} \widehat{x}^T\widetilde{y} - \widehat{y}^T\widetilde{x} \\ \sum \widetilde{y} \\ \sum \widetilde{x} \end{pmatrix}
\end{aligned}
\tag{A.4}
$$

Все элементы этих матриц выражаются через семь величин: $\sum \widehat{x}^2 + \sum \widehat{y}^2$, $\sum \widehat{x}$, $\sum \widehat{y}$, $\sum \widetilde{x}$, $\sum \widetilde{y}$, $\widehat{x}^T\widetilde{y}$, $\widehat{y}^T\widetilde{x}$, а также через m и $\overline{m}$. Если вычислить их один раз для $m = 1$ (за $\Theta(N_p)$ операций), то при «перемещении» очередной точки (x,y) с прямой l' на прямую l перечисленные величины пересчитываются так:

$$
\begin{aligned}
\sum \widehat{x}^2 + \sum \widehat{y}^2 + &= x^2 - y^2 \\
\sum \widehat{x} + &= x \\
\sum \widehat{y} - &= y \\
\sum \widetilde{x} - &= x \\
\sum \widetilde{y} + &= y \\
\widehat{x}^T\widetilde{y} + &= xy \\
\widehat{y}^T\widetilde{x} - &= xy
\end{aligned}
\tag{A.5}
$$

Таким образом, каждый шаг перебора m требует фиксированного числа операций, и итоговая временная сложность L-Shape алгоритма составляет $\Theta(N_p) + (N_p - 1)\Theta(1) = \Theta(N_p)$ вместо $\Theta(N_p^2)$ у прямой реализации.

Приложение Б

Свидетельства о государственной регистрации программ для ЭВМ

РОССИЙСКАЯ ФЕДЕРАЦИЯ

СВИДЕТЕЛЬСТВО

о государственной регистрации программы для ЭВМ

№ 2025617212

Программное обеспечение «Система восприятия N1 V3»

Правообладатель: ***Общество с ограниченной ответственностью «ЭвоКарго» (RU)***

Авторы: ***см. на обороте***

Заявка № **2025614166**
Дата поступления **03 марта 2025 г.**
Дата государственной регистрации
в Реестре программ для ЭВМ ***24 марта 2025 г.***

Руководитель Федеральной службы
по интеллектуальной собственности

ДОКУМЕНТ ПОДПИСАН ЭЛЕКТРОННОЙ ПОДПИСЬЮ
Сертификат 0692e7c1a6300bf54f240f670bcca2026
Владелец **Зубов Юрий Сергеевич**
Действителен с 10.07.2024 по 03.10.2025

Ю.С. Зубов

Авторы: ***Бабатаев Ильдар Хамзаевич (RU), Бойко Дмитрий Викторович (RU), Боровикова Полина Александровна (RU), Валюков Александр Дмитриевич (RU), Зайцева Елена Александровна (RU), Калинин Валерий Геннадьевич (RU), Козловцев Константин Анатольевич (RU), Комиссарова Полина Александровна (RU), Ларин Александр Витальевич (RU), Лысухин Даниил Дмитриевич (RU), Мозжухин Евгений Игоревич (RU), Осиповский Роман Владиславович (RU), Погодина Кристина Валерьевна (RU), Протасов Саян Константинович (RU), Прошкина Екатерина Юрьевна (RU), Рудоманенко Антон Павлович (RU), Скоробогатова Ольга Александровна (RU), Скрипицин Сергей Анатольевич (RU), Скублов Алексей Сергеевич (RU), Фокин Иван Игоревич (RU), Чалков Артём Анатольевич (RU), Чеканов Михаил Олегович (RU)***

РОССИЙСКАЯ ФЕДЕРАЦИЯ

СВИДЕТЕЛЬСТВО

о государственной регистрации программы для ЭВМ

№ 2025669744

Программное обеспечение «Способ построения карты проходимости в окрестности высокоавтоматизированного беспилотного грузового транспортного средства»

Правообладатель: ***Общество с ограниченной ответственностью «ЭвоКарго» (RU)***

Авторы: ***Чеканов Михаил Олегович (RU), Рудоманенко Антон Павлович (RU), Садовсков Кирилл Викторович (RU)***

Заявка № **2025668746**

Дата поступления **25 июня 2025 г.**

Дата государственной регистрации в Реестре программ для ЭВМ ***30 июля 2025 г.***

Руководитель Федеральной службы по интеллектуальной собственности

ДОКУМЕНТ ПОДПИСАН ЭЛЕКТРОННОЙ ПОДПИСЬЮ
Сертификат 0692e7c1a6300bf54f240f670bcca2026
Владелец **Зубов Юрий Сергеевич**
Действителен с 10.07.2024 по 03.10.2025

Ю.С. Зубов

Приложение В

Патенты

РОССИЙСКАЯ ФЕДЕРАЦИЯ

ПАТЕНТ

НА ИЗОБРЕТЕНИЕ

№ 2853062

Способ построения карты проходимости в окрестности высокоавтоматизированного беспилотного грузового транспортного средства

Патентообладатель: ***Общество с ограниченной ответственностью "Эвокарго" (RU)***

Авторы: ***Чеканов Михаил Олегович (RU), Садовсков Кирилл Викторович (RU), Рудоманенко Антон Павлович (RU)***

Заявка № **2025112125**
Приоритет изобретения **12 мая 2025 г.**
Дата государственной регистрации
в Государственном реестре изобретений
Российской Федерации **18 декабря 2025 г.**
Срок действия исключительного права
на изобретение истекает **12 мая 2045 г.**

Руководитель Федеральной службы
по интеллектуальной собственности

ДОКУМЕНТ ПОДПИСАН ЭЛЕКТРОННОЙ ПОДПИСЬЮ
Сертификат 00a570e4f7add8d531b4b8818e75f29506
Владелец **Зубов Юрий Сергеевич**
Действителен с 04.09.2025 по 28.11.2026

Ю.С. Зубов

Приложение Г

Акты о внедрении результатов диссертационного исследования

EVOCARGO

ООО «ЭВОКАРГО»
ИНН/КПП: 7714459603/771701001
ОГРН 1207700132804
Расчетный счет: 40702810038000257848
Банк: ПАО СБЕРБАНК БИК: 044525225
Корр. счет: 30101810400000000225

Адрес: 129085, г. Москва г, ул. Годовикова,
д. 9, стр. 4 Под.4.15, этаж 3, помещение. 3.9
тел.: +7 (495) 926 26 67
e-mail: info@evocargo.com

Акт о внедрении результатов диссертационного исследования

Данный акт подтверждает, что результаты исследовательской работы, полученные Чекановым М.О. и приведенные в диссертации «Алгоритмы комплексирования разнородных данных при построении модели окружающей сцены беспилотного транспортного средства», представленной на соискание ученой степени кандидата технических наук по специальности 2.3.8 Информатика и информационные процессы, были внедрены в продукт «тележка высокоавтоматизированная электрическая торговой марки EVOCARGO моделей: ВАТС ЭВО 1.1, ВАТС ЭВО-1.2.», предназначенную для оказания услуг автономной логистики (перемещение материально-технических ресурсов).

Генеральный директор
ООО «ЭвоКарго» М.И. Нигметзянов

ООО «ЭвоКарго» «EvoCargo» LLC